

AI-Based Credit Card Fraud Detection System Using Machine Learning

TABLE OF CONTENTS

1. Introduction
2. Background of the Study
3. Problem Statement
4. Objectives of the Project
5. Scope of the Project
6. Literature Survey
7. Existing System
8. Proposed System
9. System Architecture
10. Technology Stack
11. System Design
12. Database Design
13. Machine Learning Model
14. System Modules
15. API Design
16. Implementation Details
17. Testing and Validation
18. Results and Discussion
19. Conclusion
20. Future Scope
21. References

1. Introduction

With the rapid growth of digital payments and online transactions, credit card fraud has become a major concern worldwide. Financial institutions and payment gateways must continuously monitor transaction activities to detect fraudulent behavior and protect users from financial losses.

Machine Learning has emerged as a powerful solution for detecting fraud patterns in large volumes of transaction data. By analyzing historical transaction patterns, machine learning models can identify anomalies and suspicious activities in real time.

This project presents an AI-Based Credit Card Fraud Detection System that uses a Random Forest Machine Learning algorithm to classify transactions into different risk categories such as Safe, Suspicious, and Fraudulent.

2. Background of the Study

The rapid adoption of digital payment systems such as credit cards, online shopping platforms, and Unified Payments Interface (UPI) has significantly increased the number of financial transactions in India. While these technologies provide convenience and efficiency, they also increase the risk of financial fraud.

Machine Learning techniques provide a powerful alternative by learning patterns from historical data and detecting unusual transaction behavior automatically.

3. Problem Statement

Credit card fraud is a growing concern in the financial industry. Fraudulent transactions can cause financial losses for both consumers and financial institutions. Detecting fraudulent activities in real time is challenging because fraudsters often attempt to mimic legitimate user behavior.

Therefore, there is a need for an intelligent fraud detection system that can automatically analyze transaction patterns and identify suspicious behavior using machine learning techniques.

4. Objectives of the Project

- Design and develop a machine learning based credit card fraud detection system.
- Classify transactions into Safe, Suspicious, and Fraud categories.
- Provide a web-based platform for users to simulate and monitor transactions.
- Build an administrative dashboard for monitoring users and system activity.
- Analyze transaction behavior using machine learning algorithms.

5. Scope of the Project

The scope of this project includes building a web-based application capable of simulating credit card transactions and detecting fraudulent activity using machine learning.

The system allows users to perform simulated transactions and receive risk predictions while administrators can monitor system-wide activity.

6. Literature Survey

Research in financial fraud detection has explored machine learning algorithms such as Logistic Regression, Decision Trees, Support Vector Machines, and Random Forest.

Random Forest has shown strong performance due to its ensemble learning nature and ability to reduce overfitting while handling complex datasets.

7. Existing System

Traditional fraud detection systems rely on rule-based mechanisms to identify suspicious transactions. These systems depend on predefined rules such as transaction limits and location mismatches.

However, such systems struggle to detect evolving fraud patterns and often generate a high number of false positives.

8. Proposed System

The proposed system uses a machine learning model to analyze transaction behavior and detect anomalies. The system integrates a web-based interface with a backend fraud detection engine to classify transactions based on risk levels.

Users can simulate transactions and receive instant fraud predictions.

9. System Architecture

[PLACEHOLDER – SYSTEM ARCHITECTURE DIAGRAM]

The system follows a client-server architecture where the React frontend communicates with a FastAPI backend through REST APIs.

10. Technology Stack

Frontend

- React
- Vite
- Tailwind CSS
- Axios
- React Router

Backend

- Python
- FastAPI

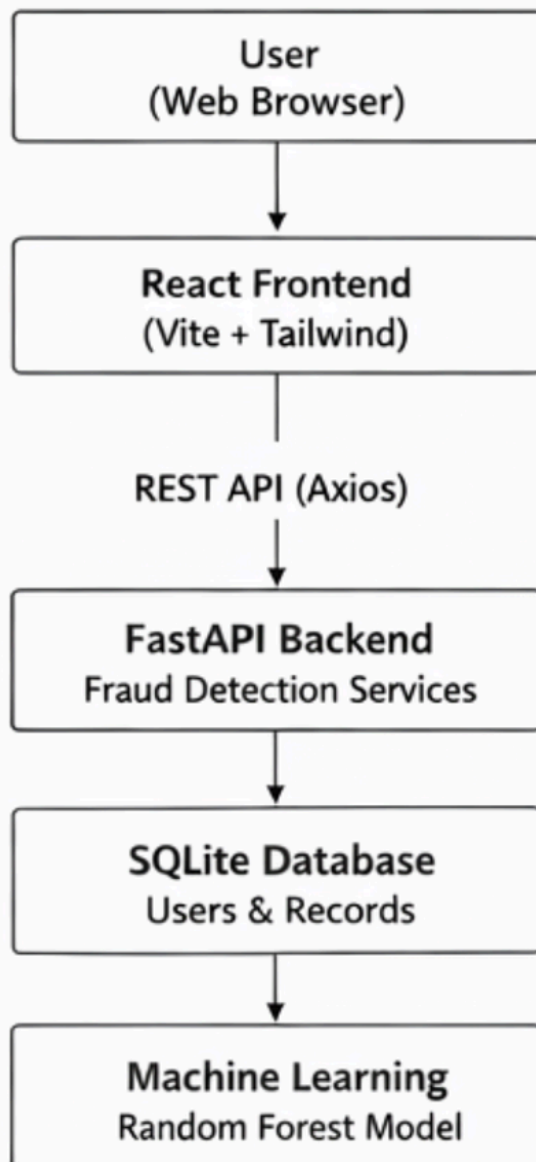
- SQLAlchemy
- SQLite

Machine Learning

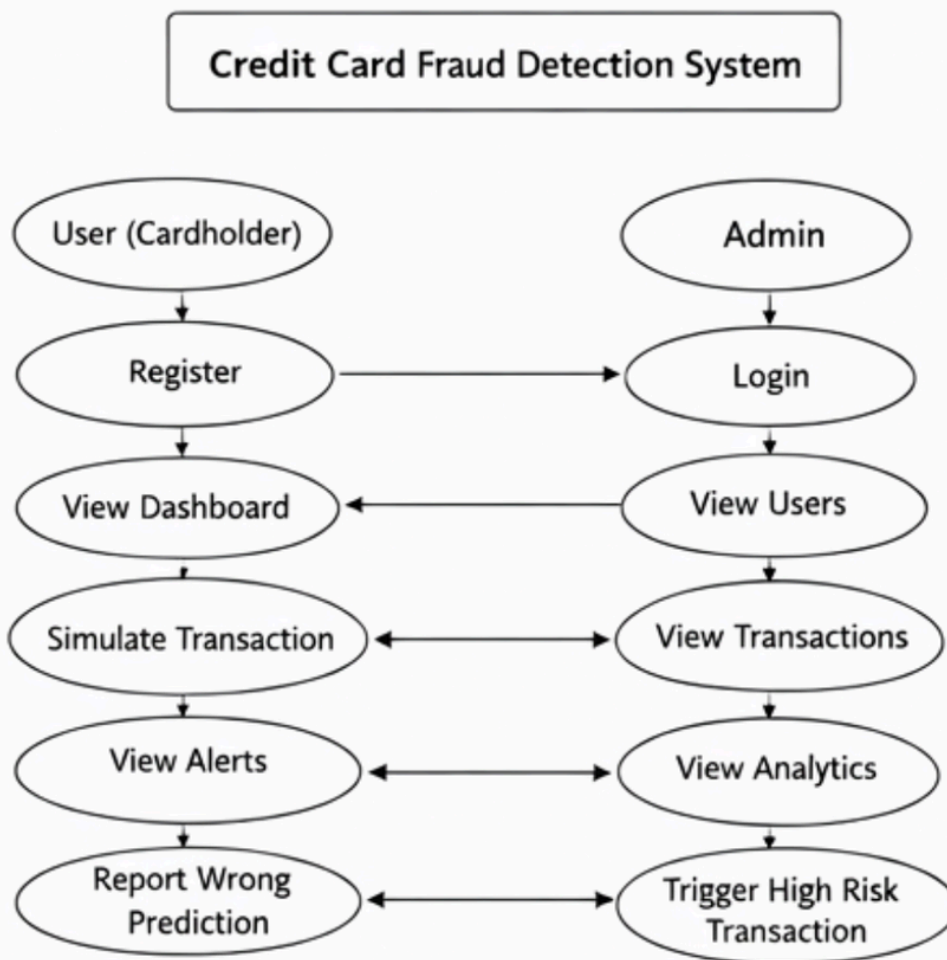
- scikit-learn
- pandas
- NumPy
- Random Forest Algorithm

11. System Design

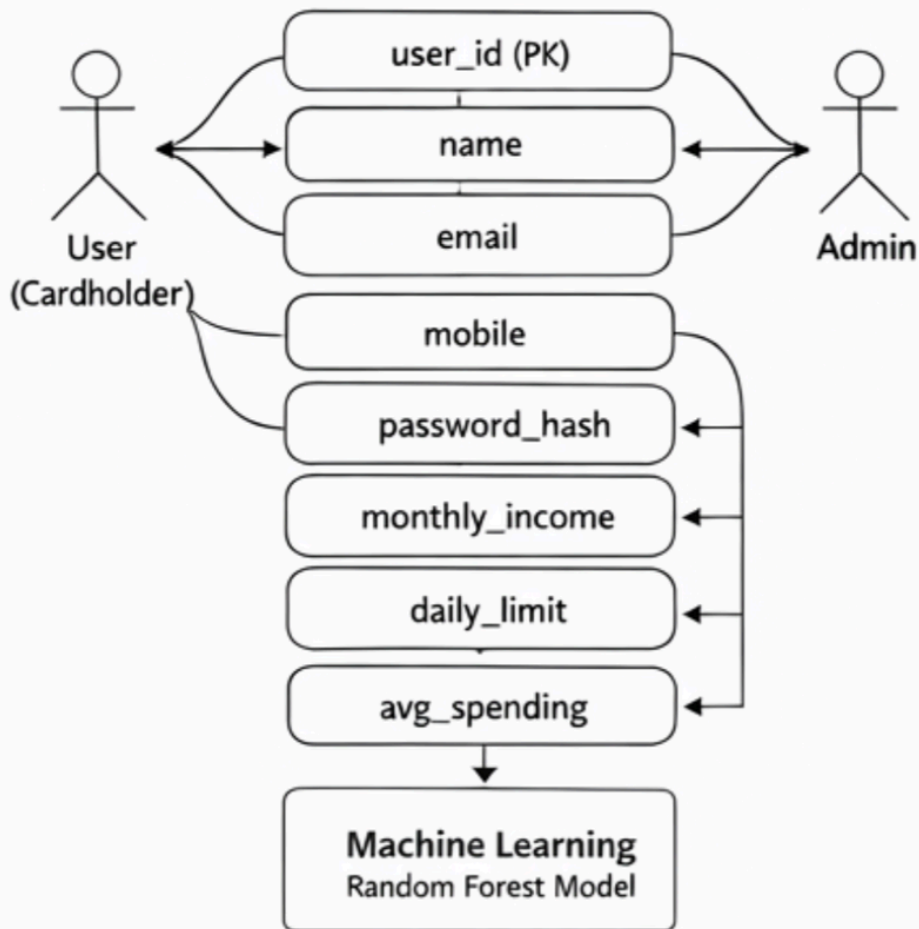
1. System Architecture



2. Use Case Diagram



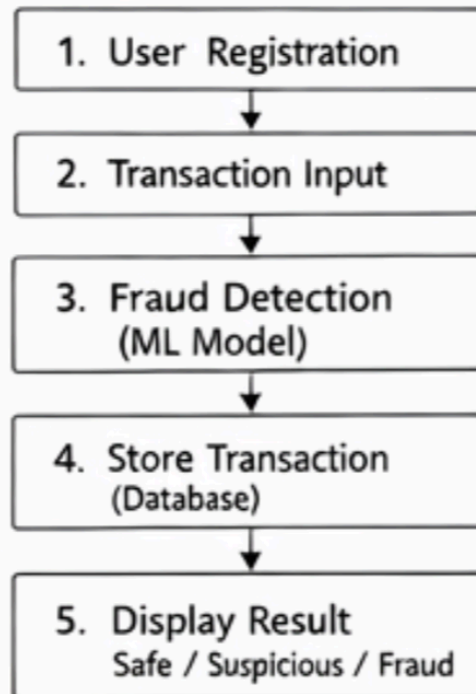
3. ER Diagram



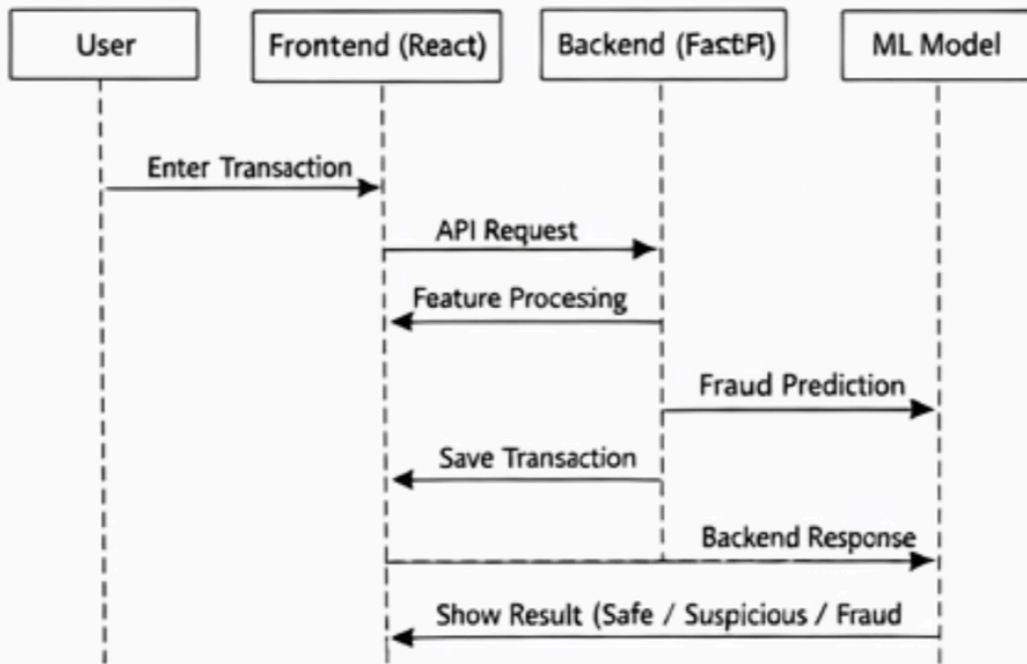
4. DFD Level 0



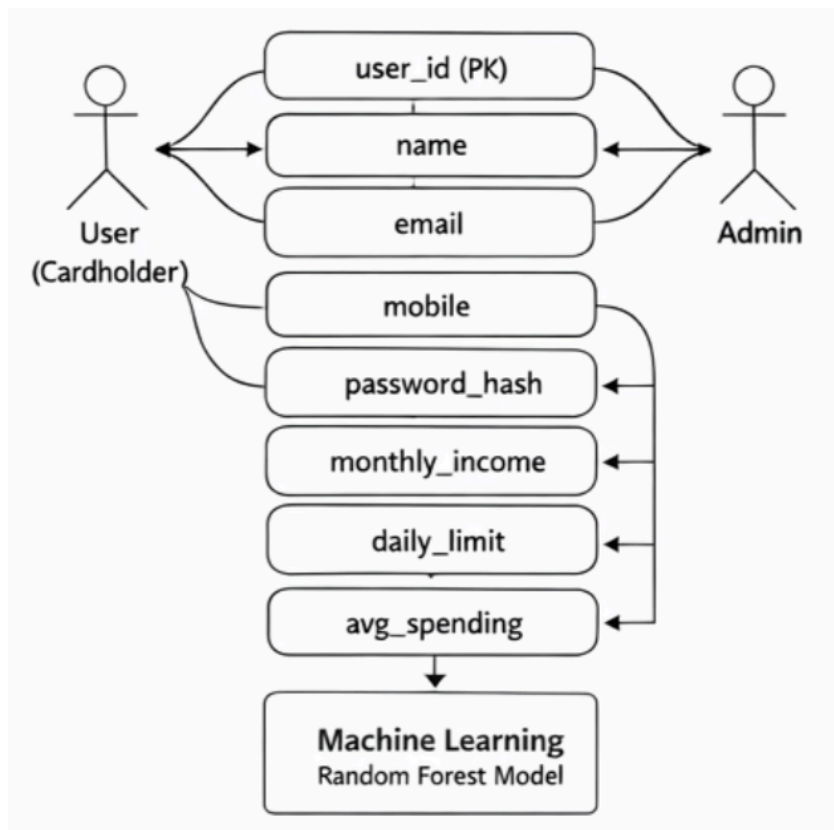
5. DFD Level 1



6. Sequence Diagram



12. Database Design



The system database includes three primary tables:

- Cardholders

- Transactions
- ML Models

13. Machine Learning Model

The system uses the Random Forest algorithm for fraud detection. The model analyzes multiple transaction attributes including transaction amount, deviation score, location, device type, payment mode, and transaction time.

The model is trained on synthetic transaction data and evaluated using accuracy, precision, recall, and F1-score metrics.

14. System Modules

User Module

- Registration
- Login
- Transaction Monitoring
- Transaction Simulation
- Fraud Alerts

Admin Module

- User Management
- Transaction Monitoring
- System Analytics
- Model Retraining

15. API Design

User APIs

- Register User
- Login User
- Get User Details

Transaction APIs

- Simulate Transaction
- Get Transactions
- Report Fraud Prediction

Admin APIs

- Admin Login
- Get All Users
- Get All Transactions
- Trigger High Risk Transaction
- Retrain Model

16. Implementation Details - Code

File: credit-card-fraud-detection/backend/config.py

```
"""
```

Configuration management for Fraud Detection System

```
"""
```

```
import os
```

```
from pathlib import Path
```

```
# Base directory
```

```
BASE_DIR = Path(__file__).parent
```

```
# Database configuration
```

```
DATABASE_URL = f"sqlite:///{BASE_DIR}/fraud_detection.db"
```

```
DATABASE_ECHO = False
```

```
# ML Model configuration
```

```
MODEL_PATH = BASE_DIR / "models" / "fraud_model_v1.pkl"
```

```
MODEL_DIR = BASE_DIR / "models"
```

```
# Simulator configuration
```

```
SIMULATOR_INTERVAL_MINUTES = 2
```

```
# Risk scoring thresholds
```

```
DAILY_LIMIT_THRESHOLD = 1.0
```

```
DEVIATION_HIGH_RISK = 2.5
```

```
DEVIATION_MEDIUM_RISK = 1.5
```

```
# ML Model parameters
```

```
RANDOM_FOREST_N_ESTIMATORS = 100
```

```
RANDOM_FOREST_MAX_DEPTH = 10
```

```
RANDOM_FOREST_RANDOM_STATE = 42
```

```
# Training data
```

```
SYNTHETIC_DATA_SIZE = 5000
```

```
TEST_DATA_SIZE = 1000
```

```
# Seed for historical transactions on user registration
```

```
SEED_TRANSACTIONS_COUNT = 10
```

File: credit-card-fraud-detection/backend/crud.py

```
"""
```

```
CRUD operations for database models
```

```
"""
```

```
from datetime import datetime, timedelta
```

```
from sqlalchemy.orm import Session
```

```
from sqlalchemy import func
```

```
from models import CardHolder, Transaction, MLModel
```

```
import schemas
```

```
from passlib.context import CryptContext
```

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")
```

```
def get_password_hash(password: str) -> str:
```

```
    return pwd_context.hash(password)
```

```

def verify_password(plain_password: str, hashed_password: str) -> bool:
    return pwd_context.verify(plain_password, hashed_password)

def create_user(db: Session, user_data: schemas.CardHolderRegisterRequest) -> CardHolder:
    """Create a new cardholder with initial avg_spending"""
    avg_spending = user_data.monthly_income / 30

    db_user = CardHolder(
        name=user_data.name,
        email=user_data.email,
        mobile=user_data.mobile,
        card_number=user_data.card_number,
        password_hash=get_password_hash(user_data.password),
        monthly_income=user_data.monthly_income,
        daily_limit=user_data.daily_limit,
        avg_spending=avg_spending
    )
    db.add(db_user)
    db.commit()
    db.refresh(db_user)
    return db_user

```

```

def get_user_by_id(db: Session, user_id: int) -> CardHolder:

```

```
"""Get user by ID"""
```

```
return db.query(CardHolder).filter(CardHolder.user_id == user_id).first()
```

```
def get_user_by_email(db: Session, email: str) -> CardHolder:
```

```
    """Get user by email"""
```

```
    return db.query(CardHolder).filter(CardHolder.email == email).first()
```

```
def authenticate_user(db: Session, email: str, password: str) -> CardHolder:
```

```
    """Authenticate a cardholder"""
```

```
    user = get_user_by_email(db, email)
```

```
    if not user:
```

```
        return None
```

```
    if not verify_password(password, user.password_hash):
```

```
        return None
```

```
    return user
```

```
def get_all_users(db: Session, skip: int = 0, limit: int = 100) -> list:
```

```
    """Get all users with pagination"""
```

```
    return db.query(CardHolder).offset(skip).limit(limit).all()
```

```
def create_transaction(
```

```
    db: Session, user_id: int, amount: float, location: str, device: str, mode: str,
```

```
    deviation_score: float, risk_score: float = None, risk_label: str = "Safe", is_fraud: bool = False
```


) -> Transaction:

```
"""Create a new transaction"""

db_transaction = Transaction(
    user_id=user_id, amount=amount, location=location, device=device, mode=mode,
    deviation_score=deviation_score, risk_score=risk_score, risk_label=risk_label,
    is_fraud=is_fraud,
    timestamp=datetime.utcnow()
)

db.add(db_transaction)

db.commit()

db.refresh(db_transaction)

return db_transaction
```

def get_user_transactions(db: Session, user_id: int, limit: int = 100, offset: int = 0) -> list:

```
"""Get all transactions for a user"""

return db.query(Transaction).filter(
    Transaction.user_id == user_id
).order_by(Transaction.timestamp.desc()).offset(offset).limit(limit).all()
```

def get_all_transactions(db: Session, limit: int = 1000, offset: int = 0) -> list:

```
"""Get all transactions"""

return db.query(Transaction).order_by(
    Transaction.timestamp.desc()
).offset(offset).limit(limit).all()
```

```

def get_transaction_risk_distribution(db: Session) -> dict:
    """Get distribution of risk labels across all transactions"""

    total = db.query(func.count(Transaction.trans_id)).scalar()

    safe = db.query(func.count(Transaction.trans_id)).filter(Transaction.risk_label ==
"Safe").scalar()

    suspicious = db.query(func.count(Transaction.trans_id)).filter(Transaction.risk_label ==
"Suspicious").scalar()

    flagged = db.query(func.count(Transaction.trans_id)).filter(Transaction.risk_label ==
"Flagged").scalar()

    fraud = db.query(func.count(Transaction.trans_id)).filter(Transaction.is_fraud ==
True).scalar()

    total = max(total, 1)

    return {
        'total_transactions': total,
        'safe_count': safe,
        'safe_percentage': (safe / total) * 100,
        'suspicious_count': suspicious,
        'suspicious_percentage': (suspicious / total) * 100,
        'flagged_count': flagged,
        'flagged_percentage': (flagged / total) * 100,
        'fraud_detected': fraud
    }

```

```

def create_or_update_ml_model(
    db: Session, version: str, accuracy: float, precision: float, recall: float,
    f1: float, training_samples: int, model_path: str
) -> MLModel:
    """Create or update ML model record"""
    db_model = db.query(MLModel).filter(MLModel.version == version).first()

    if db_model:
        db_model.accuracy_score = accuracy
        db_model.precision_score = precision
        db_model.recall_score = recall
        db_model.f1_score = f1
        db_model.last_trained_date = datetime.utcnow()
        db_model.total_training_samples = training_samples
        db_model.model_path = model_path
    else:
        db_model = MLModel(
            version=version, accuracy_score=accuracy, precision_score=precision,
            recall_score=recall, f1_score=f1, last_trained_date=datetime.utcnow(),
            total_training_samples=training_samples, model_path=model_path
        )
        db.add(db_model)

    db.commit()

    db.refresh(db_model)

```

```
return db_model
```

```
def get_latest_ml_model(db: Session) -> MLModel:
```

```
    """Get the latest trained ML model"""
```

```
    return db.query(MLModel).order_by(MLModel.last_trained_date.desc()).first()
```

```
def report_transaction(db: Session, transaction_id: int) -> Transaction:
```

```
    """Mark a transaction as reported (wrong prediction)"""
```

```
    transaction = db.query(Transaction).filter(Transaction.trans_id ==  
transaction_id).first()
```

```
    if transaction:
```

```
        transaction.reported = True
```

```
        db.commit()
```

```
        db.refresh(transaction)
```

```
    return transaction
```

```
def get_user_alerts(db: Session, user_id: int) -> dict:
```

```
    """Get all fraud/suspicious alerts (flagged transactions) for a user"""
```

```
    fraud_alerts = db.query(Transaction).filter(  
        (Transaction.user_id == user_id) &
```

```
        ((Transaction.is_fraud == True) | (Transaction.risk_label == "Fraud") |  
(Transaction.risk_label == "Flagged"))
```

```
    ).order_by(Transaction.timestamp.desc()).all()
```

```
suspicious_alerts = db.query(Transaction).filter(
    (Transaction.user_id == user_id) &
    (Transaction.risk_label == "Suspicious")
).order_by(Transaction.timestamp.desc()).all()

all_alerts = fraud_alerts + suspicious_alerts

return {
    "alerts": all_alerts,
    "fraud_count": len(fraud_alerts),
    "suspicious_count": len(suspicious_alerts),
    "total": len(all_alerts)
}
```

File: credit-card-fraud-detection/backend/database.py

```
"""
```

Database setup and session management

Uses SQLite with SQLAlchemy ORM

```
"""
```

```
from sqlalchemy import create_engine
```

```
from sqlalchemy.orm import declarative_base, sessionmaker
```

```
from config import DATABASE_URL, DATABASE_ECHO
```

```
engine = create_engine(  
    DATABASE_URL,  
    echo=DATABASE_ECHO,  
    connect_args={"check_same_thread": False}  
)
```

```
SessionLocal = sessionmaker(  
    autocommit=False,  
    autoflush=False,  
    bind=engine  
)
```

```
Base = declarative_base()
```

```
def get_db():  
    """Dependency to get database session in FastAPI endpoints"""  
    db = SessionLocal()  
    try:  
        yield db  
    finally:  
        db.close()
```

```
def create_tables():  
    """Create all tables in the database"""
```

```
Base.metadata.create_all(bind=engine)
```

```
def drop_tables():
```

```
    """Drop all tables (for testing/cleanup)"""
```

```
    Base.metadata.drop_all(bind=engine)
```

File: credit-card-fraud-detection/backend/main.py

```
"""
```

```
FastAPI Application - Fraud Detection System
```

```
"""
```

```
from datetime import datetime
```

```
from fastapi import FastAPI, Depends, HTTPException, status
```

```
from fastapi.responses import JSONResponse
```

```
from sqlalchemy.orm import Session
```

```
import crud, schemas
```

```
from database import get_db, create_tables
```

```
from models import CardHolder, Transaction
```

```
from ml_engine import fraud_model
```

```
from simulator import (
```

```
    seed_user_history, generate_high_risk_transaction,
```

```
    LOCATIONS, DEVICES, MODES, encode_categorical_feature
```

```
)
```

```
from fastapi.middleware.cors import CORSMiddleware
```

```
app = FastAPI(  
    title="Credit Card Fraud Detection System",  
    description="AI-Based Fraud Detection with RandomForest ML Model",  
    version="1.0.0"  
)
```

```
# CORS Configuration
```

```
origins = [  
    "http://localhost:8080",  
    "http://localhost:8081",  
    "http://127.0.0.1:8080",  
    "http://127.0.0.1:8081",  
    "http://localhost:3000",  
    "http://localhost:5173",  
    "https://detect.projectbro.in",  
    "https://www.detect.projectbro.in"  
]
```

```
app.add_middleware(  
    CORSMiddleware,  
    allow_origins=origins,  
    allow_credentials=True,  
    allow_methods=["*"],  
    allow_headers=["*"],
```


)

```
@app.on_event("startup")
```

```
async def startup_event():
```

```
    """Initialize application on startup"""
```

```
    print("=" * 60)
```

```
    print("🚀 CREDIT CARD FRAUD DETECTION SYSTEM STARTING")
```

```
    print("=" * 60)
```

```
    create_tables()
```

```
    print("✓ Database initialized")
```

```
    if not fraud_model.is_trained:
```

```
        print("🤖 Training ML model...")
```

```
        metrics = fraud_model.train_model()
```

```
        print(f"✓ Model trained - Accuracy: {metrics['accuracy']:.4f}")
```

```
    else:
```

```
        print("✓ ML model loaded from disk")
```

```
# Background transaction simulator disabled - only user-created transactions are stored
```

```
# scheduler = BackgroundScheduler()
```

```
# scheduler.add_job(
```

```
#     simulate_batch_transactions,
```

```
#     'interval',
```

```
#     minutes=SIMULATOR_INTERVAL_MINUTES,
```

```
# id='transaction_simulator'

# )

# scheduler.start()

print("✓ Transaction simulator disabled - only user-simulated transactions will be
stored")
```

```
print("=" * 60)
```

```
print("✅ APPLICATION READY AT http://localhost:8000")
```

```
print("=" * 60)
```

```
@app.get("/health", tags=["Health"])
```

```
async def health_check():
```

```
    """Health check endpoint"""
```

```
    return {
```

```
        "status": "healthy",
```

```
        "timestamp": datetime.utcnow().isoformat(),
```

```
        "ml_model_trained": fraud_model.is_trained
```

```
    }
```

```
# =====
```

```
# USER APIs
```

```
# =====
```

```
@app.post("/users/register", response_model=schemas.CardHolderResponse, tags=
["Users"])
```

```
async def register_user(user_data: schemas.CardHolderRegisterRequest, db: Session =
Depends(get_db)):
```

```
    """Register a new cardholder"""
```

```
    existing_user = crud.get_user_by_email(db, user_data.email)
```

```
    if existing_user:
```

```
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Email already registered")
```

```
    db_user = crud.create_user(db, user_data)
```

```
    seed_user_history(db, db_user.user_id)
```

```
    return db_user
```

```
@app.post("/users/login", response_model=schemas.CardHolderResponse, tags=["Users"])
```

```
async def login_user(creds: schemas.CardHolderLoginRequest, db: Session =
Depends(get_db)):
```

```
    """Login a cardholder"""
```

```
    user = crud.authenticate_user(db, creds.email, creds.password)
```

```
    if not user:
```

```
        raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid email or password")
```

```
    return user
```

```
@app.get("/users/{user_id}", response_model=schemas.CardHolderResponse, tags=
["Users"])
```

```
async def get_user(user_id: int, db: Session = Depends(get_db)):
```

```
"""Get user details by ID"""
```

```
user = crud.get_user_by_id(db, user_id)
```

```
if not user:
```

```
    raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"User  
{user_id} not found")
```

```
return user
```

```
@app.get("/transactions/{user_id}", response_model=schemas.UserTransactionsResponse,  
tags=["Transactions"])
```

```
async def get_user_transactions(user_id: int, limit: int = 100, offset: int = 0, db: Session =  
Depends(get_db)):
```

```
    """Get all transactions for a user with risk labels"""
```

```
    user = crud.get_user_by_id(db, user_id)
```

```
    if not user:
```

```
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"User  
{user_id} not found")
```

```
    transactions = crud.get_user_transactions(db, user_id, limit=limit, offset=offset)
```

```
    return {
```

```
        "user_id": user_id,
```

```
        "name": user.name,
```

```
        "total_transactions": len(transactions),
```

```
        "transactions": transactions
```

```
    }
```

```

@app.post("/transactions/simulate",
response_model=schemas.TransactionSimulationResponse, tags=["Transactions"])

async def simulate_transaction(request: schemas.TransactionSimulationRequest, db:
Session = Depends(get_db)):

    """Simulate a transaction to test fraud detection (demo only)"""

    user = crud.get_user_by_id(db, request.user_id)

    if not user:

        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"User
{request.user_id} not found")


    # Calculate deviation score

    deviation_score = request.amount / user.avg_spending


    # Get ML prediction

    ml_result = fraud_model.predict_transaction({

        'amount': request.amount,

        'deviation': deviation_score,

        'location_encoded': encode_categorical_feature(request.location, 'location'),

        'device_encoded': encode_categorical_feature(request.device, 'device'),

        'mode_encoded': encode_categorical_feature(request.mode, 'mode'),

        'time_feature': datetime.utcnow().hour

    })


    # Determine risk label based on ML prediction

    from simulator import calculate_risk_label

    risk_label = calculate_risk_label(request.amount, deviation_score, user.daily_limit)

    if ml_result['is_fraud_predicted']:

```

```
    risk_label = "Fraud"

# Create transaction record
transaction = Transaction(
    user_id=request.user_id,
    amount=request.amount,
    location=request.location,
    device=request.device,
    mode=request.mode,
    deviation_score=deviation_score,
    risk_score=ml_result['fraud_probability'],
    risk_label=risk_label,
    is_fraud=ml_result['is_fraud_predicted'],
    timestamp=datetime.utcnow()
)

db.add(transaction)

db.commit()

db.refresh(transaction)

# Determine prediction message
if ml_result['is_fraud_predicted']:
    prediction = "Fraud Detected"
elif risk_label == "Suspicious":
    prediction = "Suspicious Activity"
else:
    prediction = "Safe Transaction"
```

```
return {  
    "message": "Transaction simulated successfully",  
    "transaction": transaction,  
    "prediction": prediction  
}
```

```
@app.post("/transactions/report", tags=["Transactions"])
```

```
async def report_wrong_prediction(request: schemas.ReportTransactionRequest, db:  
Session = Depends(get_db)):
```

```
    """Report a transaction as wrong prediction"""
```

```
    transaction = crud.report_transaction(db, request.transaction_id)
```

```
    if not transaction:
```

```
        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND,  
detail=f"Transaction {request.transaction_id} not found")
```

```
return {  
    "message": "Transaction reported successfully. Thank you for the feedback!",  
    "transaction": transaction  
}
```

```
@app.get("/alerts/{user_id}", tags=["Alerts"])
```

```
async def get_user_alerts(user_id: int, db: Session = Depends(get_db)):
```

```
    """Get all fraud and suspicious alerts for a user"""
```

```

user = crud.get_user_by_id(db, user_id)

if not user:

    raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"User
{user_id} not found")

alerts_data = crud.get_user_alerts(db, user_id)

return {

    "user_id": user_id,

    "alerts": alerts_data["alerts"],

    "total_fraud_count": alerts_data["fraud_count"],

    "total_suspicious_count": alerts_data["suspicious_count"]

}

# =====

# ADMIN APIs

# =====

@app.post("/admin/login", tags=["Admin"])

async def admin_login(creds: schemas.AdminLoginRequest):

    """Admin hardcoded login"""

    if creds.username == "admin" and creds.password == "admin123":

        return {"success": True, "token": "admin-token", "role": "admin"}

    raise HTTPException(status_code=status.HTTP_401_UNAUTHORIZED,
detail="Invalid credentials")

```



```
@app.get("/admin/users", tags=["Admin"])
```

```
async def get_all_cardholders(skip: int = 0, limit: int = 100, db: Session = Depends(get_db)):
```

```
    """Get all cardholders for admin"""
```

```
    users = crud.get_all_users(db, skip=skip, limit=limit)
```

```
    return users
```

```
@app.get("/admin/transactions", tags=["Admin"])
```

```
async def get_all_transactions(limit: int = 100, offset: int = 0, db: Session = Depends(get_db)):
```

```
    """Get all transactions across all users"""
```

```
    transactions = crud.get_all_transactions(db, limit=limit, offset=offset)
```

```
    # Need to include the user_name in the manual query
```

```
    result = []
```

```
    for t in transactions:
```

```
        user_name = "Unknown"
```

```
        if t.user:
```

```
            user_name = t.user.name
```

```
    result.append({
```

```
        "trans_id": t.trans_id,
```

```
        "user_id": t.user_id,
```

```
"user_name": user_name,  
"amount": t.amount,  
"location": t.location,  
"device": t.device,  
"mode": t.mode,  
"deviation_score": t.deviation_score,  
"risk_label": t.risk_label,  
"timestamp": t.timestamp  
})
```

```
return {  
    "limit": limit,  
    "offset": offset,  
    "transactions": result  
}
```

```
@app.get("/admin/analytics/summary", tags=["Admin"])  
async def get_admin_summary(db: Session = Depends(get_db)):  
    """Get high level summary for admin dashboard"""  
    distribution = crud.get_transaction_risk_distribution(db)  
    total_users = db.query(CardHolder).count()  
    return {  
        "total_users": total_users,  
        "total_transactions": distribution["total_transactions"],
```

```

    "fraud_percentage": distribution["flagged_percentage"] +
distribution["fraud_detected"],

    "risk_distribution": {

        "safe": distribution["safe_count"],

        "suspicious": distribution["suspicious_count"],

        "fraudulent": distribution["flagged_count"]

    }

}

```

```

@app.post("/admin/trigger-high-risk",
response_model=schemas.HighRiskTriggerResponse, tags=["Admin"])

```

```

async def trigger_high_risk_transaction(request: schemas.HighRiskTriggerRequest, db:
Session = Depends(get_db)):

```

```

    """Trigger a high-risk transaction for testing"""

```

```

    user = crud.get_user_by_id(db, request.user_id)

```

```

    if not user:

```

```

        raise HTTPException(status_code=status.HTTP_404_NOT_FOUND, detail=f"User
{request.user_id} not found")

```

```

    transaction = generate_high_risk_transaction(db, request.user_id,
amount_multiplier=request.amount_multiplier)

```

```

    alert = f"🚨 HIGH RISK ALERT: Unusual transaction detected for {user.name}"

```

```

    return {

```

```

        "message": "High-risk transaction triggered for demonstration",

```

```

        "transaction": transaction,

```

```

        "alert": alert
    }

```

```
}
```

```
@app.post("/admin/train", response_model=schemas.TrainModelResponse, tags=
["Admin"])
```

```
async def train_model(db: Session = Depends(get_db)):
```

```
    """Retrain ML model using current database transactions"""
```

```
    transactions = crud.get_all_transactions(db, limit=100000)
```

```
    if len(transactions) < 100:
```

```
        raise HTTPException(status_code=status.HTTP_400_BAD_REQUEST,
detail="Need at least 100 transactions to train model")
```

```
import pandas as pd
```

```
data = []
```

```
for trans in transactions:
```

```
    data.append({
```

```
        'amount': trans.amount,
```

```
        'deviation': trans.deviation_score,
```

```
        'location_encoded': encode_categorical_feature(trans.location, 'location'),
```

```
        'device_encoded': encode_categorical_feature(trans.device, 'device'),
```

```
        'mode_encoded': encode_categorical_feature(trans.mode, 'mode'),
```

```
        'time_feature': trans.timestamp.hour,
```

```
        'label': 1 if trans.is_fraud else 0
```

```
    })
```

```
df = pd.DataFrame(data)
```

```
metrics = fraud_model.train_model(df)
```

```
model_record = crud.create_or_update_ml_model(  
    db=db, version="v1", accuracy=metrics['accuracy'], precision=metrics['precision'],  
    recall=metrics['recall'], f1=metrics['f1'], training_samples=metrics['training_samples'],  
    model_path=str(fraud_model.MODEL_PATH)  
)
```

```
return {  
    "message": f"Model retrained successfully with {len(transactions)} transactions",  
    "model_details": model_record  
}
```

```
@app.exception_handler(HTTPException)
```

```
async def http_exception_handler(request, exc):
```

```
    return JSONResponse(status_code=exc.status_code, content={"detail": exc.detail})
```

```
@app.exception_handler(Exception)
```

```
async def general_exception_handler(request, exc):
```

```
    return JSONResponse(status_code=500, content={"detail": "Internal server error",  
    "error": str(exc)})
```

```
if __name__ == "__main__":
```

```
import uvicorn

uvicorn.run(app, host="0.0.0.0", port=8000)
```

File: credit-card-fraud-detection/backend/ml_engine.py

```
"""

ML Engine for fraud detection using scikit-learn RandomForestClassifier

"""

import os

import pickle

from datetime import datetime

import numpy as np

import pandas as pd

from sklearn.ensemble import RandomForestClassifier

from sklearn.preprocessing import LabelEncoder

from sklearn.model_selection import train_test_split

from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score

from config import (

    MODEL_PATH, MODEL_DIR, RANDOM_FOREST_N_ESTIMATORS,

    RANDOM_FOREST_MAX_DEPTH, RANDOM_FOREST_RANDOM_STATE,

    SYNTHETIC_DATA_SIZE

)


class FraudDetectionModel:
```

"""Fraud Detection Model using RandomForest"""

def __init__(self):

"""Initialize the model and load from disk if exists"""

self.model = None

self.label_encoders = {}

self.feature_columns = [

'amount', 'deviation', 'location_encoded',

'device_encoded', 'mode_encoded', 'time_feature'

]

self.model_version = "v1"

self.is_trained = False

os.makedirs(MODEL_DIR, exist_ok=True)

self.load_model()

def _generate_synthetic_data(self, size=SYNTHETIC_DATA_SIZE):

"""Generate synthetic training data for initial model"""

np.random.seed(RANDOM_FOREST_RANDOM_STATE)

normal_size = int(size * 0.92)

normal_data = {

'amount': np.random.exponential(100, normal_size),

'deviation': np.random.normal(1.0, 0.5, normal_size),

'location_encoded': np.random.randint(0, 20, normal_size),

'device_encoded': np.random.randint(0, 5, normal_size),

'mode_encoded': np.random.randint(0, 4, normal_size),

```
'time_feature': np.random.randint(0, 24, normal_size),  
'label': np.zeros(normal_size)  
}
```

```
fraud_size = size - normal_size
```

```
fraud_data = {  
    'amount': np.random.exponential(300, fraud_size),  
    'deviation': np.random.normal(4.5, 1.5, fraud_size),  
    'location_encoded': np.random.randint(0, 20, fraud_size),  
    'device_encoded': np.random.randint(0, 5, fraud_size),  
    'mode_encoded': np.random.randint(0, 4, fraud_size),  
    'time_feature': np.random.randint(0, 24, fraud_size),  
    'label': np.ones(fraud_size)  
}
```

```
df_normal = pd.DataFrame(normal_data)
```

```
df_fraud = pd.DataFrame(fraud_data)
```

```
df = pd.concat([df_normal, df_fraud], ignore_index=True)
```

```
df = df.sample(frac=1).reset_index(drop=True)
```

```
return df
```

```
def train_model(self, data=None):
```

```
    """Train the RandomForest model"""
```

```
    if data is None:
```

```
        data = self._generate_synthetic_data(SYNTHETIC_DATA_SIZE)
```



```

required_cols = self.feature_columns + ['label']

for col in required_cols:

    if col not in data.columns:

        raise ValueError(f'Missing required column: {col}')


X = data[self.feature_columns].values
y = data['label'].values


X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=RANDOM_FOREST_RANDOM_STATE,
    stratify=y
)


self.model = RandomForestClassifier(
    n_estimators=RANDOM_FOREST_N_ESTIMATORS,
    max_depth=RANDOM_FOREST_MAX_DEPTH,
    random_state=RANDOM_FOREST_RANDOM_STATE,
    n_jobs=-1,
    class_weight='balanced'
)

self.model.fit(X_train, y_train)

self.is_trained = True


y_pred = self.model.predict(X_test)


metrics = {

```

```
'accuracy': accuracy_score(y_test, y_pred),  
'precision': precision_score(y_test, y_pred, zero_division=0),  
'recall': recall_score(y_test, y_pred, zero_division=0),  
'f1': f1_score(y_test, y_pred, zero_division=0),  
'training_samples': len(X_train),  
'test_samples': len(X_test)  
}
```

```
self.save_model()  
  
return metrics
```

```
def predict_transaction(self, features_dict):  
    """Predict if a transaction is fraudulent"""  
    if self.model is None or not self.is_trained:  
        raise ValueError("Model not trained. Please train the model first.")
```

```
features = np.array([[  
    features_dict['amount'],  
    features_dict['deviation'],  
    features_dict['location_encoded'],  
    features_dict['device_encoded'],  
    features_dict['mode_encoded'],  
    features_dict['time_feature']  
]])
```

```
prediction = self.model.predict(features)[0]
```

```
probability = self.model.predict_proba(features)[0][1]
```

```
return {  
    'is_fraud_predicted': bool(prediction),  
    'fraud_probability': float(probability)  
}
```

```
def save_model(self):  
    """Save model to disk"""  
    os.makedirs(MODEL_DIR, exist_ok=True)  
    with open(MODEL_PATH, 'wb') as f:  
        pickle.dump({  
            'model': self.model,  
            'label_encoders': self.label_encoders,  
            'feature_columns': self.feature_columns,  
            'is_trained': self.is_trained  
        }, f)
```

```
def load_model(self):  
    """Load model from disk"""  
    if os.path.exists(MODEL_PATH):  
        try:  
            with open(MODEL_PATH, 'rb') as f:  
                data = pickle.load(f)  
                self.model = data['model']  
                self.label_encoders = data.get('label_encoders', {})
```

```

        self.feature_columns = data.get('feature_columns', self.feature_columns)

        self.is_trained = data.get('is_trained', True)

    except Exception as e:

        print(f'Error loading model: {e}. Will train new model.')

        self.model = None

        self.is_trained = False

def get_model_info(self):

    """Get model metadata"""

    return {

        'version': self.model_version,

        'is_trained': self.is_trained,

        'model_path': str(MODEL_PATH),

        'n_features': len(self.feature_columns) if self.model else None

    }

def evaluate_model_on_data(self, data):

    """Evaluate model performance on provided data"""

    if self.model is None or not self.is_trained:

        raise ValueError("Model not trained")

    X = data[self.feature_columns].values

    y = data['label'].values

    y_pred = self.model.predict(X)

    metrics = {

```

```

'accuracy': accuracy_score(y, y_pred),

'precision': precision_score(y, y_pred, zero_division=0),

'recall': recall_score(y, y_pred, zero_division=0),

'f1': f1_score(y, y_pred, zero_division=0)

}

return metrics

```

```

fraud_model = FraudDetectionModel()

```

File: credit-card-fraud-detection/backend/models.py

```

"""

```

SQLAlchemy ORM models for the database

```

"""

```

```

from datetime import datetime

```

```

from sqlalchemy import Column, Integer, String, Float, Boolean, DateTime, ForeignKey

```

```

from sqlalchemy.orm import relationship

```

```

from database import Base

```

```

class CardHolder(Base):

```

```

    """User model - stores customer information"""

```

```

    __tablename__ = "cardholders"

```

```

user_id = Column(Integer, primary_key=True, index=True)

name = Column(String(255), nullable=False)

email = Column(String(255), unique=True, nullable=False, index=True)

mobile = Column(String(15), unique=True, nullable=False, index=True)

card_number = Column(String(20), unique=True, nullable=False, index=True)

password_hash = Column(String(255), nullable=False)

monthly_income = Column(Float, nullable=False)

daily_limit = Column(Float, nullable=False)

avg_spending = Column(Float, nullable=False)

created_at = Column(DateTime, default=datetime.utcnow)

updated_at = Column(DateTime, default=datetime.utcnow, onupdate=datetime.utcnow)


transactions = relationship("Transaction", back_populates="user", cascade="all, delete-orphan")


def __repr__(self):

    return f"<CardHolder(user_id={self.user_id}, name={self.name}, email={self.email})>"


class Transaction(Base):

    """Transaction model - stores all transactions"""

    __tablename__ = "transactions"

    trans_id = Column(Integer, primary_key=True, index=True)

    user_id = Column(Integer, ForeignKey("cardholders.user_id"), nullable=False, index=True)

```

```
amount = Column(Float, nullable=False)
```

```
location = Column(String(255), nullable=False)
```

```
device = Column(String(100), nullable=False)
```

```
mode = Column(String(50), nullable=False)
```

```
deviation_score = Column(Float, nullable=False)
```

```
risk_score = Column(Float, nullable=True)
```

```
risk_label = Column(String(20), nullable=False)
```

```
is_fraud = Column(Boolean, default=False)
```

```
reported = Column(Boolean, default=False, nullable=False)
```

```
timestamp = Column(DateTime, default=datetime.utcnow, index=True)
```

```
user = relationship("CardHolder", back_populates="transactions")
```

```
def __repr__(self):
```

```
    return f"<Transaction(trans_id={self.trans_id}, user_id={self.user_id}, amount={self.amount})>"
```

```
class MLModel(Base):
```

```
    """ML Model metadata - tracks trained models"""
```

```
    __tablename__ = "ml_models"
```

```
    model_id = Column(Integer, primary_key=True, index=True)
```

```
    version = Column(String(50), unique=True, nullable=False)
```

```
    accuracy_score = Column(Float, nullable=True)
```

```
    precision_score = Column(Float, nullable=True)
```

```
recall_score = Column(Float, nullable=True)

f1_score = Column(Float, nullable=True)

last_trained_date = Column(DateTime, default=datetime.utcnow)

total_training_samples = Column(Integer, nullable=True)

model_path = Column(String(500), nullable=True)


def __repr__(self):

    return f"<MLModel(version={self.version}, accuracy={self.accuracy_score})>"
```

File: credit-card-fraud-detection/backend/schemas.py

```
"""
```

Pydantic schemas for request/response validation

```
"""
```

```
from datetime import datetime
```

```
from pydantic import BaseModel, EmailStr, Field
```

```
from typing import Optional, List
```

```
class CardHolderRegisterRequest(BaseModel):
```

```
    """Request schema for cardholder registration"""
```

```
    name: str = Field(..., min_length=1, max_length=255)
```

```
    email: EmailStr
```

```
    mobile: str = Field(..., min_length=10, max_length=15)
```

```
    card_number: str = Field(..., min_length=12, max_length=20)
```



```
password: str = Field(..., min_length=6)
```

```
monthly_income: float = Field(..., gt=0)
```

```
daily_limit: float = Field(..., gt=0)
```

```
class CardHolderLoginRequest(BaseModel):
```

```
    """Request schema for cardholder login"""
```

```
    email: EmailStr
```

```
    password: str
```

```
class CardHolderResponse(BaseModel):
```

```
    """Response schema for cardholder info"""
```

```
    user_id: int
```

```
    name: str
```

```
    email: str
```

```
    mobile: str
```

```
    monthly_income: float
```

```
    daily_limit: float
```

```
    avg_spending: float
```

```
    created_at: datetime
```

```
class Config:
```

```
    from_attributes = True
```

```
class AdminLoginRequest(BaseModel):
```

```
    """Login request for admin"""
```

username: str

password: str

```
class TransactionCreateRequest(BaseModel):
```

```
    """Request schema for creating manual transaction"""
```

```
    user_id: int
```

```
    amount: float = Field(..., gt=0)
```

```
    location: str = Field(..., min_length=1, max_length=255)
```

```
    device: str = Field(..., min_length=1, max_length=100)
```

```
    mode: str = Field(..., min_length=1, max_length=50)
```

```
class TransactionResponse(BaseModel):
```

```
    """Response schema for transaction"""
```

```
    trans_id: int
```

```
    user_id: int
```

```
    amount: float
```

```
    location: str
```

```
    device: str
```

```
    mode: str
```

```
    deviation_score: float
```

```
    risk_score: Optional[float]
```

```
    risk_label: str
```

```
    is_fraud: bool
```

```
    reported: bool = False
```

timestamp: datetime

class Config:

from_attributes = True

class UserTransactionsResponse(BaseModel):

"""Response schema for user's transaction history"""

user_id: int

name: str

total_transactions: int

transactions: List[TransactionResponse]

class MLModelResponse(BaseModel):

"""Response schema for ML model info"""

version: str

accuracy_score: Optional[float]

precision_score: Optional[float]

recall_score: Optional[float]

f1_score: Optional[float]

last_trained_date: datetime

total_training_samples: Optional[int]

class Config:

from_attributes = True

```
class RiskDistributionResponse(BaseModel):
```

```
    """Risk distribution analytics response"""
```

```
    total_transactions: int
```

```
    safe_count: int
```

```
    safe_percentage: float
```

```
    suspicious_count: int
```

```
    suspicious_percentage: float
```

```
    flagged_count: int
```

```
    flagged_percentage: float
```

```
    fraud_detected: int
```

```
class HighRiskTriggerRequest(BaseModel):
```

```
    """Request to trigger high-risk transaction"""
```

```
    user_id: int
```

```
    amount_multiplier: float = Field(default=5.0, description="Multiplier for deviation  
score")
```

```
class HighRiskTriggerResponse(BaseModel):
```

```
    """Response for high-risk trigger"""
```

```
    message: str
```

```
    transaction: TransactionResponse
```

```
    alert: str
```

```
class TrainModelResponse(BaseModel):  
    """Response from model training"""  
  
    message: str  
  
    model_details: MLModelResponse  
  
    model_config = {'protected_namespaces': ()}
```

```
class StatusResponse(BaseModel):  
    """Generic status response"""  
  
    success: bool  
  
    message: str  
  
    data: Optional[dict] = None
```

```
class TransactionSimulationRequest(BaseModel):  
    """Request schema for transaction simulation"""  
  
    user_id: int  
  
    amount: float = Field(..., gt=0)  
  
    location: str = Field(..., min_length=1, max_length=255)  
  
    device: str = Field(..., min_length=1, max_length=100)  
  
    mode: str = Field(..., min_length=1, max_length=50)
```

```
class TransactionSimulationResponse(BaseModel):
```

```
    """Response for transaction simulation"""
```

```
    message: str
```

```
    transaction: TransactionResponse
```

```
    prediction: str
```

```
class ReportTransactionRequest(BaseModel):
```

```
    """Request to report a transaction as wrong prediction"""
```

```
    transaction_id: int
```

```
class AlertsResponse(BaseModel):
```

```
    """Response for user alerts (fraud transactions)"""
```

```
    user_id: int
```

```
    alerts: List[TransactionResponse]
```

```
    total_fraud_count: int
```

```
    total_suspicious_count: int
```

```
File: credit-card-fraud-detection/backend/simulator.py
```

```
"""
```

```
Background transaction simulator - Generates realistic transactions at regular intervals
```

```
"""
```

```
import random
```

```
from datetime import datetime, timedelta
```

```
from sqlalchemy.orm import Session
```

```
from database import SessionLocal
```

```
from models import CardHolder, Transaction
```

```
from ml_engine import fraud_model
```

```
import crud
```

```
from config import DEVIATION_HIGH_RISK, DEVIATION_MEDIUM_RISK,  
SEED_TRANSACTIONS_COUNT
```

```
LOCATIONS = [
```

```
    "Mumbai", "Delhi", "Bangalore", "Chennai", "Hyderabad",
```

```
    "Pune", "Kolkata", "Ahmedabad", "Jaipur", "Lucknow",
```

```
    "Chandigarh", "Goa", "Indore", "Bhopal", "Visakhapatnam",
```

```
    "Coimbatore", "Kochi", "Surat", "Nagpur", "Vadodara"
```

```
]
```

```
DEVICES = ["Mobile App", "Website", "ATM", "POS Terminal", "Phone"]
```

```
MODES = ["UPI Payment", "Online Shopping", "ATM Withdrawal", "POS Payment",  
"Direct Bank Transfer"]
```

```
def calculate_risk_label(amount: float, deviation_score: float, daily_limit: float) -> str:
```

```
    """Calculate risk label based on amount, deviation, and daily limit"""
```

```
    if amount > daily_limit:
```

```
        return "Flagged"
```

```
    if deviation_score > DEVIATION_HIGH_RISK:
```

```

    return "Flagged"

elif deviation_score > DEVIATION_MEDIUM_RISK:

    return "Suspicious"

return "Safe"


def encode_categorical_feature(feature: str, feature_type: str) -> int:

    """Encode categorical features to integers"""

    if feature_type == "location":

        return LOCATIONS.index(feature) if feature in LOCATIONS else random.randint(0,
len(LOCATIONS) - 1)

    elif feature_type == "device":

        return DEVICES.index(feature) if feature in DEVICES else random.randint(0,
len(DEVICES) - 1)

    elif feature_type == "mode":

        return MODES.index(feature) if feature in MODES else random.randint(0,
len(MODES) - 1)

    return 0


def seed_user_history(db: Session, user_id: int, count: int =
SEED_TRANSACTIONS_COUNT):

    """Create historical transactions for a new user to establish baseline behavior"""

    user = crud.get_user_by_id(db, user_id)

    if not user:

        return

```



```
for _ in range(count):

    if random.random() < 0.9:

        amount = user.avg_spending * random.uniform(0.5, 1.5)

    else:

        amount = user.avg_spending * random.uniform(1.5, 3.0)


location = random.choice(LOCATIONS)

device = random.choice(DEVICES)

mode = random.choice(MODES)

days_back = random.randint(0, 30)

timestamp = datetime.utcnow() - timedelta(days=days_back)


deviation_score = amount / user.avg_spending

risk_label = calculate_risk_label(amount, deviation_score, user.daily_limit)


ml_result = fraud_model.predict_transaction({

    'amount': amount,

    'deviation': deviation_score,

    'location_encoded': encode_categorical_feature(location, 'location'),

    'device_encoded': encode_categorical_feature(device, 'device'),

    'mode_encoded': encode_categorical_feature(mode, 'mode'),

    'time_feature': timestamp.hour

})


transaction = Transaction(

    user_id=user_id, amount=amount, location=location, device=device, mode=mode,
```

```
        deviation_score=deviation_score, risk_score=ml_result['fraud_probability'],
        risk_label=risk_label, is_fraud=ml_result['is_fraud_predicted'],
timestamp=timestamp
    )
    db.add(transaction)
```

```
db.commit()
```

```
def generate_realistic_transaction(db: Session, user_id: int) -> Transaction:
```

```
    """Generate a realistic transaction for a user"""
```

```
    user = crud.get_user_by_id(db, user_id)
```

```
    if not user:
```

```
        return None
```

```
    if random.random() < 0.85:
```

```
        amount = user.avg_spending * random.uniform(0.5, 1.5)
```

```
    elif random.random() < 0.95:
```

```
        amount = user.avg_spending * random.uniform(1.5, 2.5)
```

```
    else:
```

```
        amount = user.avg_spending * random.uniform(2.5, 5.0)
```

```
    location = random.choice(LOCATIONS)
```

```
    device = random.choice(DEVICES)
```

```
    mode = random.choice(MODES)
```

```
    current_time = datetime.utcnow()
```

```
deviation_score = amount / user.avg_spending
```

```
risk_label = calculate_risk_label(amount, deviation_score, user.daily_limit)
```

```
ml_result = fraud_model.predict_transaction({
```

```
    'amount': amount,
```

```
    'deviation': deviation_score,
```

```
    'location_encoded': encode_categorical_feature(location, 'location'),
```

```
    'device_encoded': encode_categorical_feature(device, 'device'),
```

```
    'mode_encoded': encode_categorical_feature(mode, 'mode'),
```

```
    'time_feature': current_time.hour
```

```
})
```

```
transaction = Transaction(
```

```
    user_id=user_id, amount=amount, location=location, device=device, mode=mode,
```

```
    deviation_score=deviation_score, risk_score=ml_result['fraud_probability'],
```

```
    risk_label=risk_label, is_fraud=ml_result['is_fraud_predicted'],
```

```
    timestamp=current_time
```

```
)
```

```
db.add(transaction)
```

```
db.commit()
```

```
db.refresh(transaction)
```

```
return transaction
```

```
def simulate_batch_transactions(limit: int = None):
```

```
    """Generate transactions for random users. Called by scheduler every few minutes"""
```

```

db = SessionLocal()

try:
    users = db.query(CardHolder).all()

    if not users:
        return

    num_to_process = max(1, int(len(users) * 0.3))

    selected_users = random.sample(users, min(num_to_process, len(users)))

    transactions = []

    for user in selected_users:
        try:
            transaction = generate_realistic_transaction(db, user.user_id)

            if transaction:
                transactions.append(transaction)

        except Exception as e:
            print(f"Error generating transaction for user {user.user_id}: {e}")

    print(f"[{datetime.utcnow()}] Simulator: Generated {len(transactions)} transactions")

    return transactions

except Exception as e:
    print(f"Error in simulator: {e}")

finally:
    db.close()

```

```

def generate_high_risk_transaction(db: Session, user_id: int, amount_multiplier: float = 5.0) -> Transaction:

```

```

    """Deliberately generate a high-risk transaction for testing alerts"""

```

```
user = crud.get_user_by_id(db, user_id)
```

```
if not user:
```

```
    return None
```

```
amount = user.avg_spending * amount_multiplier
```

```
location = random.choice(LOCATIONS)
```

```
device = random.choice(DEVICES)
```

```
mode = random.choice(MODES)
```

```
current_time = datetime.utcnow()
```

```
deviation_score = amount / user.avg_spending
```

```
risk_label = "Flagged"
```

```
ml_result = fraud_model.predict_transaction({
```

```
    'amount': amount,
```

```
    'deviation': deviation_score,
```

```
    'location_encoded': encode_categorical_feature(location, 'location'),
```

```
    'device_encoded': encode_categorical_feature(device, 'device'),
```

```
    'mode_encoded': encode_categorical_feature(mode, 'mode'),
```

```
    'time_feature': current_time.hour
```

```
})
```

```
transaction = Transaction(
```

```
    user_id=user_id, amount=amount, location=location, device=device, mode=mode,
```

```
    deviation_score=deviation_score, risk_score=ml_result['fraud_probability'],
```

```
    risk_label=risk_label, is_fraud=ml_result['is_fraud_predicted'],
```

```
    timestamp=current_time
```

```
)  
db.add(transaction)  
db.commit()  
db.refresh(transaction)  
return transaction
```

File: credit-card-fraud-detection/frontend/components.json

```
{  
  "$schema": "https://ui.shadcn.com/schema.json",  
  "style": "default",  
  "rsc": false,  
  "tsx": false,  
  "tailwind": {  
    "config": "tailwind.config.js",  
    "css": "src/index.css",  
    "baseColor": "slate",  
    "cssVariables": true,  
    "prefix": ""  
  },  
  "aliases": {  
    "components": "@components",  
    "utils": "@lib/Utils",  
    "ui": "@components/ui",  
    "lib": "@lib",
```

```
"hooks": "@/hooks"
}
}
```

File: credit-card-fraud-detection/frontend/eslint.config.js

```
import js from "@eslint/js";
import globals from "globals";
import reactHooks from "eslint-plugin-react-hooks";
import reactRefresh from "eslint-plugin-react-refresh";
```

```
export default [
  { ignores: ["dist"] },
  {
    extends: [js.configs.recommended],
    files: ["**/*.{js,jsx}"],
    languageOptions: {
      ecmaVersion: 2020,
      globals: globals.browser,
      parserOptions: {
        ecmaVersion: 'latest',
        ecmaFeatures: { jsx: true },
        sourceType: 'module',
      },
    },
  },
  {
    plugins: {
```

```
"react-hooks": reactHooks,
"react-refresh": reactRefresh,
},
rules: {
  ...reactHooks.configs.recommended.rules,
  "react-refresh/only-export-components": ["warn", { allowConstantExport: true }],
},
},
];
```

File: credit-card-fraud-detection/frontend/index.html

```
<!doctype html>
<html lang="en">

<head>

  <meta charset="UTF-8" />

  <meta name="viewport" content="width=device-width, initial-scale=1.0" />

  <title>Credit Card Fraud Detection</title>

  <meta name="description"

    content="Advanced machine learning system that detects fraudulent credit card
transactions in real-time. Protect your finances with intelligent threat detection and instant
alerts." />

  <meta name="author" content="Credit Card Fraud Detection" />
```



```
<link rel="preconnect" href="https://fonts.googleapis.com">

<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>

<link

  href="https://fonts.googleapis.com/css2?
family=Montserrat:wght@600;700;800&family=Inter:wght@400;500;600;700&display=sw
ap"

  rel="stylesheet">

</head>

<body>

  <div id="root"></div>

  <script type="module" src="/src/main.jsx"></script>

</body>

</html>
```

File: credit-card-fraud-detection/frontend/package.json

```
{

  "name": "vite-react-shadcn-js",

  "private": true,

  "version": "0.0.0",

  "type": "module",

  "scripts": {

    "dev": "vite",

    "build": "vite build",
```

```
"build:dev": "vite build --mode development",  
  
"lint": "eslint .",  
  
"preview": "vite preview",  
  
"test": "vitest run",  
  
"test:watch": "vitest"  
},  
  
"dependencies": {  
  
  "@hookform/resolvers": "^3.10.0",  
  
  "@radix-ui/react-accordion": "^1.2.11",  
  
  "@radix-ui/react-alert-dialog": "^1.1.14",  
  
  "@radix-ui/react-aspect-ratio": "^1.1.7",  
  
  "@radix-ui/react-avatar": "^1.1.10",  
  
  "@radix-ui/react-checkbox": "^1.3.2",  
  
  "@radix-ui/react-collapsible": "^1.1.11",  
  
  "@radix-ui/react-context-menu": "^2.2.15",  
  
  "@radix-ui/react-dialog": "^1.1.14",  
  
  "@radix-ui/react-dropdown-menu": "^2.1.15",  
  
  "@radix-ui/react-hover-card": "^1.1.14",  
  
  "@radix-ui/react-label": "^2.1.7",  
  
  "@radix-ui/react-menubar": "^1.1.15",  
  
  "@radix-ui/react-navigation-menu": "^1.2.13",  
  
  "@radix-ui/react-popover": "^1.1.14",  
  
  "@radix-ui/react-progress": "^1.1.7",  
  
  "@radix-ui/react-radio-group": "^1.3.7",  
  
  "@radix-ui/react-scroll-area": "^1.2.9",  
  
  "@radix-ui/react-select": "^2.2.5",
```

"@radix-ui/react-separator": "^1.1.7",
"@radix-ui/react-slider": "^1.3.5",
"@radix-ui/react-slot": "^1.2.3",
"@radix-ui/react-switch": "^1.2.5",
"@radix-ui/react-tabs": "^1.1.12",
"@radix-ui/react-toast": "^1.2.14",
"@radix-ui/react-toggle": "^1.1.9",
"@radix-ui/react-toggle-group": "^1.1.10",
"@radix-ui/react-tooltip": "^1.2.7",
"@tanstack/react-query": "^5.83.0",
"axios": "^1.13.6",
"class-variance-authority": "^0.7.1",
"clsx": "^2.1.1",
"cmdk": "^1.1.1",
"date-fns": "^3.6.0",
"embla-carousel-react": "^8.6.0",
"framer-motion": "^12.35.0",
"input-otp": "^1.4.2",
"lucide-react": "^0.462.0",
"next-themes": "^0.3.0",
"react": "^18.3.1",
"react-day-picker": "^8.10.1",
"react-dom": "^18.3.1",
"react-hook-form": "^7.61.1",
"react-resizable-panels": "^2.1.9",
"react-router-dom": "^6.30.1",

```
"recharts": "^2.15.4",  
"sonner": "^1.7.4",  
"tailwind-merge": "^2.6.0",  
"tailwindcss-animate": "^1.0.7",  
"vaul": "^0.9.9",  
"zod": "^3.25.76"  
},  
"devDependencies": {  
  "@eslint/js": "^9.32.0",  
  "@tailwindcss/typography": "^0.5.16",  
  "@testing-library/jest-dom": "^6.6.0",  
  "@testing-library/react": "^16.0.0",  
  "@vitejs/plugin-react-swc": "^3.11.0",  
  "autoprefixer": "^10.4.21",  
  "eslint": "^9.32.0",  
  "eslint-plugin-react-hooks": "^5.2.0",  
  "eslint-plugin-react-refresh": "^0.4.20",  
  "globals": "^15.15.0",  
  "jsdom": "^20.0.3",  
  "lovable-tagger": "^1.1.13",  
  "postcss": "^8.5.6",  
  "tailwindcss": "^3.4.17",  
  "vite": "^5.4.19",  
  "vitest": "^3.2.4"  
}  
}
```

File: credit-card-fraud-detection/frontend/postcss.config.js

```
export default {  
  plugins: [  
    tailwindcss: {},  
    autoprefixer: {},  
  ],  
};
```

File: credit-card-fraud-detection/frontend/src/App.css

```
#root {  
  max-width: 1280px;  
  margin: 0 auto;  
  padding: 2rem;  
  text-align: center;  
}  
  
.logo {  
  height: 6em;  
  padding: 1.5em;  
  will-change: filter;  
  transition: filter 300ms;
```

```
}
```

```
.logo:hover {
```

```
  filter: drop-shadow(0 0 2em #646cffaa);
```

```
}
```

```
.logo.react:hover {
```

```
  filter: drop-shadow(0 0 2em #61dafbaa);
```

```
}
```

```
@keyframes logo-spin {
```

```
  from {
```

```
    transform: rotate(0deg);
```

```
  }
```

```
  to {
```

```
    transform: rotate(360deg);
```

```
  }
```

```
}
```

```
@media (prefers-reduced-motion: no-preference) {
```

```
  a:nth-of-type(2) .logo {
```

```
    animation: logo-spin infinite 20s linear;
```

```
  }
```

```
}
```

```
.card {
```

```
  padding: 2em;
```

```
}
```

```
.read-the-docs {  
  color: #888;  
}
```

File: credit-card-fraud-detection/frontend/src/App.jsx

```
import { Toaster } from "@components/ui/toaster";  
import { Toaster as Sonner } from "@components/ui/sonner";  
import { TooltipProvider } from "@components/ui/tooltip";  
import { QueryClient, QueryClientProvider } from "@tanstack/react-query";  
import { BrowserRouter, Routes, Route } from "react-router-dom";  
import { AppProvider } from "@context/AppContext";  
import Portfolio from "./pages/Portfolio";  
import Login from "./pages/Login";  
import Register from "./pages/Register";  
import Dashboard from "./pages/Dashboard";  
import Alerts from "./pages/Alerts";  
import NotFound from "./pages/NotFound";  
  
const queryClient = new QueryClient();  
  
const App = () => (  
  <QueryClientProvider client={queryClient}>  
    <TooltipProvider>
```

```
<AuthProvider>

  <Toaster />

  <Sonner />

  <BrowserRouter>

    <Routes>

      <Route path="/" element={<Portfolio />} />

      <Route path="/login" element={<Login />} />

      <Route path="/register" element={<Register />} />

      <Route path="/dashboard" element={<Dashboard />} />

      <Route path="/alerts" element={<Alerts />} />

      <Route path="*" element={<NotFound />} />

    </Routes>

  </BrowserRouter>

</AuthProvider>

</TooltipProvider>

</QueryClientProvider>

);

export default App;
```

File: credit-card-fraud-detection/frontend/src/components/dashboard/AdminDashboard.jsx

```
import { useState, useEffect } from "react";

import { adminApi } from "@lib/api";
```



```
import { Badge } from "@components/ui/badge";

import { Button } from "@components/ui/button";

import {
  Table,
  TableHeader,
  TableRow,
  TableHead,
  TableBody,
  TableCell,
} from "@components/ui/table";

import { Shield, LogOut, LayoutDashboard, ArrowLeftRight, Loader2 } from "lucide-react";

import { useNavigate } from "react-router-dom";

import { useApp } from "@context/AppContext";

import { toast } from "sonner";

import RiskDistributionChart from "../RiskDistributionChart";

const riskBadge = (level) => {
  level = level.toLowerCase();

  if (level === "fraud" || level === "flagged")

    return <Badge className="bg-destructive/20 text-destructive border-destructive/30">Fraud</Badge>;

  if (level === "suspicious")

    return <Badge className="bg-warning/20 text-warning border-warning/30">Suspicious</Badge>;

  return <Badge className="bg-success/20 text-success border-success/30">Safe</Badge>;
```

```
};
```

```
const AdminDashboard = () => {  
  const [tab, setTab] = useState("overview");  
  const { logout, currentUser } = useApp();  
  const navigate = useNavigate();  
  
  const [summary, setSummary] = useState({ total_users: 0, total_transactions: 0,  
    fraud_percentage: 0 });  
  const [users, setUsers] = useState([]);  
  const [transactions, setTransactions] = useState([]);  
  
  useEffect(() => {  
    const fetchData = async () => {  
      try {  
        if (tab === "overview") {  
          const s = await adminApi.getSummary();  
          setSummary(s);  
          const u = await adminApi.getUsers();  
          setUsers(u);  
        } else if (tab === "transactions") {  
          const t = await adminApi.getTransactions(500, 0);  
          setTransactions(t.transactions || []);  
        }  
      } catch (err) {  
        console.error(err);  
      }  
    }  
  });  
}
```

```
    }  
  };  
  fetchData();  
}, [tab]);
```

```
const handleLogout = () => {  
  logout();  
  navigate("/");  
};
```

```
const trainModel = async () => {  
  const loadingToast = toast.loading("Training ML model...");  
  try {  
    const res = await adminApi.trainModel();  
    toast.dismiss(loadingToast);  
    toast.success(`Model retrained successfully!\nAccuracy:  
${(res.model_details?.accuracy_score * 100).toFixed(2)}%`);  
  } catch (e) {  
    toast.dismiss(loadingToast);  
    const errorMsg = e.response?.data?.detail || e.message || "Failed to train model";  
    toast.error(errorMsg);  
  }  
};
```

```
const triggerHighRisk = async () => {  
  try {
```

```

    if (users.length === 0) {

        toast.error("No users available to trigger transaction");

        return;

    }

    const randomUser = users[Math.floor(Math.random() * users.length)];

    const loadingToast = toast.loading(`Creating high-risk transaction for
    ${randomUser.name}...`);

    await adminApi.triggerHighRisk(randomUser.user_id, 5.0);

    toast.dismiss(loadingToast);

    toast.success(`High-risk transaction triggered for ${randomUser.name}`);

} catch (e) {

    const errorMsg = e.response?.data?.detail || e.message || "Failed to trigger high-risk
    transaction";

    toast.error(errorMsg);

}

};

const maskStr = (str) => {

    if (!str) return "***";

    if (str.includes("@")) {

        const [u, d] = str.split("@");

        return u.substring(0, 2) + "***@" + d;

    }

    return str.substring(0, 2) + "***" + str.substring(str.length - 2);

};

```

```
if (!currentUser) return null;
```

```
return (
```

```
<div className="min-h-screen flex flex-col md:flex-row bg-background">
```

```
  { /* Sidebar */ }
```

```
  <aside className="hidden md:flex md:w-64 sticky top-0 h-screen border-b md:border-b-0 md:border-r border-border bg-card flex-col">
```

```
    <div className="p-4 border-b border-border">
```

```
      <div className="flex items-center gap-3 mb-3">
```

```
        <div className="w-10 h-10 rounded-lg bg-primary/20 flex items-center justify-center">
```

```
          <Shield className="w-5 h-5 text-primary" />
```

```
        </div>
```

```
        <span className="font-heading font-bold text-sm">Admin Control</span>
```

```
      </div>
```

```
    <div className="bg-secondary/50 rounded-lg p-3 text-xs">
```

```
      <p className="text-muted-foreground text-xs mb-1">Administrator</p>
```

```
      <p className="font-semibold text-sm">{currentUser.name || "Administrator"}</p>
```

```
      <p className="text-muted-foreground text-xs">Admin Account</p>
```

```
    </div>
```

```
  </div>
```

```
<nav className="flex-1 p-4 space-y-2 overflow-y-auto">
```

```
  <button
```

```
    onClick={() => setTab("overview")}>
```

```
    className={`w-full flex items-center gap-3 px-3 py-2 rounded-lg text-sm transition-all text-left ${tab === "overview" ? "bg-primary/15 text-primary" : "text-muted-foreground hover:bg-secondary"}`
```

```
}``}
```

```
>
```

```
<LayoutDashboard className="w-4 h-4" /> Overview & Users
```

```
</button>
```

```
<button
```

```
  onClick={() => setTab("transactions")}
```

```
  className={`w-full flex items-center gap-3 px-3 py-2 rounded-lg text-sm transition-
all text-left ${tab === "transactions" ? "bg-primary/15 text-primary" : "text-muted-
foreground hover:bg-secondary"}
```

```
}``}
```

```
>
```

```
<ArrowLeftRight className="w-4 h-4" /> All Transactions
```

```
</button>
```

```
</nav>
```

```
<div className="p-4 border-t border-border">
```

```
  <Button variant="ghost" className="w-full justify-start text-muted-foreground
gap-2 text-sm" onClick={handleLogout}>
```

```
    <LogOut className="w-4 h-4" /> Sign Out
```

```
  </Button>
```

```
</div>
```

```
</aside>
```

```
{/* Mobile Header */}
```

```
<div className="md:hidden border-b border-border bg-card p-3 flex items-center
justify-between">
```

```
  <div className="flex items-center gap-2">
```

```
    <Shield className="w-5 h-5 text-primary" />
```

```

<div className="flex flex-col">

  <span className="font-heading font-bold text-sm">Admin Panel</span>

  <span className="text-xs text-muted-foreground">{currentUser.name ||
"Administrator"}</span>

</div>

</div>

<Button variant="ghost" className="text-muted-foreground text-xs p-2" onClick=
{handleLogout}>

  <LogOut className="w-4 h-4" />

</Button>

</div>

{/* Mobile Tabs */}

<div className="md:hidden border-b border-border bg-card flex gap-2 p-2 overflow-
x-auto">

  <button

    onClick={() => setTab("overview")}

    className={`flex-shrink-0 py-2 px-3 rounded text-xs transition-all whitespace-
nowrap ${tab === "overview" ? "bg-primary/15 text-primary" : "text-muted-
foreground"}`} `>

    >

    Overview

  </button>

  <button

    onClick={() => setTab("transactions")}

    className={`flex-shrink-0 py-2 px-3 rounded text-xs transition-all whitespace-
nowrap ${tab === "transactions" ? "bg-primary/15 text-primary" : "text-muted-
foreground"}`} `>

```

>

Transactions

</button>

</div>

{/* Main Content */}

<main className="flex-1 p-3 md:p-6 overflow-auto">

{tab === "overview" && (

<div className="space-y-4 md:space-y-6">

<div className="grid grid-cols-1 md:grid-cols-3 gap-3 md:gap-6">

<div className="glass-card rounded-xl p-4 md:p-6">

<h3 className="text-sm font-medium text-muted-foreground">Total
Users</h3>
<p className="text-3xl font-heading font-bold mt-2">{summary.total_users}
</p>

</div>

<div className="glass-card rounded-xl p-6">

<h3 className="text-sm font-medium text-muted-foreground">Total
Transactions</h3>
<p className="text-3xl font-heading font-bold mt-2">
{summary.total_transactions}</p>

</div>

<div className="glass-card rounded-xl p-6">

<h3 className="text-sm font-medium text-destructive">Fraud %</h3>

<p className="text-3xl font-heading font-bold mt-2 text-destructive">

{summary.fraud_percentage.toFixed(2)}%

</p>

</div>

</div>

<div className="flex gap-4">

<Button onClick={trainModel}>Train ML Model</Button>

<Button variant="destructive" onClick={triggerHighRisk}>Trigger High Risk Transaction</Button>

</div>

<RiskDistributionChart data={summary} />

<div className="glass-card rounded-xl overflow-hidden mt-6">

<div className="p-4 border-b border-border">

<h2 className="font-heading font-semibold text-lg">Card Holders</h2>

</div>

<Table>

<TableHeader>

<TableRow className="border-border">

<TableHead>Name</TableHead>

<TableHead>Email</TableHead>

<TableHead>Mobile</TableHead>

<TableHead>Daily Limit</TableHead>

<TableHead>Avg Spending</TableHead>

</TableRow>

</TableHeader>

<TableBody>

```

    {users.map((u) => (
      <TableRow key={u.user_id} className="border-border hover:bg-secondary/50">
        <TableCell>{u.name}</TableCell>
        <TableCell>{maskStr(u.email)}</TableCell>
        <TableCell>{maskStr(u.mobile)}</TableCell>
        <TableCell>₹ {u.daily_limit?.toFixed(2)}</TableCell>
        <TableCell>₹ {u.avg_spending?.toFixed(2)}</TableCell>
      </TableRow>
    ))}
  </TableBody>
</Table>
</div>
</div>
))}

```

```

{tab === "transactions" && (
  <div className="space-y-6">
    <div className="glass-card rounded-xl overflow-hidden">
      <div className="p-4 border-b border-border">
        <h2 className="font-heading font-semibold text-lg">All Transactions</h2>
      </div>
      <Table>
        <TableHeader>
          <TableRow className="border-border">
            <TableHead>User Name</TableHead>

```

```

        <TableHead>Amount</TableHead>

        <TableHead>Deviation Score</TableHead>

        <TableHead>Risk Label</TableHead>

        <TableHead>Timestamp</TableHead>

    </TableRow>

</TableHeader>

<TableBody>

    {transactions.map((t) => (

        <TableRow key={t.trans_id} className="border-border hover:bg-
secondary/50">

            <TableCell className="font-semibold">{t.user_name}</TableCell>

            <TableCell>₹ {t.amount?.toFixed(2)}</TableCell>

            <TableCell>{t.deviation_score?.toFixed(2)}x</TableCell>

            <TableCell>{riskBadge(t.risk_label)}</TableCell>

            <TableCell className="text-sm text-muted-foreground">{new
Date(t.timestamp).toLocaleString()}</TableCell>

        </TableRow>

    )})

</TableBody>

</Table>

</div>

</div>

)}

</main>

</div>

);

```

```
};
```

```
export default AdminDashboard;
```

File: credit-card-fraud-
detection/frontend/src/components/dashboard/RiskDistributionChart.jsx

```
import { PieChart, Pie, Cell, Legend, Tooltip, ResponsiveContainer } from 'recharts';
```

```
const RiskDistributionChart = ({ data }) => {
```

```
  if (!data || !data.risk_distribution) {
```

```
    return (
```

```
      <div className="glass-card rounded-xl p-6 h-96 flex items-center justify-center">
```

```
        <p className="text-muted-foreground">No data available</p>
```

```
      </div>
```

```
    );
```

```
  }
```

```
  const chartData = [
```

```
    { name: 'Safe', value: data.risk_distribution.safe, fill: '#22c55e' },
```

```
    { name: 'Suspicious', value: data.risk_distribution.suspicious, fill: '#f59e0b' },
```

```
    { name: 'Fraudulent', value: data.risk_distribution.fraudulent, fill: '#ef4444' }
```

```
  ];
```

```
  return (
```

```
<div className="glass-card rounded-xl p-6">
```

```
  <h3 className="font-heading font-semibold text-lg mb-4">Risk Score  
Distribution</h3>
```

```
  <ResponsiveContainer width="100%" height={300}>
```

```
    <PieChart>
```

```
      <Pie
```

```
        data={chartData}
```

```
        cx="50%"
```

```
        cy="50%"
```

```
        labelLine={false}
```

```
        label={({ name, value }) => `${name}: ${value}`}
```

```
        outerRadius={100}
```

```
        fill="#8884d8"
```

```
        dataKey="value"
```

```
      >
```

```
        {chartData.map((entry, index) => (
```

```
          <Cell key={`cell-${index}`} fill={entry.fill} />
```

```
        )})
```

```
    </Pie>
```

```
    <Tooltip
```

```
      formatter={(value) => [`${value} transactions`, 'Count']}
```

```
      contentStyle={{ backgroundColor: '#1f2937', border: 'none', borderRadius: '8px',  
color: 'fff' }}
```

```
    />
```

```
    <Legend />
```

```
  </PieChart>
```

```
    </ResponsiveContainer>

</div>

);

};

export default RiskDistributionChart;
```

File: credit-card-fraud-
detection/frontend/src/components/dashboard/TransactionSimulator.jsx

```
import { useState } from "react";

import { transactionApi } from "@lib/api";

import { Button } from "@components/ui/button";

import { Input } from "@components/ui/input";

import { Label } from "@components/ui/label";

import {

  Dialog,

  DialogContent,

  DialogDescription,

  DialogHeader,

  DialogTitle,

  DialogTrigger,

} from "@components/ui/dialog";

import {

  Select,
```

```
SelectContent,  
  
SelectItem,  
  
SelectTrigger,  
  
SelectValue,  
  
} from "@components/ui/select";  
  
import { Badge } from "@components/ui/badge";  
  
import { Zap, Loader2, Check, AlertTriangle, ShieldAlert, AlertCircle } from "lucide-react";  
  
import { toast } from "sonner";  
  
const LOCATIONS = [  
  "Mumbai", "Delhi", "Bangalore", "Chennai", "Hyderabad",  
  "Pune", "Kolkata", "Ahmedabad", "Jaipur", "Lucknow",  
  "Chandigarh", "Goa", "Indore", "Bhopal", "Visakhapatnam",  
  "Coimbatore", "Kochi", "Surat", "Nagpur", "Vadodara"  
];  
  
const DEVICES = ["Mobile App", "Website", "ATM", "POS Terminal", "Phone"];  
  
const MODES = ["UPI Payment", "Online Shopping", "ATM Withdrawal", "POS Payment", "Direct Bank Transfer"];  
  
const resultConfig = {  
  "Fraud Detected": {  
    badge: <Badge className="bg-destructive/20 text-destructive border-destructive/30">Fraud Detected</Badge>,  
    icon: <ShieldAlert className="w-8 h-8 text-destructive" />,  
    message: "This transaction was flagged as fraudulent!"
```

```
    },  
  
    "Suspicious Activity": {  
  
      badge: <Badge className="bg-warning/20 text-warning border-warning/30">Suspicious</Badge>,  
  
      icon: <AlertTriangle className="w-8 h-8 text-warning" />,  
  
      message: "This transaction appears suspicious. Please verify."  
  
    },  
  
    "Safe Transaction": {  
  
      badge: <Badge className="bg-success/20 text-success border-success/30">Safe</Badge>,  
  
      icon: <Check className="w-8 h-8 text-success" />,  
  
      message: "This transaction is safe and normal."  
  
    }  
  
  };  
};
```

```
const TransactionSimulator = ({ userId }) => {  
  
  const [open, setOpen] = useState(false);  
  
  const [loading, setLoading] = useState(false);  
  
  const [result, setResult] = useState(null);  
  
  const [errors, setErrors] = useState({});  
  
  const [formData, setFormData] = useState({  
  
    amount: "",  
  
    location: "",  
  
    device: "",  
  
    mode: "",  
  
  });  
};
```



```
const handleInputChange = (e) => {  
  const { name, value } = e.target;  
  setFormData((prev) => ({  
    ...prev,  
    [name]: value,  
  }));  
  // Clear error for this field  
  if (errors[name]) {  
    setErrors((prev) => ({ ...prev, [name]: "" }));  
  }  
};
```

```
const handleSelectChange = (name, value) => {  
  setFormData((prev) => ({  
    ...prev,  
    [name]: value,  
  }));  
  // Clear error for this field  
  if (errors[name]) {  
    setErrors((prev) => ({ ...prev, [name]: "" }));  
  }  
};
```

```
const validateForm = () => {  
  const newErrors = {};
```

```
if (!formData.amount) newErrors.amount = "Amount is required";

else if (parseFloat(formData.amount) <= 0) newErrors.amount = "Amount must be
greater than 0";


if (!formData.location) newErrors.location = "Location is required";

if (!formData.device) newErrors.device = "Device is required";

if (!formData.mode) newErrors.mode = "Mode is required";


setErrors(newErrors);

return Object.keys(newErrors).length === 0;

};


const handleSubmit = async (e) => {

  e.preventDefault();

  if (!validateForm()) {

    toast.error("Please fill in all required fields");

    return;

  }

  setLoading(true);

  try {

    const response = await transactionApi.simulate({

      user_id: userId,

      amount: parseFloat(formData.amount),
```

```

    location: formData.location,

    device: formData.device,

    mode: formData.mode,

  });

  setResult(response);

  setFormData({ amount: "", location: "", device: "", mode: "" });

  toast.success("Transaction simulated successfully!");

} catch (error) {

  console.error("Error simulating transaction:", error);

  const errorMsg = error.response?.data?.detail || "Failed to simulate transaction. Please try again.";

  toast.error(errorMsg);

} finally {

  setLoading(false);

}

};

const handleReset = () => {

  setResult(null);

  setFormData({ amount: "", location: "", device: "", mode: "" });

  // setOpen(false);

};

return (

  <Dialog open={open} onOpenChange={setOpen}>

```

<DialogTrigger asChild>

<Button variant="default" size="sm" className="gap-1 text-xs md:text-sm">

<Zap className="w-3 h-3 md:w-4 md:h-4" /> Simulate

</Button>

</DialogTrigger>

<DialogContent className="max-w-md w-[95vw] md:w-full max-h-[90vh] md:max-h-none overflow-y-auto md:overflow-visible p-4 md:p-6">

<DialogHeader>

<DialogTitle className="text-base md:text-lg">Transaction Simulator</DialogTitle>

<DialogDescription className="text-xs md:text-sm">

Test the fraud detection system with demo transactions

</DialogDescription>

</DialogHeader>

{result ? (

<div className="space-y-3">

<div className="flex justify-center">

{resultConfig[result.prediction].icon}

</div>

<div className="space-y-1.5 text-center">

<p className="text-xs md:text-sm text-muted-foreground">Prediction Result</p>

<div className="flex justify-center">

{resultConfig[result.prediction].badge}

</div>

<p className="text-xs md:text-sm font-medium">
{resultConfig[result.prediction].message}</p>

</div>

<div className="bg-secondary/50 rounded-lg p-3 space-y-2 text-xs md:text-sm">

<div className="flex justify-between">

Amount:

₹ {result.transaction.amount.toFixed(0)}

</div>

<div className="flex justify-between">

Location:

{result.transaction.location.substring(0, 12)}

</div>

<div className="flex justify-between">

Device:

{result.transaction.device.substring(0, 12)}

</div>

<div className="flex justify-between">

Mode:

{result.transaction.mode.substring(0, 12)}

</div>

<div className="flex justify-between pt-2 border-t border-border">

Risk Score:

```
      <span className="font-semibold">{(result.transaction.risk_score *  
100).toFixed(1)}%</span>
```

```
    </div>
```

```
</div>
```

```
<Button onClick={handleReset} className="w-full text-sm">
```

```
  Simulate Another Transaction
```

```
</Button>
```

```
</div>
```

```
): (
```

```
<form onSubmit={handleSubmit} className="space-y-3">
```

```
<div className="space-y-1.5">
```

```
  <Label htmlFor="amount" className="text-xs md:text-sm">Amount (₹)  
</Label>
```

```
  <Input
```

```
    id="amount"
```

```
    name="amount"
```

```
    type="number"
```

```
    step="0.01"
```

```
    placeholder="10000.00"
```

```
    value={formData.amount}
```

```
    onChange={handleInputChange}
```

```
    disabled={loading}
```

```
    className={`text-sm ${errors.amount ? "border-destructive" : ""}`}
```

```
  />
```

```
  {errors.amount && <p className="text-xs text-destructive flex items-center gap-  
1"><AlertCircle className="w-3 h-3" />{errors.amount}</p>}
```

```
</div>
```

```
<div className="space-y-1.5">
```

```
  <Label htmlFor="location" className="text-xs md:text-sm">Location</Label>
```

```
  <Select value={formData.location} onChange={(value) =>
handleSelectChange("location", value)} disabled={loading}>
```

```
    <SelectTrigger id="location" className={`text-sm ${errors.location ? "border-
destructive" : ""}`}>
```

```
      <SelectValue placeholder="Select location" />
```

```
    </SelectTrigger>
```

```
    <SelectContent>
```

```
      {LOCATIONS.map((loc) => (
```

```
        <SelectItem key={loc} value={loc} className="text-sm">
```

```
          {loc}
```

```
        </SelectItem>
```

```
      )))
```

```
    </SelectContent>
```

```
  </Select>
```

```
  {errors.location && <p className="text-xs text-destructive flex items-center gap-
1"><AlertCircle className="w-3 h-3" />{errors.location}</p>}
```

```
</div>
```

```
<div className="space-y-1.5">
```

```
  <Label htmlFor="device" className="text-xs md:text-sm">Device</Label>
```

```
  <Select value={formData.device} onChange={(value) =>
handleSelectChange("device", value)} disabled={loading}>
```

```
    <SelectTrigger id="device" className={`text-sm ${errors.device ? "border-
destructive" : ""}`}>
```

```

        <SelectValue placeholder="Select device" />

    </SelectTrigger>

    <SelectContent>

        {DEVICES.map((dev) => (

            <SelectItem key={dev} value={dev} className="text-sm">

                {dev}

            </SelectItem>

        )))}

    </SelectContent>

</Select>

{errors.device && <p className="text-xs text-destructive flex items-center gap-1"><AlertCircle className="w-3 h-3" />{errors.device}</p>}}

</div>

```

```

<div className="space-y-1.5">

    <Label htmlFor="mode" className="text-xs md:text-sm">Mode</Label>

    <Select value={formData.mode} onValueChange={({value}) =>
handleSelectChange("mode", value)} disabled={loading}>

        <SelectTrigger id="mode" className={`text-sm ${errors.mode ? "border-destructive" : ""}`}>

            <SelectValue placeholder="Select mode" />

        </SelectTrigger>

        <SelectContent>

            {MODES.map((m) => (

                <SelectItem key={m} value={m} className="text-sm">

                    {m}

                </SelectItem>

            ))}

        </SelectContent>

    </Select>


```



```
    )})
```

```
  </SelectContent>
```

```
</Select>
```

```
    {errors.mode && <p className="text-xs text-destructive flex items-center gap-1"><AlertCircle className="w-3 h-3" />{errors.mode}</p>}
```

```
  </div>
```

```
<Button type="submit" className="w-full text-sm" disabled={loading}>
```

```
  {loading ? (
```

```
    <>
```

```
      <Loader2 className="w-4 h-4 mr-2 animate-spin" />
```

```
      Simulating...
```

```
    </>
```

```
  ) : (
```

```
    <>
```

```
      <Zap className="w-4 h-4 mr-2" />
```

```
      Simulate Transaction
```

```
    </>
```

```
  )}
```

```
</Button>
```

```
</form>
```

```
  )}
```

```
</DialogContent>
```

```
</Dialog>
```

```
);
```

```
};
```

```
export default TransactionSimulator;
```

File: credit-card-fraud-detection/frontend/src/components/dashboard/UserDashboard.jsx

```
import { useEffect, useState } from "react";
import { useApp } from "@context/AppContext";
import { transactionApi, alertsApi } from "@lib/api";
import { Badge } from "@components/ui/badge";
import { Button } from "@components/ui/button";
import {
  Table,
  TableHeader,
  TableRow,
  TableHead,
  TableBody,
  TableCell,
} from "@components/ui/table";
import { Shield, LogOut, Flag, AlertTriangle, Bell, Zap } from "lucide-react";
import { useNavigate } from "react-router-dom";
import { toast } from "sonner";
import TransactionSimulator from "../TransactionSimulator";

const riskBadge = (level) => {
  level = level.toLowerCase();
```

```

    if (level === "fraud" || level === "flagged")

      return <Badge className="bg-destructive/20 text-destructive border-destructive/30">Fraud</Badge>;

    if (level === "suspicious")

      return <Badge className="bg-warning/20 text-warning border-warning/30">Suspicious</Badge>;

    return <Badge className="bg-success/20 text-success border-success/30">Safe</Badge>;

  };

```

```

const UserDashboard = () => {

  const { currentUser, logout } = useApp();

  const [transactions, setTransactions] = useState([]);

  const [alerts, setAlerts] = useState([]);

  const [stats, setStats] = useState({ total: 0, fraud: 0, suspicious: 0 });

  const navigate = useNavigate();

  const [tab, setTab] = useState("overview");

  useEffect(() => {

    const fetchData = async () => {

      if (!currentUser?.user_id) return;

      try {

        if (tab === "overview" || tab === "transactions") {

          const res = await transactionApi.getByUserId(currentUser.user_id);

          const txs = res.transactions || [];

          setTransactions(txs);

```

```
let total = txs.length;

let fraud = 0;

let suspicious = 0;

txs.forEach((t) => {

  const risk = t.risk_label.toLowerCase();

  if (risk === "fraud" || risk === "flagged") fraud++;

  if (risk === "suspicious") suspicious++;

});

setStats({ total, fraud, suspicious });

} else if (tab === "alerts") {

  const alertsRes = await alertsApi.getAlerts(currentUser.user_id);

  setAlerts(alertsRes.alerts || []);

}

} catch (err) {

  console.error("Error fetching data", err);

}

};

fetchData();

const interval = setInterval(fetchData, 15000);

return () => clearInterval(interval);

}, [currentUser, tab]);

const handleLogout = () => {

  logout();
```

```

navigate("/");

};

const handleReport = async (id) => {

  try {

    await transactionApi.report(id);

    setTransactions((prev) =>

      prev.map((t) => (t.trans_id === id ? { ...t, reported: true } : t))

    );

    toast.success("Thank you! Your feedback has been recorded.");

  } catch (error) {

    console.error("Error reporting transaction:", error);

    const errorMsg = error.response?.data?.detail || "Failed to report transaction";

    toast.error(errorMsg);

  }

};

```

```

if (!currentUser) return null;

```

```

return (

  <div className="min-h-screen flex flex-col md:flex-row bg-background">

    {/* Sidebar */}

    <aside className="hidden md:flex md:w-64 sticky top-0 h-screen border-b md:border-b-0 md:border-r border-border bg-card flex-col">

      <div className="p-4 border-b border-border">

        <div className="flex items-center gap-3 mb-3">

```

```
<div className="w-10 h-10 rounded-lg bg-primary/20 flex items-center justify-center">
```

```
  <Shield className="w-5 h-5 text-primary" />
```

```
</div>
```

```
<span className="font-heading font-bold text-sm">My Dashboard</span>
```

```
</div>
```

```
<div className="bg-secondary/50 rounded-lg p-3 text-xs">
```

```
  <p className="text-muted-foreground text-xs mb-1">Logged in as</p>
```

```
  <p className="font-semibold text-sm">{currentUser.name}</p>
```

```
  <p className="text-muted-foreground text-xs truncate">{currentUser.email}</p>
```

```
</div>
```

```
</div>
```

```
<nav className="flex-1 p-4 space-y-2 overflow-y-auto">
```

```
  <button
```

```
    onClick={() => setTab("overview")}>
```

```
    className={`w-full flex items-center gap-3 px-3 py-2 rounded-lg text-sm transition-all text-left ${
```

```
      tab === "overview"
```

```
        ? "bg-primary/15 text-primary"
```

```
        : "text-muted-foreground hover:bg-secondary"
```

```
    }`>
```

```
>
```

```
  <Zap className="w-4 h-4" /> Overview
```

```
</button>
```

```
<button
```

```
  onClick={() => setTab("transactions")}>
```

```

        className={`w-full flex items-center gap-3 px-3 py-2 rounded-lg text-sm transition-all text-left ${
            tab === "transactions"
                ? "bg-primary/15 text-primary"
                : "text-muted-foreground hover:bg-secondary"
        }}
    >

    <AlertTriangle className="w-4 h-4" /> All Transactions
</button>

<button
    onClick={() => setTab("alerts")}
    className={`w-full flex items-center gap-3 px-3 py-2 rounded-lg text-sm transition-all text-left ${
        tab === "alerts"
            ? "bg-primary/15 text-primary"
            : "text-muted-foreground hover:bg-secondary"
        }}
    >

    <Bell className="w-4 h-4" /> Alerts ({stats.fraud + stats.suspicious})
</button>

</nav>

<div className="p-4 border-t border-border space-y-2">
    <TransactionSimulator userId={currentUser.user_id} />

    <Button
        variant="ghost"
        className="w-full justify-start text-muted-foreground gap-2 text-sm"

```

```

        onClick={handleLogout}

    >

    <LogOut className="w-4 h-4" /> Sign Out

</Button>

</div>

</aside>

{/* Mobile Header */}

<div className="md:hidden border-b border-border bg-card p-3 flex items-center
justify-between">

    <div className="flex items-center gap-2">

        <Shield className="w-5 h-5 text-primary" />

        <div className="flex flex-col">

            <span className="font-heading font-bold text-sm">Dashboard</span>

            <span className="text-xs text-muted-foreground">{currentUser.name}</span>

        </div>

    </div>

    <Button variant="ghost" className="text-muted-foreground text-xs p-2" onClick=
{handleLogout}>

        <LogOut className="w-4 h-4" />

    </Button>

</div>

{/* Mobile Tabs */}

<div className="md:hidden border-b border-border bg-card flex gap-2 p-2 overflow-
x-auto">

    <button

```



```
onClick={() => setTab("overview")}  
  
className={`flex-shrink-0 py-2 px-3 rounded text-xs transition-all whitespace-  
nowrap ${  
  tab === "overview" ? "bg-primary/15 text-primary" : "text-muted-foreground"  
}}  
  
>  
  
  Overview  
  
</button>  
  
<button  
  
  onClick={() => setTab("transactions")}  
  
  className={`flex-shrink-0 py-2 px-3 rounded text-xs transition-all whitespace-  
nowrap ${  
    tab === "transactions" ? "bg-primary/15 text-primary" : "text-muted-foreground"  
  }}  
  
>  
  
    Transactions  
  
</button>  
  
<button  
  
  onClick={() => setTab("alerts")}  
  
  className={`flex-shrink-0 py-2 px-3 rounded text-xs transition-all whitespace-  
nowrap ${  
    tab === "alerts" ? "bg-primary/15 text-primary" : "text-muted-foreground"  
  }}  
  
>  
  
    Alerts ({stats.fraud + stats.suspicious})  
  
</button>  
  
</div>
```

```
    { /* Main Content */ }

    <main className="flex-1 p-3 md:p-6 overflow-auto">

      { tab === "overview" && (

        <div className="space-y-4 md:space-y-6">

          <div className="glass-card rounded-xl p-4 md:p-6">

            <h2 className="font-heading font-semibold text-lg md:text-xl mb-2">Welcome,
{currentUser.name}!</h2>

            <p className="text-sm md:text-base text-muted-foreground">Monitor your
transactions and account security</p>

          </div>

          <div className="grid grid-cols-1 md:grid-cols-3 gap-3 md:gap-6">

            <div className="glass-card rounded-xl p-4 md:p-6 flex flex-col justify-center">

              <h3 className="text-xs md:text-sm font-medium text-muted-foreground">Total
Transactions</h3>

              <p className="text-3xl font-heading font-bold mt-2">{stats.total}</p>

            </div>

            <div className="glass-card rounded-xl p-4 md:p-6 flex flex-col justify-center">

              <h3 className="text-xs md:text-sm font-medium text-
warning">Suspicious</h3>

              <p className="text-3xl font-heading font-bold mt-2 text-warning">
{stats.suspicious}</p>

            </div>

            <div className="glass-card rounded-xl p-4 md:p-6 flex flex-col justify-center">

              <h3 className="text-xs md:text-sm font-medium text-
destructive">Fraudulent</h3>
```

```
<p className="text-3xl font-heading font-bold mt-2 text-destructive">
{stats.fraud}</p>
```

```
</div>
```

```
</div>
```

```
<TransactionSimulator userId={currentUser.user_id} showFullButton={true} />
```

```
<div className="glass-card rounded-xl overflow-hidden">
```

```
<div className="p-4 border-b border-border">
```

```
<h2 className="font-heading font-semibold text-lg">Recent Transactions</h2>
```

```
</div>
```

```
<Table>
```

```
<TableHeader>
```

```
<TableRow className="border-border">
```

```
<TableHead>Amount</TableHead>
```

```
<TableHead className="hidden md:table-cell">Location</TableHead>
```

```
<TableHead className="hidden sm:table-cell">Device</TableHead>
```

```
<TableHead className="hidden lg:table-cell">Mode</TableHead>
```

```
<TableHead>Risk</TableHead>
```

```
<TableHead className="hidden md:table-cell">Date</TableHead>
```

```
<TableHead>Action</TableHead>
```

```
</TableRow>
```

```
</TableHeader>
```

```
<TableBody>
```

```
{transactions.length === 0 ? (
```

```
<TableRow>
```

```
<TableCell colSpan={7} className="h-24 text-center text-muted-foreground text-xs md:text-sm">
```

No transactions yet. Create one to test the system.

```
</TableCell>
```

```
</TableRow>
```

```
): (
```

```
transactions.map((t) => (
```

```
<TableRow key={t.trans_id} className="border-border hover:bg-secondary/50">
```

```
<TableCell className="font-semibold text-xs md:text-sm">₹  
{t.amount.toFixed(0)}</TableCell>
```

```
<TableCell className="hidden md:table-cell text-xs">{t.location}  
</TableCell>
```

```
<TableCell className="hidden sm:table-cell text-xs">{t.device.substring(0,  
8)}</TableCell>
```

```
<TableCell className="hidden lg:table-cell text-xs">{t.mode.substring(0,  
10)}</TableCell>
```

```
<TableCell className="text-xs">{riskBadge(t.risk_label === "Flagged" ?  
"Fraud" : t.risk_label)}</TableCell>
```

```
<TableCell className="hidden md:table-cell text-xs text-muted-  
foreground">
```

```
{new Date(t.timestamp).toLocaleDateString()}
```

```
</TableCell>
```

```
<TableCell>
```

```
{t.reported ? (
```

```
<Badge className="bg-primary/20 text-primary border-primary/30 text-  
xs">Reported</Badge>
```

```
): (
```

```
<Button
```

```

        variant="ghost"

        size="sm"

        onClick={() => handleReport(t.trans_id)}

        className="text-muted-foreground hover:text-warning gap-0.5 text-xs
md:text-sm p-1 md:p-2 h-auto"

    >

        <Flag className="w-3 h-3" /> <span className="hidden
md:inline">Report</span>

    </Button>

    )}

</TableCell>

</TableRow>

))

)}

</TableBody>

</Table>

</div>

</div>

)}

```

```

{tab === "transactions" && (
  <div className="space-y-6">
    <div className="glass-card rounded-xl overflow-hidden">
      <div className="p-4 border-b border-border">
        <h2 className="font-heading font-semibold text-lg">All Transactions</h2>
      </div>

```

```
<Table>
```

```
<TableHeader>
```

```
<TableRow className="border-border">
```

```
<TableHead>Amount</TableHead>
```

```
<TableHead className="hidden md:table-cell">Location</TableHead>
```

```
<TableHead className="hidden sm:table-cell">Device</TableHead>
```

```
<TableHead className="hidden lg:table-cell">Mode</TableHead>
```

```
<TableHead>Risk</TableHead>
```

```
<TableHead className="hidden md:table-cell">Date</TableHead>
```

```
<TableHead>Action</TableHead>
```

```
</TableRow>
```

```
</TableHeader>
```

```
<TableBody>
```

```
{transactions.length === 0 ? (
```

```
<TableRow>
```

```
<TableCell colspan={7} className="h-24 text-center text-muted-foreground  
text-xs md:text-sm">
```

```
  No transactions yet.
```

```
</TableCell>
```

```
</TableRow>
```

```
): (
```

```
  transactions.map((t) => (
```

```
<TableRow key={t.trans_id} className="border-border hover:bg-  
secondary/50">
```

```
<TableCell className="font-semibold text-xs md:text-sm">₹  
{t.amount.toFixed(0)}</TableCell>
```

```

        <TableCell className="hidden md:table-cell text-xs">{t.location}
</TableCell>

        <TableCell className="hidden sm:table-cell text-xs">{t.device.substring(0,
8)}</TableCell>

        <TableCell className="hidden lg:table-cell text-xs">{t.mode.substring(0,
10)}</TableCell>

        <TableCell className="text-xs">{riskBadge(t.risk_label === "Flagged" ?
"Fraud" : t.risk_label)}</TableCell>

        <TableCell className="hidden md:table-cell text-xs text-muted-
foreground">

            {new Date(t.timestamp).toLocaleDateString()}

        </TableCell>

        <TableCell>

            {t.reported ? (

                <Badge className="bg-primary/20 text-primary border-primary/30 text-
xs">Reported</Badge>

            ) : (

                <Button

                    variant="ghost"

                    size="sm"

                    onClick={() => handleReport(t.trans_id)}

                    className="text-muted-foreground hover:text-warning gap-0.5 text-xs
md:text-sm p-1 md:p-2 h-auto"

                >

                    <Flag className="w-3 h-3" /> <span className="hidden
md:inline">Report</span>

                </Button>

            )}

        </TableCell>

```

```
</TableRow>
```

```
))
```

```
}}
```

```
</TableBody>
```

```
</Table>
```

```
</div>
```

```
</div>
```

```
}}
```

```
{tab === "alerts" && (
```

```
<div className="space-y-6">
```

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-3 md:gap-6">
```

```
<div className="glass-card rounded-xl p-4 md:p-6 flex flex-col justify-center  
border-l-4 border-l-destructive">
```

```
<h3 className="text-xs md:text-sm font-medium text-destructive"> Fraud  
Alerts</h3>
```

```
<p className="text-2xl md:text-3xl font-heading font-bold mt-2 text-  
destructive">{stats.fraud}</p>
```

```
</div>
```

```
<div className="glass-card rounded-xl p-4 md:p-6 flex flex-col justify-center  
border-l-4 border-l-warning">
```

```
<h3 className="text-xs md:text-sm font-medium text-warning"> Suspicious  
Alerts</h3>
```

```
<p className="text-2xl md:text-3xl font-heading font-bold mt-2 text-warning">  
{stats.suspicious}</p>
```

```
</div>
```

```
</div>
```



```
<div className="glass-card rounded-xl overflow-hidden">
```

```
<div className="p-3 md:p-4 border-b border-border">
```

```
<h2 className="font-heading font-semibold text-base md:text-lg flex items-center gap-2">
```

```
<AlertTriangle className="w-4 h-4 md:w-5 md:h-5 text-destructive" />
```

```
Security Alerts ({alerts.length})
```

```
</h2>
```

```
</div>
```

```
{alerts.length === 0 ? (
```

```
<div className="p-8 md:p-12 text-center">
```

```
<Shield className="w-10 h-10 md:w-12 md:h-12 text-success opacity-50 mx-auto mb-4" />
```

```
<h3 className="text-base md:text-lg font-semibold text-foreground">All Clear! 🎉</h3>
```

```
<p className="text-xs md:text-sm text-muted-foreground mt-2">No fraudulent or suspicious transactions detected.</p>
```

```
</div>
```

```
): (
```

```
<div className="overflow-x-auto">
```

```
<Table className="text-xs md:text-sm">
```

```
<TableHeader>
```

```
<TableRow className="border-border">
```

```
<TableHead className="text-xs">Date</TableHead>
```

```
<TableHead className="text-xs">Amount</TableHead>
```

```
<TableHead className="text-xs hidden md:table-cell">Location</TableHead>
```

```
<TableHead className="text-xs hidden lg:table-cell">Device</TableHead>
```

```

<TableHead className="text-xs hidden xl:table-cell">Mode</TableHead>

<TableHead className="text-xs">Type</TableHead>

<TableHead className="text-xs hidden sm:table-cell">Risk</TableHead>

</TableRow>

</TableHeader>

<TableBody>

  {alerts.map((alert) => (

    <TableRow key={alert.trans_id} className="border-border hover:bg-
secondary/50">

      <TableCell className="text-xs text-muted-foreground">

        {new Date(alert.timestamp).toLocaleDateString()}

      </TableCell>

      <TableCell className="font-semibold text-xs">₹
{alert.amount.toFixed(0)} </TableCell>

      <TableCell className="hidden md:table-cell text-xs">{alert.location}
</TableCell>

      <TableCell className="hidden lg:table-cell text-xs">{alert.device}
</TableCell>

      <TableCell className="hidden xl:table-cell text-xs">{alert.mode}
</TableCell>

      <TableCell className="text-xs min-w-[80px]">
{riskBadge(alert.risk_label)} </TableCell>

      <TableCell className="hidden sm:table-cell">

        <div className="flex items-center gap-1">

          <div className="w-10 bg-secondary rounded-full h-1.5">

            <div

              className={`h-full rounded-full ${

                alert.risk_score > 0.7

```

? "bg-destructive"

: alert.risk_score > 0.4

? "bg-warning"

: "bg-success"

}`}

style={{ width: `\${(alert.risk\_score || 0) \* 100}%` }}

/>

</div>

{((alert.risk_score || 0) * 100).toFixed(0)}%

</div>

</TableCell>

</TableRow>

)))

</TableBody>

</Table>

</div>

))

</div>

</div>

))

</main>

</div>

);

};

```
export default UserDashboard;
```

File: credit-card-fraud-detection/frontend/src/components/ui/accordion.jsx

```
import * as React from "react";
```

```
import * as AccordionPrimitive from "@radix-ui/react-accordion";
```

```
import { ChevronDown } from "lucide-react";
```

```
import { cn } from "@lib/utils";
```

```
const Accordion = AccordionPrimitive.Root;
```

```
const AccordionItem = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <AccordionPrimitive.Item
```

```
    ref={ref}
```

```
    className={cn("border-b", className)}
```

```
    {...props}
```

```
  />
```

```
));
```

```
AccordionItem.displayName = "AccordionItem";
```

```
const AccordionTrigger = React.forwardRef(
```

```
  ({ className, children, ...props }, ref) => (
```

```
    <AccordionPrimitive.Header className="flex">
```

```

<AccordionPrimitive.Trigger
  ref={ref}
  className={cn(
    "flex flex-1 items-center justify-between py-4 font-medium transition-all
    hover:underline [&[data-state=open]>svg]:rotate-180",
    className,
  )}
  {...props}
>
  {children}
  <ChevronDown className="h-4 w-4 shrink-0 transition-transform duration-200" />
</AccordionPrimitive.Trigger>
</AccordionPrimitive.Header>
),
);

```

```

AccordionTrigger.displayName = AccordionPrimitive.Trigger.displayName;

```

```

const AccordionContent = React.forwardRef(
  ({ className, children, ...props }, ref) => (
    <AccordionPrimitive.Content
      ref={ref}
      className="overflow-hidden text-sm transition-all data-[state=closed]:animate-
      accordion-up data-[state=open]:animate-accordion-down"
      {...props}
    >
      <div className={cn("pb-4 pt-0", className)}>{children}</div>
    </div>
  )
);

```

```
    </AccordionPrimitive.Content>

  ),

);

AccordionContent.displayName = AccordionPrimitive.Content.displayName;

export { Accordion, AccordionItem, AccordionTrigger, AccordionContent };
```

File: credit-card-fraud-detection/frontend/src/components/ui/alert-dialog.jsx

```
import * as React from "react";

import * as AlertDialogPrimitive from "@radix-ui/react-alert-dialog";

import { cn } from "@lib/utils";

import { buttonVariants } from "@components/ui/button";

const AlertDialog = AlertDialogPrimitive.Root;

const AlertDialogTrigger = AlertDialogPrimitive.Trigger;

const AlertDialogPortal = AlertDialogPrimitive.Portal;

const AlertDialogOverlay = React.forwardRef(({ className, ...props }, ref) => (

  <AlertDialogPrimitive.Overlay

    className={cn(
```

```

    "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-
[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",

    className,

  )}

  {...props}

  ref={ref}

/>

));

```

```

AlertDialogOverlay.displayName = AlertDialogPrimitive.Overlay.displayName;

```

```

const AlertDialogContent = React.forwardRef(({ className, ...props }, ref) => (

  <AlertDialogPortal>

    <AlertDialogOverlay />

    <AlertDialogPrimitive.Content

      ref={ref}

      className={cn(

        "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-
[-50%] gap-4 border bg-background p-6 shadow-lg duration-200 data-
[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0
data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95
data-[state=closed]:slide-out-to-left-1/2 data-[state=closed]:slide-out-to-top-[48%] data-
[state=open]:slide-in-from-left-1/2 data-[state=open]:slide-in-from-top-[48%] sm:rounded-
lg",

        className,

      )}

      {...props}

    />

  </AlertDialogPortal>

```

```
));
```

```
AlertDialogContent.displayName = AlertDialogPrimitive.Content.displayName;
```

```
const AlertDialogHeader = ({ className, ...props }) => (
```

```
  <div
```

```
    className={cn(
```

```
      "flex flex-col space-y-2 text-center sm:text-left",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
  />
```

```
);
```

```
AlertDialogHeader.displayName = "AlertDialogHeader";
```

```
const AlertDialogFooter = ({ className, ...props }) => (
```

```
  <div
```

```
    className={cn(
```

```
      "flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
  />
```

```
);
```

```
AlertDialogFooter.displayName = "AlertDialogFooter";
```

```
const AlertDialogTitle = React.forwardRef(({ className, ...props }, ref) => (
```



```
<AlertDialogPrimitive.Title
  ref={ref}
  className={cn("text-lg font-semibold", className)}
  {...props}
/>

));

AlertDialogTitle.displayName = AlertDialogPrimitive.Title.displayName;
```

```
const AlertDialogDescription = React.forwardRef(
  ({ className, ...props }, ref) => (
    <AlertDialogPrimitive.Description
      ref={ref}
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  ),
);

AlertDialogDescription.displayName =
  AlertDialogPrimitive.Description.displayName;

const AlertDialogAction = React.forwardRef(({ className, ...props }, ref) => (
  <AlertDialogPrimitive.Action
    ref={ref}
    className={cn(buttonVariants(), className)}
    {...props}
  />
```

```
));
```

```
AlertDialogAction.displayName = AlertDialogPrimitive.Action.displayName;
```

```
const AlertDialogCancel = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <AlertDialogPrimitive.Cancel
```

```
    ref={ref}
```

```
    className={cn(
```

```
      buttonVariants({ variant: "outline" }),
```

```
      "mt-2 sm:mt-0",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
  />
```

```
));
```

```
AlertDialogCancel.displayName = AlertDialogPrimitive.Cancel.displayName;
```

```
export {
```

```
  AlertDialog,
```

```
  AlertDialogPortal,
```

```
  AlertDialogOverlay,
```

```
  AlertDialogTrigger,
```

```
  AlertDialogContent,
```

```
  AlertDialogHeader,
```

```
  AlertDialogFooter,
```

```
  AlertDialogTitle,
```

```
  AlertDialogDescription,
```

```
AlertDialogAction,  
AlertDialogCancel,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/alert.jsx

```
import * as React from "react";  
  
import { cva } from "class-variance-authority";  
  
import { cn } from "@lib/utls";  
  
const alertVariants = cva(  
  "relative w-full rounded-lg border p-4 [&>svg~*]:pl-7 [&>svg+div]:translate-y-[-3px]  
  [&>svg]:absolute [&>svg]:left-4 [&>svg]:top-4 [&>svg]:text-foreground",  
  {  
    variants: {  
      variant: {  
        default: "bg-background text-foreground",  
        destructive:  
          "border-destructive/50 text-destructive dark:border-destructive [&>svg]:text-  
destructive",  
      },  
    },  
    defaultVariants: {  
      variant: "default",  
    },  
  },  
);
```

```
    },  
  },  
);  
  
const Alert = React.forwardRef(({ className, variant, ...props }, ref) => (  
  <div  
    ref={ref}  
    role="alert"  
    className={cn(alertVariants({ variant }), className)}  
    {...props}  
  />  
));  
Alert.displayName = "Alert";
```

```
const AlertTitle = React.forwardRef(({ className, ...props }, ref) => (  
  <h5  
    ref={ref}  
    className={cn("mb-1 font-medium leading-none tracking-tight", className)}  
    {...props}  
  />  
));  
AlertTitle.displayName = "AlertTitle";
```

```
const AlertDescription = React.forwardRef(({ className, ...props }, ref) => (  
  <div  
    ref={ref}
```

```
      className={cn("text-sm [&_p]:leading-relaxed", className)}  
      {...props}  
    />  
  ));  
  AlertDescription.displayName = "AlertDescription";
```

```
export { Alert, AlertTitle, AlertDescription };
```

File: credit-card-fraud-detection/frontend/src/components/ui/aspect-ratio.jsx

```
import * as AspectRatioPrimitive from "@radix-ui/react-aspect-ratio";  
  
const AspectRatio = AspectRatioPrimitive.Root;  
  
export { AspectRatio };
```

File: credit-card-fraud-detection/frontend/src/components/ui/avatar.jsx

```
import * as React from "react";  
import * as AvatarPrimitive from "@radix-ui/react-avatar";  
  
import { cn } from "@lib/utils";  
  
const Avatar = React.forwardRef(({ className, ...props }, ref) => (
```

```

<AvatarPrimitive.Root
  ref={ref}
  className={cn(
    "relative flex h-10 w-10 shrink-0 overflow-hidden rounded-full",
    className,
  )}
  {...props}
/>

));

Avatar.displayName = AvatarPrimitive.Root.displayName;

const AvatarImage = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Image
    ref={ref}
    className={cn("aspect-square h-full w-full", className)}
    {...props}
  />
));

AvatarImage.displayName = AvatarPrimitive.Image.displayName;

const AvatarFallback = React.forwardRef(({ className, ...props }, ref) => (
  <AvatarPrimitive.Fallback
    ref={ref}
    className={cn(
      "flex h-full w-full items-center justify-center rounded-full bg-muted",
      className,

```

```
    )}

    {...props}

  />

));

AvatarFallback.displayName = AvatarPrimitive.Fallback.displayName;

export { Avatar, AvatarImage, AvatarFallback };
```

File: credit-card-fraud-detection/frontend/src/components/ui/badge.jsx

```
import * as React from "react";

import { cva } from "class-variance-authority";

import { cn } from "@lib/utils";

const badgeVariants = cva(
  "inline-flex items-center rounded-full border px-2.5 py-0.5 text-xs font-semibold transition-colors focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2",
  {
    variants: {
      variant: {
        default:
          "border-transparent bg-primary text-primary-foreground",
        secondary:
          "border-transparent bg-secondary text-secondary-foreground",
```

```
destructive:
  "border-transparent bg-destructive text-destructive-foreground",
outline: "text-foreground",
},
},
defaultVariants: {
  variant: "default",
},
},
);

function Badge({ className, variant, ...props }) {
  return (
    <div className={cn(badgeVariants({ variant }), className)} {...props} />
  );
}

export { Badge, badgeVariants };
```

File: credit-card-fraud-detection/frontend/src/components/ui/breadcrumb.jsx

```
import * as React from "react";
import { Slot } from "@radix-ui/react-slot";
import { ChevronRight, MoreHorizontal } from "lucide-react";
```



```
import { cn } from "@lib/utils";
```

```
const Breadcrumb = React.forwardRef(({ ...props }, ref) => (
```

```
  <nav ref={ref} aria-label="breadcrumb" {...props} />
```

```
));
```

```
Breadcrumb.displayName = "Breadcrumb";
```

```
const BreadcrumbList = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <ol
```

```
    ref={ref}
```

```
    className={cn(
```

```
      "flex flex-wrap items-center gap-1.5 break-words text-sm text-muted-foreground  
sm:gap-2.5",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
  />
```

```
));
```

```
BreadcrumbList.displayName = "BreadcrumbList";
```

```
const BreadcrumbItem = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <li
```

```
    ref={ref}
```

```
    className={cn("inline-flex items-center gap-1.5", className)}
```

```
    {...props}
```

```
  />
```

```
));
```

```
BreadcrumbItem.displayName = "BreadcrumbItem";
```

```
const BreadcrumbLink = React.forwardRef(
```

```
  ({ asChild, className, ...props }, ref) => {
```

```
    const Comp = asChild ? Slot : "a";
```

```
    return (
```

```
      <Comp
```

```
        ref={ref}
```

```
        className={cn("transition-colors hover:text-foreground", className)}
```

```
        {...props}
```

```
      />
```

```
    );
```

```
  },
```

```
);
```

```
BreadcrumbLink.displayName = "BreadcrumbLink";
```

```
const BreadcrumbPage = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <span
```

```
    ref={ref}
```

```
    role="link"
```

```
    aria-disabled="true"
```

```
    aria-current="page"
```

```
    className={cn("font-normal text-foreground", className)}
```

```
    {...props}
```

```

/>

));

BreadcrumbPage.displayName = "BreadcrumbPage";

const BreadcrumbSeparator = ({ children, className, ...props }) => (
  <li
    role="presentation"
    aria-hidden="true"
    className={cn("[&>svg]:size-3.5", className)}
    {...props}
  >
    {children ?? <ChevronRight />}
  </li>
);

BreadcrumbSeparator.displayName = "BreadcrumbSeparator";

```

```

const BreadcrumbEllipsis = ({ className, ...props }) => (
  <span
    role="presentation"
    aria-hidden="true"
    className={cn("flex h-9 w-9 items-center justify-center", className)}
    {...props}
  >
    <MoreHorizontal className="h-4 w-4" />
    <span className="sr-only">More</span>
  </span>
);

```

```
);  
  
BreadcrumbEllipsis.displayName = "BreadcrumbElipssis";  
  
export {  
  Breadcrumb,  
  BreadcrumbList,  
  BreadcrumbItem,  
  BreadcrumbLink,  
  BreadcrumbPage,  
  BreadcrumbSeparator,  
  BreadcrumbEllipsis,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/button.jsx

```
import * as React from "react";  
  
import { Slot } from "@radix-ui/react-slot";  
  
import { cva } from "class-variance-authority";  
  
import { cn } from "@lib/utls";  
  
const buttonVariants = cva(  
  "inline-flex items-center justify-center gap-2 whitespace-nowrap rounded-md text-sm font-  
medium ring-offset-background transition-colors focus-visible:outline-none focus-  
visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 disabled:pointer-events-none  
disabled:opacity-50 [&_svg]:pointer-events-none [&_svg]:size-4 [&_svg]:shrink-0",
```

```
{
  variants: {
    variant: {
      default: "bg-primary text-primary-foreground hover:bg-primary/90",
      destructive:
        "bg-destructive text-destructive-foreground hover:bg-destructive/90",
      outline:
        "border border-input bg-background hover:bg-accent hover:text-accent-foreground",
      secondary:
        "bg-secondary text-secondary-foreground hover:bg-secondary/80",
      ghost: "hover:bg-accent hover:text-accent-foreground",
      link: "text-primary underline-offset-4 hover:underline",
    },
    size: {
      default: "h-10 px-4 py-2",
      sm: "h-9 rounded-md px-3",
      lg: "h-11 rounded-md px-8",
      icon: "h-10 w-10",
    },
  },
  defaultVariants: {
    variant: "default",
    size: "default",
  },
};
```

```
const Button = React.forwardRef(  
  ({ className, variant, size, asChild = false, ...props }, ref) => {  
    const Comp = asChild ? Slot : "button";  
    return (  
      <Comp  
        className={cn(buttonVariants({ variant, size, className })))}  
        ref={ref}  
        {...props}  
      />  
    );  
  },  
);  
Button.displayName = "Button";  
  
export { Button, buttonVariants };
```

File: credit-card-fraud-detection/frontend/src/components/ui/calendar.jsx

```
import * as React from "react";  
import { ChevronLeft, ChevronRight } from "lucide-react";  
import { DayPicker } from "react-day-picker";  
  
import { cn } from "@lib/utls";  
import { buttonVariants } from "@components/ui/button";
```

```

function Calendar({ className, classNames, showOutsideDays = true, ...props }) {
  return (
    <DatePicker
      showOutsideDays={showOutsideDays}
      className={cn("p-3", className)}
      classNames={{
        months: "flex flex-col sm:flex-row space-y-4 sm:space-x-4 sm:space-y-0",
        month: "space-y-4",
        caption: "flex justify-center pt-1 relative items-center",
        caption_label: "text-sm font-medium",
        nav: "space-x-1 flex items-center",
        nav_button: cn(
          buttonVariants({ variant: "outline" }),
          "h-7 w-7 bg-transparent p-0 opacity-50 hover:opacity-100",
        ),
        nav_button_previous: "absolute left-1",
        nav_button_next: "absolute right-1",
        table: "w-full border-collapse space-y-1",
        head_row: "flex",
        head_cell:
          "text-muted-foreground rounded-md w-9 font-normal text-[0.8rem]",
        row: "flex w-full mt-2",
        cell: "h-9 w-9 text-center text-sm p-0 relative [&:has([aria-selected].day-range-end)]:rounded-r-md [&:has([aria-selected].day-outside)]:bg-accent/50 [&:has([aria-selected])]:bg-accent first:[&:has([aria-selected])]:rounded-l-md last:[&:has([aria-selected])]:rounded-r-md focus-within:relative focus-within:z-20",
      }}
    />
  )
}

```

```

day: cn(
  buttonVariants({ variant: "ghost" }),
  "h-9 w-9 p-0 font-normal aria-selected:opacity-100",
),
day_range_end: "day-range-end",
day_selected:
  "bg-primary text-primary-foreground hover:bg-primary hover:text-primary-foreground focus:bg-primary focus:text-primary-foreground",
day_today: "bg-accent text-accent-foreground",
day_outside:
  "day-outside text-muted-foreground opacity-50 aria-selected:bg-accent/50 aria-selected:text-muted-foreground aria-selected:opacity-30",
day_disabled: "text-muted-foreground opacity-50",
day_range_middle:
  "aria-selected:bg-accent aria-selected:text-accent-foreground",
day_hidden: "invisible",
...classNames,
}}
components={{
  IconLeft: ({ ..._props }) => <ChevronLeft className="h-4 w-4" />,
  IconRight: ({ ..._props }) => <ChevronRight className="h-4 w-4" />,
}}
{...props}
/>
);
}

```



```
Calendar.displayName = "Calendar";
```

```
export { Calendar };
```

File: credit-card-fraud-detection/frontend/src/components/ui/card.jsx

```
import * as React from "react";
```

```
import { cn } from "@lib/utils";
```

```
const Card = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <div
```

```
    ref={ref}
```

```
    className={cn(
```

```
      "rounded-lg border bg-card text-card-foreground shadow-sm",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
  />
```

```
));
```

```
Card.displayName = "Card";
```

```
const CardHeader = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <div
```

```
    ref={ref}
```

```
      className={cn("flex flex-col space-y-1.5 p-6", className)}
      {...props}
    />
  ));
  CardHeader.displayName = "CardHeader";

const CardTitle = React.forwardRef(({ className, ...props }, ref) => (
  <h3
    ref={ref}
    className={cn(
      "text-2xl font-semibold leading-none tracking-tight",
      className,
    )}
    {...props}
  />
));
CardTitle.displayName = "CardTitle";

const CardDescription = React.forwardRef(({ className, ...props }, ref) => (
  <p
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
));
CardDescription.displayName = "CardDescription";
```

```
const CardContent = React.forwardRef(({ className, ...props }, ref) => (  
  <div ref={ref} className={cn("p-6 pt-0", className)} {...props} />  
));
```

```
CardContent.displayName = "CardContent";
```

```
const CardFooter = React.forwardRef(({ className, ...props }, ref) => (  
  <div  
    ref={ref}  
    className={cn("flex items-center p-6 pt-0", className)}  
    {...props}  
  />  
));
```

```
CardFooter.displayName = "CardFooter";
```

```
export {  
  Card,  
  CardHeader,  
  CardFooter,  
  CardTitle,  
  CardDescription,  
  CardContent,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/carousel.jsx

```
import * as React from "react";

import useEmblaCarousel from "embla-carousel-react";

import { ArrowLeft, ArrowRight } from "lucide-react";


import { cn } from "@lib/utls";

import { Button } from "@components/ui/button";


const CarouselContext = React.createContext(null);


function useCarousel() {

  const context = React.useContext(CarouselContext);

  if (!context) {
    throw new Error("useCarousel must be used within a <Carousel />");
  }

  return context;
}


const Carousel = React.forwardRef(
  (
    {
      orientation = "horizontal",
      opts,
      setApi,
```

```
    plugins,  
    className,  
    children,  
    ...props  
  },  
  ref,  
) => {  
  const [carouselRef, api] = useEmblaCarousel(  
    {  
      ...opts,  
      axis: orientation === "horizontal" ? "x" : "y",  
    },  
    plugins,  
  );  
  const [canScrollPrev, setCanScrollPrev] = React.useState(false);  
  const [canScrollNext, setCanScrollNext] = React.useState(false);  
  
  const onSelect = React.useCallback((api) => {  
    if (!api) {  
      return;  
    }  
  
    setCanScrollPrev(api.canScrollPrev());  
    setCanScrollNext(api.canScrollNext());  
  }, []);
```

```
const scrollPrev = React.useCallback(() => {  
  api?.scrollPrev();  
}, [api]);
```

```
const scrollNext = React.useCallback(() => {  
  api?.scrollNext();  
}, [api]);
```

```
const handleKeyDown = React.useCallback(  
  (event) => {  
    if (event.key === "ArrowLeft") {  
      event.preventDefault();  
      scrollPrev();  
    } else if (event.key === "ArrowRight") {  
      event.preventDefault();  
      scrollNext();  
    }  
  },  
  [scrollPrev, scrollNext],  
);
```

```
React.useEffect(() => {  
  if (!api || !setApi) {  
    return;  
  }  
},
```

```
    setApi(api);  
  }, [api, setApi]);
```

```
React.useEffect(() => {  
  if (!api) {  
    return;  
  }  

```

```
  onSelect(api);  
  api.on("reInit", onSelect);  
  api.on("select", onSelect);
```

```
  return () => {  
    api?.off("select", onSelect);  
  };  
}, [api, onSelect]);
```

```
return (  
  <CarouselContext.Provider  
    value={{  
      carouselRef,  
      api: api,  
      opts,  
      orientation:  
        orientation || (opts?.axis === "y" ? "vertical" : "horizontal"),  
      scrollPrev,
```

```

        scrollNext,

        canScrollPrev,

        canScrollNext,
    }}
>

<div
  ref={ref}
  onKeyDownCapture={handleKeyDown}
  className={cn("relative", className)}
  role="region"
  aria-roledescription="carousel"
  {...props}
>
  {children}
</div>
</CarouselContext.Provider>

);

},

);

Carousel.displayName = "Carousel";

const CarouselContent = React.forwardRef(({ className, ...props }, ref) => {
  const { carouselRef, orientation } = useCarousel();

  return (
    <div ref={carouselRef} className="overflow-hidden">

```



```

<div
  ref={ref}
  className={cn(
    "flex",
    orientation === "horizontal" ? "-ml-4" : "-mt-4 flex-col",
    className,
  )}
  {...props}
/>
</div>

);

});

CarouselContent.displayName = "CarouselContent";

```

```

const CarouselItem = React.forwardRef(({ className, ...props }, ref) => {
  const { orientation } = useCarousel();

  return (
    <div
      ref={ref}
      role="group"
      aria-roledescription="slide"
      className={cn(
        "min-w-0 shrink-0 grow-0 basis-full",
        orientation === "horizontal" ? "pl-4" : "pt-4",
        className,

```

```

    })
    {...props}
  />

);

});

CarouselItem.displayName = "CarouselItem";

const CarouselPrevious = React.forwardRef(
  ({ className, variant = "outline", size = "icon", ...props }, ref) => {
    const { orientation, scrollPrev, canScrollPrev } = useCarousel();

    return (
      <Button
        ref={ref}
        variant={variant}
        size={size}
        className={cn(
          "absolute h-8 w-8 rounded-full",
          orientation === "horizontal"
            ? "-left-12 top-1/2 -translate-y-1/2"
            : "-top-12 left-1/2 -translate-x-1/2 rotate-90",
          className,
        )}
        disabled={!canScrollPrev}
        onClick={scrollPrev}
        {...props}

```

```

>

<ArrowLeft className="h-4 w-4" />

<span className="sr-only">Previous slide</span>

</Button>

);

},

);

CarouselPrevious.displayName = "CarouselPrevious";

const CarouselNext = React.forwardRef(

  ({ className, variant = "outline", size = "icon", ...props }, ref) => {

    const { orientation, scrollNext, canScrollNext } = useCarousel();

    return (

      <Button

        ref={ref}

        variant={variant}

        size={size}

        className={cn(

          "absolute h-8 w-8 rounded-full",

          orientation === "horizontal"

            ? "-right-12 top-1/2 -translate-y-1/2"

            : "-bottom-12 left-1/2 -translate-x-1/2 rotate-90",

          className,

        )}

        disabled={!canScrollNext}

```

```
onClick={scrollNext}

 {...props}
>

<ArrowRight className="h-4 w-4" />

<span className="sr-only">Next slide</span>

</Button>

);

},

);

CarouselNext.displayName = "CarouselNext";

export {

  Carousel,

  CarouselContent,

  CarouselItem,

  CarouselPrevious,

  CarouselNext,

};
```

File: credit-card-fraud-detection/frontend/src/components/ui/chart.jsx

```
import * as React from "react";

import * as RechartsPrimitive from "recharts";

import { cn } from "@lib/utls";
```

```
// Format: { THEME_NAME: CSS_SELECTOR }
```

```
const THEMES = { light: "", dark: ".dark" };
```

```
const ChartContext = React.createContext(null);
```

```
function useChart() {
```

```
  const context = React.useContext(ChartContext);
```

```
  if (!context) {
```

```
    throw new Error("useChart must be used within a <ChartContainer />");
```

```
  }
```

```
  return context;
```

```
}
```

```
const ChartContainer = React.forwardRef(
```

```
  ({ id, className, children, config, ...props }, ref) => {
```

```
    const uniqueId = React.useId();
```

```
    const chartId = `chart-${id || uniqueId.replace(/:/g, "")}`;
```

```
    return (
```

```
      <ChartContext.Provider value={{ config }}>
```

```
        <div
```

```
          data-chart={chartId}
```

```
          ref={ref}
```

```

className={cn(
  "flex aspect-video justify-center text-xs [&_.recharts-cartesian-axis-tick_text]:fill-
muted-foreground [&_.recharts-cartesian-grid_line[stroke='#ccc']]:stroke-border/50
[&_.recharts-curve.recharts-tooltip-cursor]:stroke-border [&_.recharts-
dot[stroke='#fff']]:stroke-transparent [&_.recharts-layer]:outline-none [&_.recharts-polar-
grid_[stroke='#ccc']]:stroke-border [&_.recharts-radial-bar-background-sector]:fill-muted
[&_.recharts-rectangle.recharts-tooltip-cursor]:fill-muted [&_.recharts-reference-
line_[stroke='#ccc']]:stroke-border [&_.recharts-sector[stroke='#fff']]:stroke-transparent
[&_.recharts-sector]:outline-none [&_.recharts-surface]:outline-none",
  className,
)}
{...props}
>
<ChartStyle id={chartId} config={config} />
<RechartsPrimitive.ResponsiveContainer>
  {children}
</RechartsPrimitive.ResponsiveContainer>
</div>
</ChartContext.Provider>
);
},
);
ChartContainer.displayName = "Chart";

const ChartStyle = ({ id, config }) => {
  const colorConfig = Object.entries(config).filter(
    ([_, config]) => config.theme || config.color,
  );

```

```

if (!colorConfig.length) {

    return null;

}

return (

    <style

        dangerouslySetInnerHTML={{

            __html: Object.entries(THEMES)

                .map(

                    ([theme, prefix]) => `

${prefix} [data-chart=${id}] {

${colorConfig

    .map(([key, itemConfig]) => {

        const color = itemConfig.theme?.[theme] || itemConfig.color;

        return color ? ` --color-${key}: ${color};` : null;

    })

    .join("\n")}

}

`,

        )

        .join("\n"),

    }}

    />

);

};

```

```
const ChartTooltip = RechartsPrimitive.Tooltip;
```

```
const ChartTooltipContent = React.forwardRef(
```

```
(
```

```
{
```

```
  active,
```

```
  payload,
```

```
  className,
```

```
  indicator = "dot",
```

```
  hideLabel = false,
```

```
  hideIndicator = false,
```

```
  label,
```

```
  labelFormatter,
```

```
  labelClassName,
```

```
  formatter,
```

```
  color,
```

```
  nameKey,
```

```
  labelKey,
```

```
},
```

```
ref,
```

```
) => {
```

```
  const { config } = useChart();
```

```
  const tooltipLabel = React.useMemo(() => {
```

```
    if (hideLabel || !payload?.length) {
```



```
    return null;
}
```

```
const [item] = payload;

const key = `${labelKey || item.dataKey || item.name || "value"}`;

const itemConfig = getPayloadConfigFromPayload(config, item, key);

const value =

    !labelKey && typeof label === "string"

    ? config[label]?.label || label

    : itemConfig?.label;
```

```
if (labelFormatter) {

    return (

        <div className={cn("font-medium", labelClassName)}>

            {labelFormatter(value, payload)}

        </div>

    );

}
```

```
if (!value) {

    return null;

}
```

```
return <div className={cn("font-medium", labelClassName)}>{value}</div>;

}, [

    label,
```

```
labelFormatter,  
payload,  
hideLabel,  
labelClassName,  
config,  
labelKey,  
]);
```

```
if (!active || !payload?.length) {  
  return null;  
}
```

```
const nestLabel = payload.length === 1 && indicator !== "dot";
```

```
return (  
  <div  
    ref={ref}  
    className={cn(  
      "grid min-w-[8rem] items-start gap-1.5 rounded-lg border border-border/50 bg-  
background px-2.5 py-1.5 text-xs shadow-xl",  
      className,  
    )}  
  >  
    {!nestLabel ? tooltipLabel : null}  
    <div className="grid gap-1.5">  
      {payload.map((item, index) => {
```

```
const key = `${nameKey || item.name || item.dataKey || "value"}`;
const itemConfig = getPayloadConfigFromPayload(config, item, key);
const indicatorColor = color || item.payload.fill || item.color;
```

```
return (
```

```
  <div
```

```
    key={item.dataKey}
```

```
    className={cn(
```

```
      "flex w-full flex-wrap items-stretch gap-2 [&>svg]:h-2.5 [&>svg]:w-2.5  
[&>svg]:text-muted-foreground",
```

```
      indicator === "dot" && "items-center",
```

```
    )}
```

```
  >
```

```
    {formatter && item?.value !== undefined && item.name ? (
```

```
      formatter(item.value, item.name, item, index, item.payload)
```

```
    ) : (
```

```
      <>
```

```
        {itemConfig?.icon ? (
```

```
          <itemConfig.icon />
```

```
        ) : (
```

```
          !hideIndicator && (
```

```
            <div
```

```
              className={cn(
```

```
                "shrink-0 rounded-[2px] border-[--color-border] bg-[--color-bg]",
```

```
                {
```

```
                  "h-2.5 w-2.5": indicator === "dot",
```

```

        "w-1": indicator === "line",

        "w-0 border-[1.5px] border-dashed bg-transparent":
            indicator === "dashed",

        "my-0.5": nestLabel && indicator === "dashed",

    },

)}

style={{

    "--color-bg": indicatorColor,

    "--color-border": indicatorColor,

}}

/>

)

)}

<div

    className={cn(

        "flex flex-1 justify-between leading-none",

        nestLabel ? "items-end" : "items-center",

    )}

>

    <div className="grid gap-1.5">

        {nestLabel ? tooltipLabel : null}

        <span className="text-muted-foreground">

            {itemConfig?.label || item.name}

        </span>

    </div>

    {item.value && (

```

```
        <span className="font-mono font-medium tabular-nums text-foreground">
```

```
            {item.value.toLocaleString()}
```

```
        </span>
```

```
    )}
```

```
</div>
```

```
</>
```

```
    )}
```

```
</div>
```

```
);
```

```
}}}
```

```
</div>
```

```
</div>
```

```
);
```

```
},
```

```
);
```

```
ChartTooltipContent.displayName = "ChartTooltip";
```

```
const ChartLegend = RechartsPrimitive.Legend;
```

```
const ChartLegendContent = React.forwardRef(
```

```
(
```

```
  { className, hideIcon = false, payload, verticalAlign = "bottom", nameKey },
```

```
  ref,
```

```
) => {
```

```
  const { config } = useChart();
```

```
if (!payload?.length) {  
  return null;  
}
```

```
return (  
  <div  
    ref={ref}  
    className={cn(  
      "flex items-center justify-center gap-4",  
      verticalAlign === "top" ? "pb-3" : "pt-3",  
      className,  
    )}  
  >  
    {payload.map((item) => {  
      const key = `${nameKey || item.dataKey || "value"}`;  
      const itemConfig = getPayloadConfigFromPayload(config, item, key);  
  
      return (  
        <div  
          key={item.value}  
          className={cn(  
            "flex items-center gap-1.5 [&>svg]:h-3 [&>svg]:w-3 [&>svg]:text-muted-  
foreground",  
          )}  
        >
```

```

    {itemConfig?.icon && !hideIcon ? (
      <itemConfig.icon />
    ) : (
      <div
        className="h-2 w-2 shrink-0 rounded-[2px]"
        style={{
          backgroundColor: item.color,
        }}
      />
    )}
    {itemConfig?.label}
  </div>

);

}}

</div>

);

},

);

ChartLegendContent.displayName = "ChartLegend";

// Helper to extract item config from a payload.
function getPayloadConfigFromPayload(config, payload, key) {
  if (typeof payload !== "object" || payload === null) {
    return undefined;
  }
}

```

```

const payloadPayload =
  "payload" in payload &&
  typeof payload.payload === "object" &&
  payload.payload !== null
    ? payload.payload
    : undefined;

let configLabelKey = key;

if (key in payload && typeof payload[key] === "string") {
  configLabelKey = payload[key];
} else if (
  payloadPayload &&
  key in payloadPayload &&
  typeof payloadPayload[key] === "string"
) {
  configLabelKey = payloadPayload[key];
}

return configLabelKey in config ? config[configLabelKey] : config[key];
}

export {
  ChartContainer,
  ChartTooltip,
  ChartTooltipContent,

```



```
ChartLegend,  
ChartLegendContent,  
ChartStyle,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/checkbox.jsx

```
import * as React from "react";  
import * as CheckboxPrimitive from "@radix-ui/react-checkbox";  
import { Check } from "lucide-react";  
  
import { cn } from "@lib/utils";  
  
const Checkbox = React.forwardRef(({ className, ...props }, ref) => (  
  <CheckboxPrimitive.Root  
    ref={ref}  
    className={cn(  
      "peer h-4 w-4 shrink-0 rounded-sm border border-primary ring-offset-background  
data-[state=checked]:bg-primary data-[state=checked]:text-primary-foreground focus-  
visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2  
disabled:cursor-not-allowed disabled:opacity-50",  
      className,  
    )}  
    {...props}  
  >  
    <CheckboxPrimitive.Indicator
```

```
      className={cn("flex items-center justify-center text-current")}
    >
      <Check className="h-4 w-4" />
    </CheckboxPrimitive.Indicator>
  </CheckboxPrimitive.Root>
));
Checkbox.displayName = CheckboxPrimitive.Root.displayName;

export { Checkbox };
```

File: credit-card-fraud-detection/frontend/src/components/ui/collapsible.jsx

```
import * as CollapsiblePrimitive from "@radix-ui/react-collapsible";

const Collapsible = CollapsiblePrimitive.Root;

const CollapsibleTrigger = CollapsiblePrimitive.CollapsibleTrigger;

const CollapsibleContent = CollapsiblePrimitive.CollapsibleContent;

export { Collapsible, CollapsibleTrigger, CollapsibleContent };
```

File: credit-card-fraud-detection/frontend/src/components/ui/command.jsx

```

import * as React from "react";

import { Command as CommandPrimitive } from "cmdk";

import { Search } from "lucide-react";

import { cn } from "@lib/utls";

import { Dialog, DialogContent } from "@components/ui/dialog";

const Command = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive
    ref={ref}
    className={cn(
      "flex h-full w-full flex-col overflow-hidden rounded-md bg-popover text-popover-
foreground",
      className,
    )}
    {...props}
  />
));

Command.displayName = CommandPrimitive.displayName;

const CommandDialog = ({ children, ...props }) => {
  return (
    <Dialog {...props}>
      <DialogContent className="overflow-hidden p-0 shadow-lg">
        <Command className="[&_cmdk-group-heading]:px-2 [&_cmdk-group-
heading]:font-medium [&_cmdk-group-heading]:text-muted-foreground [&_cmdk-
group]:not([hidden])_~[cmdk-group]:pt-0 [&_cmdk-group]:px-2 [&_cmdk-input-

```

```
wrapper]_svg]:h-5 [&_[cmdk-input-wrapper]_svg]:w-5 [&_[cmdk-input]]:h-12 [&_[cmdk-item]]:px-2 [&_[cmdk-item]]:py-3 [&_[cmdk-item]_svg]:h-5 [&_[cmdk-item]_svg]:w-5">
```

```
  {children}
```

```
</Command>
```

```
</DialogContent>
```

```
</Dialog>
```

```
);
```

```
};
```

```
const CommandInput = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <div className="flex items-center border-b px-3" cmdk-input-wrapper="">
```

```
    <Search className="mr-2 h-4 w-4 shrink-0 opacity-50" />
```

```
    <CommandPrimitive.Input
```

```
      ref={ref}
```

```
      className={cn(
```

```
        "flex h-11 w-full rounded-md bg-transparent py-3 text-sm outline-none  
placeholder:text-muted-foreground disabled:cursor-not-allowed disabled:opacity-50",
```

```
        className,
```

```
      )}
```

```
      {...props}
```

```
    />
```

```
  </div>
```

```
));
```

```
CommandInput.displayName = CommandPrimitive.Input.displayName;
```

```

const CommandList = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive.List
    ref={ref}
    className={cn("max-h-[300px] overflow-y-auto overflow-x-hidden", className)}
    {...props}
  />
));

```

CommandList.displayName = CommandPrimitive.List.displayName;

```

const CommandEmpty = React.forwardRef((props, ref) => (
  <CommandPrimitive.Empty
    ref={ref}
    className="py-6 text-center text-sm"
    {...props}
  />
));

```

CommandEmpty.displayName = CommandPrimitive.Empty.displayName;

```

const CommandGroup = React.forwardRef(({ className, ...props }, ref) => (
  <CommandPrimitive.Group
    ref={ref}
    className={cn(
      "overflow-hidden p-1 text-foreground [&_cmdk-group-heading]:px-2 [&_cmdk-group-
heading]:py-1.5 [&_cmdk-group-heading]:text-xs [&_cmdk-group-heading]:font-

```

```
medium [&_[cmdk-group-heading]]:text-muted-foreground",
```

```
  className,
```

```
)}
```

```
{...props}
```

```
/>
```

```
));
```

```
CommandGroup.displayName = CommandPrimitive.Group.displayName;
```

```
const CommandSeparator = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <CommandPrimitive.Separator
```

```
    ref={ref}
```

```
    className={cn("-mx-1 h-px bg-border", className)}
```

```
    {...props}
```

```
/>
```

```
));
```

```
CommandSeparator.displayName = CommandPrimitive.Separator.displayName;
```

```
const CommandItem = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <CommandPrimitive.Item
```

```
    ref={ref}
```

```
    className={cn(
```

```
      "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm  
outline-none data-[disabled=true]:pointer-events-none data-[selected='true']:bg-accent  
data-[selected=true]:text-accent-foreground data-[disabled=true]:opacity-50",
```

```
    className,
```

```
    )}  
    {...props}  
  />  
));
```

```
CommandItem.displayName = CommandPrimitive.Item.displayName;
```

```
const CommandShortcut = ({ className, ...props }) => {  
  return (  
    <span  
      className={cn(  
        "ml-auto text-xs tracking-widest text-muted-foreground",  
        className,  
      )}  
      {...props}  
    />  
  );  
};  
  
CommandShortcut.displayName = "CommandShortcut";
```

```
export {  
  Command,  
  CommandDialog,  
  CommandInput,  
  CommandList,  
  CommandEmpty,
```

```
CommandGroup,  
CommandItem,  
CommandShortcut,  
CommandSeparator,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/context-menu.jsx

```
import * as React from "react";  
  
import * as ContextMenuPrimitive from "@radix-ui/react-context-menu";  
  
import { Check, ChevronRight, Circle } from "lucide-react";  
  
import { cn } from "@lib/utils";  
  
const ContextMenu = ContextMenuPrimitive.Root;  
  
const ContextMenuTrigger = ContextMenuPrimitive.Trigger;  
  
const ContextMenuGroup = ContextMenuPrimitive.Group;  
  
const ContextMenuPortal = ContextMenuPrimitive.Portal;  
  
const ContextMenuSub = ContextMenuPrimitive.Sub;  
  
const ContextMenuRadioGroup = ContextMenuPrimitive.RadioGroup;
```



```

const ContextMenuSubTrigger = React.forwardRef(
  ({ className, inset, children, ...props }, ref) => (
    <ContextMenuPrimitive.SubTrigger
      ref={ref}
      className={cn(
        "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-
        none data-[state=open]:bg-accent data-[state=open]:text-accent-foreground focus:bg-
        accent focus:text-accent-foreground",
        inset && "pl-8",
        className,
      )}
      {...props}
    >
      {children}
    <ChevronRight className="ml-auto h-4 w-4" />
  </ContextMenuPrimitive.SubTrigger>
),
);

ContextMenuSubTrigger.displayName = ContextMenuPrimitive.SubTrigger.displayName;

```

```

const ContextMenuSubContent = React.forwardRef(
  ({ className, ...props }, ref) => (
    <ContextMenuPrimitive.SubContent
      ref={ref}
      className={cn(

```

```
"z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
```

```
  className,  
  
  )}  
  
  {...props}  
  
/>  
  
,  
  
);
```

```
ContextMenuSubContent.displayName =  
ContextMenuPrimitive.SubContent.displayName;
```

```
const ContextMenuContent = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <ContextMenuPrimitive.Portal>
```

```
    <ContextMenuPrimitive.Content
```

```
      ref={ref}
```

```
      className={cn(
```

```
        "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-foreground shadow-md animate-in fade-in-80 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
```

```
        className,  
  
      )}  
  
      {...props}  
  
/>
```

```

</ContextMenuPrimitive.Portal>

));

ContextMenuContent.displayName = ContextMenuPrimitive.Content.displayName;

const ContextMenuItem = React.forwardRef(
  ({ className, inset, ...props }, ref) => (
    <ContextMenuPrimitive.Item
      ref={ref}
      className={cn(
        "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm
outline-none data-[disabled]:pointer-events-none data-[disabled]:opacity-50 focus:bg-accent
focus:text-accent-foreground",
        inset && "pl-8",
        className,
      )}
      {...props}
    />
  ),
);

ContextMenuItem.displayName = ContextMenuPrimitive.Item.displayName;

const ContextMenuCheckboxItem = React.forwardRef(
  ({ className, children, checked, ...props }, ref) => (
    <ContextMenuPrimitive.CheckboxItem
      ref={ref}
      className={cn(

```

```

    "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-
sm outline-none data-[disabled]:pointer-events-none data-[disabled]:opacity-50 focus:bg-
accent focus:text-accent-foreground",

    className,

  )}

  checked={checked}

  {...props}
>

  <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">

    <ContextMenuPrimitive.ItemIndicator>

      <Check className="h-4 w-4" />

    </ContextMenuPrimitive.ItemIndicator>

  </span>

  {children}

</ContextMenuPrimitive.CheckboxItem>

),

);

ContextMenuCheckboxItem.displayName =

  ContextMenuPrimitive.CheckboxItem.displayName;

const ContextMenuRadioItem = React.forwardRef(

  ({ className, children, ...props }, ref) => (

    <ContextMenuPrimitive.RadioItem

      ref={ref}

      className={cn(

        "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-
sm outline-none data-[disabled]:pointer-events-none data-[disabled]:opacity-50 focus:bg-

```

```
accent focus:text-accent-foreground",
```

```
  className,
```

```
  )}
```

```
  {...props}
```

```
>
```

```
<span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
```

```
  <ContextMenuPrimitive.ItemIndicator>
```

```
    <Circle className="h-2 w-2 fill-current" />
```

```
  </ContextMenuPrimitive.ItemIndicator>
```

```
</span>
```

```
{children}
```

```
</ContextMenuPrimitive.RadioItem>
```

```
),
```

```
);
```

```
ContextMenuRadioItem.displayName = ContextMenuPrimitive.RadioItem.displayName;
```

```
const ContextMenuLabel = React.forwardRef(
```

```
  ({ className, inset, ...props }, ref) => (
```

```
    <ContextMenuPrimitive.Label
```

```
      ref={ref}
```

```
      className={cn(
```

```
        "px-2 py-1.5 text-sm font-semibold text-foreground",
```

```
        inset && "pl-8",
```

```
        className,
```

```
      )}
```

```
      {...props}
```

```

    />

    ),

);

ContextMenuLabel.displayName = ContextMenuPrimitive.Label.displayName;

const ContextMenuSeparator = React.forwardRef(
  ({ className, ...props }, ref) => (
    <ContextMenuPrimitive.Separator
      ref={ref}
      className={cn("-mx-1 my-1 h-px bg-border", className)}
      {...props}
    />
  ),
);

ContextMenuSeparator.displayName = ContextMenuPrimitive.Separator.displayName;

const ContextMenuShortcut = ({ className, ...props }) => {
  return (
    <span
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground",
        className,
      )}
      {...props}
    />
  );
};

```

```
};

ContextMenuShortcut.displayName = "ContextMenuShortcut";

export {
  ContextMenu,
  ContextMenuTrigger,
  ContextMenuContent,
  ContextMenuItem,
  ContextMenuCheckboxItem,
  ContextMenuRadioItem,
  ContextMenuLabel,
  ContextMenuSeparator,
  ContextMenuShortcut,
  ContextMenuGroup,
  ContextMenuPortal,
  ContextMenuSub,
  ContextMenuSubContent,
  ContextMenuSubTrigger,
  ContextMenuRadioGroup,
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/dialog.jsx

```
import * as React from "react";

import * as DialogPrimitive from "@radix-ui/react-dialog";
```

```
import { X } from "lucide-react";
```

```
import { cn } from "@lib/utls";
```

```
const Dialog = DialogPrimitive.Root;
```

```
const DialogTrigger = DialogPrimitive.Trigger;
```

```
const DialogPortal = DialogPrimitive.Portal;
```

```
const DialogClose = DialogPrimitive.Close;
```

```
const DialogOverlay = React.forwardRef(({ className, ...props }, ref) => (  
  <DialogPrimitive.Overlay  
    ref={ref}  
    className={cn(  
      "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-  
[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",  
      className,  
    )}  
    {...props}  
  />  
));
```

```
DialogOverlay.displayName = DialogPrimitive.Overlay.displayName;
```

```
const DialogContent = React.forwardRef(  
  <DialogPrimitive.Content  
    ref={ref}  
    className={cn(  
      "z-50 gap-4 p-6",  
      className,  
    )}  
    {...props}  
  />  
));
```



```
( { className, children, ...props }, ref) => (
  <DialogPortal>
    <DialogOverlay />
    <DialogPrimitive.Content
      ref={ref}
      className={cn(
        "fixed left-[50%] top-[50%] z-50 grid w-full max-w-lg translate-x-[-50%] translate-y-[-50%] gap-4 border bg-background p-6 shadow-lg duration-200 data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[state=closed]:slide-out-to-left-1/2 data-[state=closed]:slide-out-to-top-[48%] data-[state=open]:slide-in-from-left-1/2 data-[state=open]:slide-in-from-top-[48%] sm:rounded-lg",
        className,
      )}
      {...props}
    >
      {children}
    <DialogPrimitive.Close className="absolute right-4 top-4 rounded-sm opacity-70 ring-offset-background transition-opacity data-[state=open]:bg-accent data-[state=open]:text-muted-foreground hover:opacity-100 focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:pointer-events-none">
      <X className="h-4 w-4" />
      <span className="sr-only">Close</span>
    </DialogPrimitive.Close>
  </DialogPrimitive.Content>
</DialogPortal>
),
);
```

```
DialogContent.displayName = DialogPrimitive.Content.displayName;
```

```
const DialogHeader = ({ className, ...props }) => (
```

```
  <div
```

```
    className={cn(
```

```
      "flex flex-col space-y-1.5 text-center sm:text-left",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
  />
```

```
);
```

```
DialogHeader.displayName = "DialogHeader";
```

```
const DialogFooter = ({ className, ...props }) => (
```

```
  <div
```

```
    className={cn(
```

```
      "flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
  />
```

```
);
```

```
DialogFooter.displayName = "DialogFooter";
```

```
const DialogTitle = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <DialogPrimitive.Title
```

```
    ref={ref}

    className={cn(
      "text-lg font-semibold leading-none tracking-tight",
      className,
    )}

    {...props}
  />
));

DialogTitle.displayName = DialogPrimitive.Title.displayName;

const DialogDescription = React.forwardRef(({ className, ...props }, ref) => (
  <DialogPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
));

DialogDescription.displayName = DialogPrimitive.Description.displayName;

export {
  Dialog,
  DialogPortal,
  DialogOverlay,
  DialogClose,
  DialogTrigger,
  DialogContent,
```

```
DialogHeader,  
DialogFooter,  
DialogTitle,  
DialogDescription,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/drawer.jsx

```
import * as React from "react";  
  
import { Drawer as DrawerPrimitive } from "vaul";  
  
import { cn } from "@lib/utils";  
  
const Drawer = ({ shouldScaleBackground = true, ...props }) => (  
  <DrawerPrimitive.Root  
    shouldScaleBackground={shouldScaleBackground}  
    {...props}  
  />  
);  
  
Drawer.displayName = "Drawer";  
  
const DrawerTrigger = DrawerPrimitive.Trigger;  
  
const DrawerPortal = DrawerPrimitive.Portal;
```

```
const DrawerClose = DrawerPrimitive.Close;
```

```
const DrawerOverlay = React.forwardRef(({ className, ...props }, ref) => (  
  <DrawerPrimitive.Overlay  
    ref={ref}  
    className={cn("fixed inset-0 z-50 bg-black/80", className)}  
    {...props}  
  />  
));
```

```
DrawerOverlay.displayName = DrawerPrimitive.Overlay.displayName;
```

```
const DrawerContent = React.forwardRef(  
  ({ className, children, ...props }, ref) => (  
    <DrawerPortal>  
      <DrawerOverlay />  
      <DrawerPrimitive.Content  
        ref={ref}  
        className={cn(  
          "fixed inset-x-0 bottom-0 z-50 mt-24 flex h-auto flex-col rounded-t-[10px] border bg-background",  
          className,  
        )}  
        {...props}  
      >  
        <div className="mx-auto mt-4 h-2 w-[100px] rounded-full bg-muted" />  
        {children}
```

```

    </DrawerPrimitive.Content>

  </DrawerPortal>

),

);

DrawerContent.displayName = "DrawerContent";

const DrawerHeader = ({ className, ...props }) => (

  <div

    className={cn("grid gap-1.5 p-4 text-center sm:text-left", className)}

    {...props}

  />

);

DrawerHeader.displayName = "DrawerHeader";

const DrawerFooter = ({ className, ...props }) => (

  <div

    className={cn("mt-auto flex flex-col gap-2 p-4", className)}

    {...props}

  />

);

DrawerFooter.displayName = "DrawerFooter";

const DrawerTitle = React.forwardRef(({ className, ...props }, ref) => (

  <DrawerPrimitive.Title

    ref={ref}

    className={cn(

```

```

    "text-lg font-semibold leading-none tracking-tight",
    className,
  )}
  {...props}
/>
));

DrawerTitle.displayName = DrawerPrimitive.Title.displayName;

const DrawerDescription = React.forwardRef(({ className, ...props }, ref) => (
  <DrawerPrimitive.Description
    ref={ref}
    className={cn("text-sm text-muted-foreground", className)}
    {...props}
  />
));

DrawerDescription.displayName = DrawerPrimitive.Description.displayName;

export {
  Drawer,
  DrawerPortal,
  DrawerOverlay,
  DrawerTrigger,
  DrawerClose,
  DrawerContent,
  DrawerHeader,
  DrawerFooter,

```

```
DrawerTitle,  
DrawerDescription,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/dropdown-menu.jsx

```
import * as React from "react";  
  
import * as DropdownMenuPrimitive from "@radix-ui/react-dropdown-menu";  
import { Check, ChevronRight, Circle } from "lucide-react";  
  
import { cn } from "@lib/utls";  
  
const DropdownMenu = DropdownMenuPrimitive.Root;  
  
const DropdownMenuTrigger = DropdownMenuPrimitive.Trigger;  
  
const DropdownMenuGroup = DropdownMenuPrimitive.Group;  
  
const DropdownMenuPortal = DropdownMenuPrimitive.Portal;  
  
const DropdownMenuSub = DropdownMenuPrimitive.Sub;  
  
const DropdownMenuRadioGroup = DropdownMenuPrimitive.RadioGroup;  
  
const DropdownMenuSubTrigger = React.forwardRef(  
  props,
```



```

({ className, inset, children, ...props }, ref) => (
  <DropdownMenuPrimitive.SubTrigger
    ref={ref}
    className={cn(
      "flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-
none data-[state=open]:bg-accent focus:bg-accent",
      inset && "pl-8",
      className,
    )}
    {...props}
  >
    {children}
    <ChevronRight className="ml-auto h-4 w-4" />
  </DropdownMenuPrimitive.SubTrigger>
),
);

DropdownMenuSubTrigger.displayName =
  DropdownMenuPrimitive.SubTrigger.displayName;

const DropdownMenuSubContent = React.forwardRef(
  ({ className, ...props }, ref) => (
    <DropdownMenuPrimitive.SubContent
      ref={ref}
      className={cn(
        "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-
foreground shadow-lg data-[state=open]:animate-in data-[state=closed]:animate-out data-
[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95

```

```

data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-
in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-
2",

    className,

  )}

  {...props}

/>

),

);

DropdownMenuSubContent.displayName =

  DropdownMenuPrimitive.SubContent.displayName;

const DropdownMenuContent = React.forwardRef(

  ({ className, sideOffset = 4, ...props }, ref) => (

    <DropdownMenuPrimitive.Portal>

      <DropdownMenuPrimitive.Content

        ref={ref}

        sideOffset={sideOffset}

        className={cn(

          "z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-
popover-foreground shadow-md data-[state=open]:animate-in data-[state=closed]:animate-
out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-
out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-
[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-
in-from-bottom-2",

            className,

          )}

        {...props}

```

```

    />

    </DropdownMenuPrimitive.Portal>

  ),

);

DropdownMenuContent.displayName = DropdownMenuPrimitive.Content.displayName;

const DropdownMenuItem = React.forwardRef(
  ({ className, inset, ...props }, ref) => (
    <DropdownMenuPrimitive.Item
      ref={ref}
      className={cn(
        "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm
outline-none transition-colors data-[disabled]:pointer-events-none data-[disabled]:opacity-
50 focus:bg-accent focus:text-accent-foreground",
        inset && "pl-8",
        className,
      )}
      {...props}
    />
  ),
);

DropdownMenuItem.displayName = DropdownMenuPrimitive.Item.displayName;

const DropdownMenuCheckboxItem = React.forwardRef(
  ({ className, children, checked, ...props }, ref) => (
    <DropdownMenuPrimitive.CheckboxItem

```

```

    ref={ref}

    className={cn(
      "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none transition-colors data-[disabled]:pointer-events-none data-[disabled]:opacity-50 focus:bg-accent focus:text-accent-foreground",
      className,
    )}
    checked={checked}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <DropdownMenuPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </DropdownMenuPrimitive.ItemIndicator>
    </span>
    {children}
  </DropdownMenuPrimitive.CheckboxItem>
),
);

DropdownMenuCheckboxItem.displayName =
  DropdownMenuPrimitive.CheckboxItem.displayName;

const DropdownMenuRadioItem = React.forwardRef(
  ({ className, children, ...props }, ref) => (
    <DropdownMenuPrimitive.RadioItem
      ref={ref}

```

```

className={cn(
  "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none transition-colors data-[disabled]:pointer-events-none data-[disabled]:opacity-50 focus:bg-accent focus:text-accent-foreground",
  className,
)}
{...props}
>
<span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
  <DropdownMenuPrimitive.ItemIndicator>
    <Circle className="h-2 w-2 fill-current" />
  </DropdownMenuPrimitive.ItemIndicator>
</span>
{children}
</DropdownMenuPrimitive.RadioItem>
),
);

DropdownMenuRadioItem.displayName =
DropdownMenuPrimitive.RadioItem.displayName;

```

```

const DropdownMenuLabel = React.forwardRef(
  ({ className, inset, ...props }, ref) => (
    <DropdownMenuPrimitive.Label
      ref={ref}
      className={cn(
        "px-2 py-1.5 text-sm font-semibold",
        inset && "pl-8",

```

```

      className,
    )}
    {...props}
  />
),
);
DropdownMenuLabel.displayName = DropdownMenuPrimitive.Label.displayName;

```

```

const DropdownMenuSeparator = React.forwardRef(
  ({ className, ...props }, ref) => (
    <DropdownMenuPrimitive.Separator
      ref={ref}
      className={cn("-mx-1 my-1 h-px bg-muted", className)}
      {...props}
    />
  ),
);
DropdownMenuSeparator.displayName =
DropdownMenuPrimitive.Separator.displayName;

```

```

const DropdownMenuShortcut = ({ className, ...props }) => {
  return (
    <span
      className={cn("ml-auto text-xs tracking-widest opacity-60", className)}
      {...props}
    />
  )
}

```

```
);  
  
};  
  
DropdownMenuShortcut.displayName = "DropdownMenuShortcut";  
  
export {  
  DropdownMenu,  
  DropdownMenuTrigger,  
  DropdownMenuContent,  
  DropdownMenuItem,  
  DropdownMenuCheckboxItem,  
  DropdownMenuRadioItem,  
  DropdownMenuLabel,  
  DropdownMenuSeparator,  
  DropdownMenuShortcut,  
  DropdownMenuGroup,  
  DropdownMenuPortal,  
  DropdownMenuSub,  
  DropdownMenuSubContent,  
  DropdownMenuSubTrigger,  
  DropdownMenuRadioGroup,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/form.jsx

```
import * as React from "react";
```

```
import { Slot } from "@radix-ui/react-slot";

import { Controller, FormProvider, useFormContext } from "react-hook-form";

import { cn } from "@lib/utils";

import { Label } from "@components/ui/label";

const Form = FormProvider;

const FormFieldContext = React.createContext({});

const FormField = ({ ...props }) => {

  return (

    <FormFieldContext.Provider value={{ name: props.name }}>

      <Controller {...props} />

    </FormFieldContext.Provider>

  );

};

const useFormField = () => {

  const fieldContext = React.useContext(FormFieldContext);

  const itemContext = React.useContext(FormItemContext);

  const { getFieldState, formState } = useFormContext();

  const fieldState = getFieldState(fieldContext.name, formState);

  if (!fieldContext) {
```



```
    throw new Error("useFormField should be used within <FormField>");
  }
```

```
const { id } = itemContext;
```

```
return {
  id,
  name: fieldContext.name,
  formItemId: `${id}-form-item`,
  formDescriptionId: `${id}-form-item-description`,
  formMessageId: `${id}-form-item-message`,
  ...fieldState,
};
};
```

```
const FormItemContext = React.createContext({});
```

```
const FormItem = React.forwardRef(({ className, ...props }, ref) => {
  const id = React.useId();

  return (
    <FormItemContext.Provider value={{ id }}>
      <div ref={ref} className={cn("space-y-2", className)} {...props} />
    </FormItemContext.Provider>
  );
});
```

```
FormItem.displayName = "FormItem";
```

```
const FormLabel = React.forwardRef(({ className, ...props }, ref) => {
```

```
  const { error, formItemId } = useFormField();
```

```
  return (
```

```
    <Label
```

```
      ref={ref}
```

```
      className={cn(error && "text-destructive", className)}
```

```
      htmlFor={formItemId}
```

```
      {...props}
```

```
    />
```

```
  );
```

```
});
```

```
FormLabel.displayName = "FormLabel";
```

```
const FormControl = React.forwardRef(({ ...props }, ref) => {
```

```
  const { error, formItemId, formDescriptionId, formMessageId } =
```

```
    useFormField();
```

```
  return (
```

```
    <Slot
```

```
      ref={ref}
```

```
      id={formItemId}
```

```
      aria-describedby={
```

```
        !error
```

```

      ? `${formDescriptionId}`
      : `${formDescriptionId} ${formMessageId}`
    }
    aria-invalid={!error}
    {...props}
  />
);
});

FormControl.displayName = "FormControl";

const FormDescription = React.forwardRef(({ className, ...props }, ref) => {
  const { formDescriptionId } = useFormField();

  return (
    <p
      ref={ref}
      id={formDescriptionId}
      className={cn("text-sm text-muted-foreground", className)}
      {...props}
    />
  );
});

FormDescription.displayName = "FormDescription";

const FormMessage = React.forwardRef(
  ({ className, children, ...props }, ref) => {

```

```
const { error, formMessageId } = useFormField();

const body = error ? String(error?.message) : children;

if (!body) {
  return null;
}

return (
  <p
    ref={ref}
    id={formMessageId}
    className={cn("text-sm font-medium text-destructive", className)}
    {...props}
  >
    {body}
  </p>
);
},
);

FormMessage.displayName = "FormMessage";

export {
  useFormField,
  Form,
  FormItem,
  FormLabel,
```

```
    FormControl,  
    FormDescription,  
    FormMessage,  
    FormField,  
  };
```

File: credit-card-fraud-detection/frontend/src/components/ui/hover-card.jsx

```
import * as React from "react";  
  
import * as HoverCardPrimitive from "@radix-ui/react-hover-card";  
  
import { cn } from "@lib/utils";  
  
const HoverCard = HoverCardPrimitive.Root;  
  
const HoverCardTrigger = HoverCardPrimitive.Trigger;  
  
const HoverCardContent = React.forwardRef(  
  ({ className, align = "center", sideOffset = 4, ...props }, ref) => (  
    <HoverCardPrimitive.Content  
      ref={ref}  
      align={align}  
      sideOffset={sideOffset}  
      className={cn(  

```

```
"z-50 w-64 rounded-md border bg-popover p-4 text-popover-foreground shadow-md outline-none data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
```

```
  className,  
  
  )}  
  
  {...props}  
  
/>  
  
,  
  
);
```

```
HoverCardContent.displayName = HoverCardPrimitive.Content.displayName;
```

```
export { HoverCard, HoverCardTrigger, HoverCardContent };
```

File: credit-card-fraud-detection/frontend/src/components/ui/input-otp.jsx

```
import * as React from "react";  
  
import { OTPInput, OTPInputContext } from "input-otp";  
  
import { Dot } from "lucide-react";  
  
  
import { cn } from "@lib/utls";  
  
  
const InputOTP = React.forwardRef(  
  ({ className, containerClassName, ...props }, ref) => (  
    <OTPInput
```

```

    ref={ref}

    className={cn(
      "flex items-center gap-2 has-[:disabled]:opacity-50",
      className,
    )}

    {...props}
  />
),
);

InputOTP.displayName = "InputOTP";

const InputOTPGroup = React.forwardRef(({ className, ...props }, ref) => (
  <div ref={ref} className={cn("flex items-center", className)} {...props} />
));

InputOTPGroup.displayName = "InputOTPGroup";

const InputOTPSlot = React.forwardRef(({ index, className, ...props }, ref) => {
  const inputOTPContext = React.useContext(OTPInputContext);
  const { char, hasFakeCaret, isActive } = inputOTPContext.slots[index];

  return (
    <div
      ref={ref}
      className={cn(

```

```

    "relative flex h-10 w-10 items-center justify-center border-y border-r border-input text-sm transition-all first:rounded-l-md first:border-l last:rounded-r-md",

    isActive && "z-10 ring-2 ring-ring ring-offset-background",

    className,

  )}

  {...props}

>

  {char}

  {hasFakeCaret && (

    <div className="pointer-events-none absolute inset-0 flex items-center justify-center">

      <div className="animate-caret-blink h-4 w-px bg-foreground duration-1000" />

    </div>

  )}

</div>

);

});

InputOTPSlot.displayName = "InputOTPSlot";

```

```

const InputOTPSeparator = React.forwardRef(({ ...props }, ref) => (

  <div ref={ref} role="separator" {...props}>

    <Dot />

  </div>

));

InputOTPSeparator.displayName = "InputOTPSeparator";

```



```
export { InputOTP, InputOTPGroup, InputOTPSlot, InputOTPSeparator };
```

File: credit-card-fraud-detection/frontend/src/components/ui/input.jsx

```
import * as React from "react";
```

```
import { cn } from "@lib/utils";
```

```
const Input = React.forwardRef(({ className, type, ...props }, ref) => {
```

```
  return (
```

```
    <input
```

```
      type={type}
```

```
      className={cn(
```

```
        "flex h-10 w-full rounded-md border border-input bg-background px-3 py-2 text-base  
ring-offset-background file:border-0 file:bg-transparent file:text-sm file:font-medium  
file:text-foreground placeholder:text-muted-foreground focus-visible:outline-none focus-  
visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 disabled:cursor-not-allowed  
disabled:opacity-50 md:text-sm",
```

```
        className,
```

```
      )}
```

```
      ref={ref}
```

```
      {...props}
```

```
    />
```

```
  );
```

```
});
```

```
Input.displayName = "Input";
```

```
export { Input };
```

File: credit-card-fraud-detection/frontend/src/components/ui/label.jsx

```
import * as React from "react";
```

```
import * as LabelPrimitive from "@radix-ui/react-label";
```

```
import { cva } from "class-variance-authority";
```

```
import { cn } from "@lib/utils";
```

```
const labelVariants = cva(
```

```
  "text-sm font-medium leading-none peer-disabled:cursor-not-allowed peer-disabled:opacity-70",
```

```
);
```

```
const Label = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <LabelPrimitive.Root
```

```
    ref={ref}
```

```
    className={cn(labelVariants(), className)}
```

```
    {...props}
```

```
  />
```

```
));
```

```
Label.displayName = LabelPrimitive.Root.displayName;
```

```
export { Label };
```

File: credit-card-fraud-detection/frontend/src/components/ui/menubar.jsx

```
import * as React from "react";
```

```
import * as MenubarPrimitive from "@radix-ui/react-menubar";
```

```
import { Check, ChevronRight, Circle } from "lucide-react";
```

```
import { cn } from "@lib/utils";
```

```
const MenubarMenu = MenubarPrimitive.Menu;
```

```
const MenubarGroup = MenubarPrimitive.Group;
```

```
const MenubarPortal = MenubarPrimitive.Portal;
```

```
const MenubarSub = MenubarPrimitive.Sub;
```

```
const MenubarRadioGroup = MenubarPrimitive.RadioGroup;
```

```
const Menubar = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <MenubarPrimitive.Root
```

```
    ref={ref}
```

```
    className={cn(
```

```
      "flex h-10 items-center space-x-1 rounded-md border bg-background p-1",
```

```

        className,

    )}

    {...props}

/>

));

Menubar.displayName = MenubarPrimitive.Root.displayName;

const MenubarTrigger = React.forwardRef(({ className, ...props }, ref) => (

    <MenubarPrimitive.Trigger

        ref={ref}

        className={cn(

            "flex cursor-default select-none items-center rounded-sm px-3 py-1.5 text-sm font-
            medium outline-none data-[state=open]:bg-accent data-[state=open]:text-accent-
            foreground focus:bg-accent focus:text-accent-foreground",

            className,

        )}

        {...props}

    />

));

MenubarTrigger.displayName = MenubarPrimitive.Trigger.displayName;

const MenubarSubTrigger = React.forwardRef(

    ({ className, inset, children, ...props }, ref) => (

        <MenubarPrimitive.SubTrigger

            ref={ref}

            className={cn(

```

```
"flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm outline-
none data-[state=open]:bg-accent data-[state=open]:text-accent-foreground focus:bg-
accent focus:text-accent-foreground",
```

```
inset && "pl-8",
```

```
className,
```

```
))
```

```
{...props}
```

```
>
```

```
{children}
```

```
<ChevronRight className="ml-auto h-4 w-4" />
```

```
</MenubarPrimitive.SubTrigger>
```

```
),
```

```
);
```

```
MenubarSubTrigger.displayName = MenubarPrimitive.SubTrigger.displayName;
```

```
const MenubarSubContent = React.forwardRef(({ className, ...props }, ref) => (
```

```
<MenubarPrimitive.SubContent
```

```
ref={ref}
```

```
className={cn(
```

```
"z-50 min-w-[8rem] overflow-hidden rounded-md border bg-popover p-1 text-popover-
foreground data-[state=open]:animate-in data-[state=closed]:animate-out data-
[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-
in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-
2",
```

```
className,
```

```
))
```

```
{...props}
```

```

/>

));

MenubarSubContent.displayName = MenubarPrimitive.SubContent.displayName;

const MenubarContent = React.forwardRef(
  (
    { className, align = "start", alignOffset = -4, sideOffset = 8, ...props },
    ref,
  ) => (
    <MenubarPrimitive.Portal>
      <MenubarPrimitive.Content
        ref={ref}
        align={align}
        alignOffset={alignOffset}
        sideOffset={sideOffset}
        className={cn(
          "z-50 min-w-[12rem] overflow-hidden rounded-md border bg-popover p-1 text-
popover-foreground shadow-md data-[state=open]:animate-in data-[state=closed]:fade-
out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95 data-
[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-
from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",
          className,
        )}
        {...props}
      />
    </MenubarPrimitive.Portal>
  ),

```

```
);
```

```
MenubarContent.displayName = MenubarPrimitive.Content.displayName;
```

```
const MenubarItem = React.forwardRef(({ className, inset, ...props }, ref) => (
```

```
  <MenubarPrimitive.Item
```

```
    ref={ref}
```

```
    className={cn(
```

```
      "relative flex cursor-default select-none items-center rounded-sm px-2 py-1.5 text-sm  
outline-none data-[disabled]:pointer-events-none data-[disabled]:opacity-50 focus:bg-accent  
focus:text-accent-foreground",
```

```
      inset && "pl-8",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
  />
```

```
));
```

```
MenubarItem.displayName = MenubarPrimitive.Item.displayName;
```

```
const MenubarCheckboxItem = React.forwardRef(
```

```
  ({ className, children, checked, ...props }, ref) => (
```

```
    <MenubarPrimitive.CheckboxItem
```

```
      ref={ref}
```

```
      className={cn(
```

```
        "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm  
outline-none data-[disabled]:pointer-events-none data-[disabled]:opacity-50 focus:bg-accent  
focus:text-accent-foreground",
```

```
        className,
```

```

    })
    checked={checked}
    {...props}
  >
    <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
      <MenubarPrimitive.ItemIndicator>
        <Check className="h-4 w-4" />
      </MenubarPrimitive.ItemIndicator>
    </span>
    {children}
  </MenubarPrimitive.CheckboxItem>
),
);

MenubarCheckboxItem.displayName = MenubarPrimitive.CheckboxItem.displayName;

const MenubarRadioItem = React.forwardRef(
  ({ className, children, ...props }, ref) => (
    <MenubarPrimitive.RadioItem
      ref={ref}
      className={cn(
        "relative flex cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-2 text-sm outline-none data-[disabled]:pointer-events-none data-[disabled]:opacity-50 focus:bg-accent focus:text-accent-foreground",
        className,
      )}
      {...props}
    >

```


>

```
<span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
```

```
<MenubarPrimitive.ItemIndicator>
```

```
<Circle className="h-2 w-2 fill-current" />
```

```
</MenubarPrimitive.ItemIndicator>
```

```
</span>
```

```
{children}
```

```
</MenubarPrimitive.RadioItem>
```

```
),
```

```
);
```

```
MenubarRadioItem.displayName = MenubarPrimitive.RadioItem.displayName;
```

```
const MenubarLabel = React.forwardRef(({ className, inset, ...props }, ref) => (
```

```
<MenubarPrimitive.Label
```

```
ref={ref}
```

```
className={cn(
```

```
"px-2 py-1.5 text-sm font-semibold",
```

```
inset && "pl-8",
```

```
className,
```

```
))
```

```
{...props}
```

```
/>
```

```
));
```

```
MenubarLabel.displayName = MenubarPrimitive.Label.displayName;
```

```
const MenubarSeparator = React.forwardRef(({ className, ...props }, ref) => (
```

```

<MenubarPrimitive.Separator
  ref={ref}
  className={cn("-mx-1 my-1 h-px bg-muted", className)}
  {...props}
/>
));

MenubarSeparator.displayName = MenubarPrimitive.Separator.displayName;

const MenubarShortcut = ({ className, ...props }) => {
  return (
    <span
      className={cn(
        "ml-auto text-xs tracking-widest text-muted-foreground",
        className,
      )}
      {...props}
    />
  );
};

MenubarShortcut.displayName = "MenubarShortcut";

export {
  Menubar,
  MenubarMenu,
  MenubarTrigger,
  MenubarContent,

```

```
MenubarItem,  
MenubarSeparator,  
MenubarLabel,  
MenubarCheckboxItem,  
MenubarRadioGroup,  
MenubarRadioItem,  
MenubarPortal,  
MenubarSubContent,  
MenubarSubTrigger,  
MenubarGroup,  
MenubarSub,  
MenubarShortcut,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/navigation-menu.jsx

```
import * as React from "react";  
  
import * as NavigationMenuPrimitive from "@radix-ui/react-navigation-menu";  
import { cva } from "class-variance-authority";  
import { ChevronDown } from "lucide-react";  
  
import { cn } from "@lib/utils";  
  
const NavigationMenu = React.forwardRef(  
  ({ className, children, ...props }, ref) => (  
    <NavigationMenuPrimitive.Root  
      className={cn("border-collapse: collapse", className)}  
      {...props} ref={ref}>  
      {children}
```

```

<NavigationMenuPrimitive.Root
  ref={ref}
  className={cn(
    "relative z-10 flex max-w-max flex-1 items-center justify-center",
    className,
  )}
  {...props}
>
  {children}
  <NavigationMenuViewport />
</NavigationMenuPrimitive.Root>
),
);

NavigationMenu.displayName = NavigationMenuPrimitive.Root.displayName;

const NavigationMenuList = React.forwardRef(({ className, ...props }, ref) => (
  <NavigationMenuPrimitive.List
    ref={ref}
    className={cn(
      "group flex flex-1 list-none items-center justify-center space-x-1",
      className,
    )}
    {...props}
  />
));

NavigationMenuList.displayName = NavigationMenuPrimitive.List.displayName;

```

```
const NavigationMenuItem = NavigationMenuPrimitive.Item;
```

```
const navigationMenuTriggerStyle = cva(
```

```
  "group inline-flex h-10 w-max items-center justify-center rounded-md bg-background px-4 py-2 text-sm font-medium transition-colors hover:bg-accent hover:text-accent-foreground focus:bg-accent focus:text-accent-foreground focus:outline-none disabled:pointer-events-none disabled:opacity-50 data-[active]:bg-accent/50 data-[state=open]:bg-accent/50",
```

```
);
```

```
const NavigationMenuTrigger = React.forwardRef(
```

```
  ({ className, children, ...props }, ref) => (
```

```
    <NavigationMenuPrimitive.Trigger
```

```
      ref={ref}
```

```
      className={cn(navigationMenuTriggerStyle(), "group", className)}
```

```
      {...props}
```

```
    >
```

```
      {children} {" "}
```

```
      <ChevronDown
```

```
        className="relative top-[1px] ml-1 h-3 w-3 transition duration-200 group-data-[state=open]:rotate-180"
```

```
        aria-hidden="true"
```

```
      />
```

```
    </NavigationMenuPrimitive.Trigger>
```

```
  ),
```

```
);
```

```
NavigationMenuTrigger.displayName = NavigationMenuPrimitive.Trigger.displayName;
```

```

const NavigationMenuContent = React.forwardRef(
  ({ className, ...props }, ref) => (
    <NavigationMenuPrimitive.Content
      ref={ref}
      className={cn(
        "left-0 top-0 w-full data-[motion^=from-]:animate-in data-[motion^=to-]:animate-out
        data-[motion^=from-]:fade-in data-[motion^=to-]:fade-out data-[motion=from-end]:slide-
        in-from-right-52 data-[motion=from-start]:slide-in-from-left-52 data-[motion=to-
        end]:slide-out-to-right-52 data-[motion=to-start]:slide-out-to-left-52 md:absolute md:w-
        auto",
        className,
      )}
      {...props}
    />
  ),
);

NavigationMenuContent.displayName = NavigationMenuPrimitive.Content.displayName;

```

```

const NavigationMenuLink = NavigationMenuPrimitive.Link;

```

```

const NavigationMenuViewport = React.forwardRef(
  ({ className, ...props }, ref) => (
    <div className={cn("absolute left-0 top-full flex justify-center")}>
      <NavigationMenuPrimitive.Viewport
        className={cn(
          "origin-top-center relative mt-1.5 h-[var(--radix-navigation-menu-viewport-height)]
          w-full overflow-hidden rounded-md border bg-popover text-popover-foreground shadow-

```

```

lg data-[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:zoom-out-95 data-[state=open]:zoom-in-90 md:w-[var(--radix-navigation-menu-viewport-width)]",

    className,

  )}

  ref={ref}

  {...props}

/>

</div>

),

);

NavigationMenuViewport.displayName =

  NavigationMenuPrimitive.Viewport.displayName;

const NavigationMenuIndicator = React.forwardRef(

  ({ className, ...props }, ref) => (

    <NavigationMenuPrimitive.Indicator

      ref={ref}

      className={cn(

        "top-full z-[1] flex h-1.5 items-end justify-center overflow-hidden data-[state=visible]:animate-in data-[state=hidden]:animate-out data-[state=hidden]:fade-out data-[state=visible]:fade-in",

        className,

      )}

      {...props}

    >

      <div className="relative top-[60%] h-2 w-2 rotate-45 rounded-tl-sm bg-border shadow-md" />

```

```
    </NavigationMenuPrimitive.Indicator>

  ),

);

NavigationMenuIndicator.displayName =

  NavigationMenuPrimitive.Indicator.displayName;

export {

  navigationMenuTriggerStyle,

  NavigationMenu,

  NavigationMenuList,

  NavigationMenuItem,

  NavigationMenuContent,

  NavigationMenuTrigger,

  NavigationMenuLink,

  NavigationMenuIndicator,

  NavigationMenuViewport,

};
```

File: credit-card-fraud-detection/frontend/src/components/ui/pagination.jsx

```
import * as React from "react";

import { ChevronLeft, ChevronRight, MoreHorizontal } from "lucide-react";

import { cn } from "@lib/utls";

import { buttonVariants } from "@components/ui/button";
```



```
const Pagination = ({ className, ...props }) => (  
  <nav  
    role="navigation"  
    aria-label="pagination"  
    className={cn("mx-auto flex w-full justify-center", className)}  
    {...props}  
  />  
);  
  
Pagination.displayName = "Pagination";
```

```
const PaginationContent = React.forwardRef(({ className, ...props }, ref) => (  
  <ul  
    ref={ref}  
    className={cn("flex flex-row items-center gap-1", className)}  
    {...props}  
  />  
));  
  
PaginationContent.displayName = "PaginationContent";
```

```
const PaginationItem = React.forwardRef(({ className, ...props }, ref) => (  
  <li ref={ref} className={cn("", className)} {...props} />  
));  
  
PaginationItem.displayName = "PaginationItem";
```

```
const PaginationLink = ({ className, isActive, size = "icon", ...props }) => (  
  <a href="#" className={cn("text-current", className)} {...props}>
```

```

<a
  aria-current={isActive ? "page" : undefined}
  className={cn(
    buttonVariants({
      variant: isActive ? "outline" : "ghost",
      size,
    }),
    className,
  )}
  {...props}
/>
);

PaginationLink.displayName = "PaginationLink";

const PaginationPrevious = ({ className, ...props }) => (
  <PaginationLink
    aria-label="Go to previous page"
    size="default"
    className={cn("gap-1 pl-2.5", className)}
    {...props}
  >
    <ChevronLeft className="h-4 w-4" />
    <span>Previous</span>
  </PaginationLink>
);

PaginationPrevious.displayName = "PaginationPrevious";

```

```
const PaginationNext = ({ className, ...props }) => (  
  <PaginationLink  
    aria-label="Go to next page"  
    size="default"  
    className={cn("gap-1 pr-2.5", className)}  
    {...props}  
  >  
    <span>Next</span>  
    <ChevronRight className="h-4 w-4" />  
  </PaginationLink>  
);
```

```
PaginationNext.displayName = "PaginationNext";
```

```
const PaginationEllipsis = ({ className, ...props }) => (  
  <span  
    aria-hidden  
    className={cn("flex h-9 w-9 items-center justify-center", className)}  
    {...props}  
  >  
    <MoreHorizontal className="h-4 w-4" />  
    <span className="sr-only">More pages</span>  
  </span>  
);
```

```
PaginationEllipsis.displayName = "PaginationEllipsis";
```

```
export {  
  Pagination,  
  PaginationContent,  
  PaginationEllipsis,  
  PaginationItem,  
  PaginationLink,  
  PaginationNext,  
  PaginationPrevious,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/popover.jsx

```
import * as React from "react";  
  
import * as PopoverPrimitive from "@radix-ui/react-popover";  
  
import { cn } from "@lib/utils";  
  
const Popover = PopoverPrimitive.Root;  
  
const PopoverTrigger = PopoverPrimitive.Trigger;  
  
const PopoverContent = React.forwardRef(  
  ({ className, align = "center", sideOffset = 4, ...props }, ref) => (  
    <PopoverPrimitive.Portal>  
      <PopoverPrimitive.Content
```

```

    ref={ref}

    align={align}

    sideOffset={sideOffset}

    className={cn(
      "z-50 w-72 rounded-md border bg-popover p-4 text-popover-foreground shadow-md
outline-none data-[state=open]:animate-in data-[state=closed]:animate-out data-
[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-[state=closed]:zoom-out-95
data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-
in-from-right-2 data-[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-
2",
      className,
    )}
    {...props}
  />
</PopoverPrimitive.Portal>
),
);

PopoverContent.displayName = PopoverPrimitive.Content.displayName;

export { Popover, PopoverTrigger, PopoverContent };

```

File: credit-card-fraud-detection/frontend/src/components/ui/progress.jsx

```

import * as React from "react";

import * as ProgressPrimitive from "@radix-ui/react-progress";

import { cn } from "@lib/utls";

```

```

const Progress = React.forwardRef(({ className, value, ...props }, ref) => (
  <ProgressPrimitive.Root
    ref={ref}
    className={cn(
      "relative h-4 w-full overflow-hidden rounded-full bg-secondary",
      className,
    )}
    {...props}
  >
    <ProgressPrimitive.Indicator
      className="h-full w-full flex-1 bg-primary transition-all"
      style={{ transform: `translateX(-${100 - (value || 0)}%)` }}
    />
  </ProgressPrimitive.Root>
));

Progress.displayName = ProgressPrimitive.Root.displayName;

export { Progress };

```

File: credit-card-fraud-detection/frontend/src/components/ui/radio-group.jsx

```

import * as React from "react";

import * as RadioGroupPrimitive from "@radix-ui/react-radio-group";

import { Circle } from "lucide-react";

```

```
import { cn } from "@lib/utils";
```

```
const RadioGroup = React.forwardRef(({ className, ...props }, ref) => {  
  return (  
    <RadioGroupPrimitive.Root  
      className={cn("grid gap-2", className)}  
      {...props}  
      ref={ref}  
    />  
  );  
});
```

```
RadioGroup.displayName = RadioGroupPrimitive.Root.displayName;
```

```
const RadioGroupItem = React.forwardRef(({ className, ...props }, ref) => {  
  return (  
    <RadioGroupPrimitive.Item  
      ref={ref}  
      className={cn(  
        "aspect-square h-4 w-4 rounded-full border border-primary text-primary ring-offset-  
background focus:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-  
visible:ring-offset-2 disabled:cursor-not-allowed disabled:opacity-50",  
        className,  
      )}  
      {...props}  
    >
```

```

    <RadioGroupPrimitive.Indicator className="flex items-center justify-center">
      <Circle className="h-2.5 w-2.5 fill-current text-current" />
    </RadioGroupPrimitive.Indicator>
  </RadioGroupPrimitive.Item>
);
});
RadioGroupItem.displayName = RadioGroupPrimitive.Item.displayName;

export { RadioGroup, RadioGroupItem };

```

File: credit-card-fraud-detection/frontend/src/components/ui/resizable.jsx

```

import { GripVertical } from "lucide-react";
import * as ResizablePrimitive from "react-resizable-panels";

import { cn } from "@lib/utls";

const ResizablePanelGroup = ({ className, ...props }) => (
  <ResizablePrimitive.PanelGroup
    className={cn(
      "flex h-full w-full data-[panel-group-direction=vertical]:flex-col",
      className,
    )}
    {...props}
  />

```



```
);
```

```
const ResizablePanel = ResizablePrimitive.Panel;
```

```
const ResizableHandle = ({ withHandle, className, ...props }) => (
```

```
  <ResizablePrimitive.PanelResizeHandle
```

```
    className={cn(
```

```
      "relative flex w-px items-center justify-center bg-border after:absolute after:inset-y-0  
after:left-1/2 after:w-1 after:-translate-x-1/2 data-[panel-group-direction=vertical]:h-px  
data-[panel-group-direction=vertical]:w-full data-[panel-group-  
direction=vertical]:after:left-0 data-[panel-group-direction=vertical]:after:h-1 data-[panel-  
group-direction=vertical]:after:w-full data-[panel-group-direction=vertical]:after:-translate-  
y-1/2 data-[panel-group-direction=vertical]:after:translate-x-0 focus-visible:outline-none  
focus-visible:ring-1 focus-visible:ring-ring focus-visible:ring-offset-1 [&[data-panel-group-  
direction=vertical]>div]:rotate-90",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
  >
```

```
    {withHandle && (
```

```
      <div className="z-10 flex h-4 w-3 items-center justify-center rounded-sm border bg-  
border">
```

```
        <GripVertical className="h-2.5 w-2.5" />
```

```
      </div>
```

```
    )}
```

```
  </ResizablePrimitive.PanelResizeHandle>
```

```
);
```

```
export { ResizablePanelGroup, ResizablePanel, ResizableHandle };
```

File: credit-card-fraud-detection/frontend/src/components/ui/scroll-area.jsx

```
import * as React from "react";

import * as ScrollAreaPrimitive from "@radix-ui/react-scroll-area";

import { cn } from "@lib/utils";

const ScrollArea = React.forwardRef(
  ({ className, children, ...props }, ref) => (
    <ScrollAreaPrimitive.Root
      ref={ref}
      className={cn("relative overflow-hidden", className)}
      {...props}
    >
      <ScrollAreaPrimitive.Viewport className="h-full w-full rounded-[inherit]">
        {children}
      </ScrollAreaPrimitive.Viewport>
      <ScrollBar />
      <ScrollAreaPrimitive.Corner />
    </ScrollAreaPrimitive.Root>
  ),
);

ScrollArea.displayName = ScrollAreaPrimitive.Root.displayName;
```

```

const ScrollBar = React.forwardRef(
  ({ className, orientation = "vertical", ...props }, ref) => (
    <ScrollAreaPrimitive.ScrollAreaScrollbar
      ref={ref}
      orientation={orientation}
      className={cn(
        "flex touch-none select-none transition-colors",
        orientation === "vertical" &&
          "h-full w-2.5 border-l border-l-transparent p-[1px]",
        orientation === "horizontal" &&
          "h-2.5 flex-col border-t border-t-transparent p-[1px]",
        className,
      )}
      {...props}
    >
      <ScrollAreaPrimitive.ScrollAreaThumb className="relative flex-1 rounded-full bg-border" />
    </ScrollAreaPrimitive.ScrollAreaScrollbar>
  ),
);

ScrollBar.displayName = ScrollAreaPrimitive.ScrollAreaScrollbar.displayName;

export { ScrollArea, ScrollBar };

```

File: credit-card-fraud-detection/frontend/src/components/ui/select.jsx

```
import * as React from "react";

import * as SelectPrimitive from "@radix-ui/react-select";

import { Check, ChevronDown, ChevronUp } from "lucide-react";

import { cn } from "@lib/utils";

const Select = SelectPrimitive.Root;

const SelectGroup = SelectPrimitive.Group;

const SelectValue = SelectPrimitive.Value;

const SelectTrigger = React.forwardRef(
  ({ className, children, ...props }, ref) => (
    <SelectPrimitive.Trigger
      ref={ref}
      className={cn(
        "flex h-10 w-full items-center justify-between rounded-md border border-input bg-
background px-3 py-2 text-sm ring-offset-background placeholder:text-muted-foreground
focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:cursor-not-
allowed disabled:opacity-50 [>span]:line-clamp-1",
        className,
      )}
      {...props}
    >
      {children}
    </SelectPrimitive.Trigger>
  )
);
```

```

    <SelectPrimitive.Icon asChild>
      <ChevronDown className="h-4 w-4 opacity-50" />
    </SelectPrimitive.Icon>
  </SelectPrimitive.Trigger>
),
);

SelectTrigger.displayName = SelectPrimitive.Trigger.displayName;

const SelectScrollUpButton = React.forwardRef(
  ({ className, ...props }, ref) => (
    <SelectPrimitive.ScrollUpButton
      ref={ref}
      className={cn(
        "flex cursor-default items-center justify-center py-1",
        className,
      )}
      {...props}
    >
      <ChevronUp className="h-4 w-4" />
    </SelectPrimitive.ScrollUpButton>
  ),
);

SelectScrollUpButton.displayName = SelectPrimitive.ScrollUpButton.displayName;

const SelectScrollDownButton = React.forwardRef(
  ({ className, ...props }, ref) => (

```

```

<SelectPrimitive.ScrollDownButton
  ref={ref}
  className={cn(
    "flex cursor-default items-center justify-center py-1",
    className,
  )}
  {...props}
>
  <ChevronDown className="h-4 w-4" />
</SelectPrimitive.ScrollDownButton>
),
);

SelectScrollDownButton.displayName =
  SelectPrimitive.ScrollDownButton.displayName;

const SelectContent = React.forwardRef(
  ({ className, children, position = "popper", ...props }, ref) => (
    <SelectPrimitive.Portal>
      <SelectPrimitive.Content
        ref={ref}
        className={cn(
          "relative z-50 max-h-96 min-w-[8rem] overflow-hidden rounded-md border bg-
popover text-popover-foreground shadow-md data-[state=open]:animate-in data-
[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0 data-
[state=closed]:zoom-out-95 data-[state=open]:zoom-in-95 data-[side=bottom]:slide-in-
from-top-2 data-[side=left]:slide-in-from-right-2 data-[side=right]:slide-in-from-left-2 data-
[side=top]:slide-in-from-bottom-2",
          position === "popper" &&

```

```

      "data-[side=bottom]:translate-y-1 data-[side=left]:-translate-x-1 data-
[side=right]:translate-x-1 data-[side=top]:-translate-y-1",
      className,
    )}
    position={position}
    {...props}
  >
    <SelectScrollUpButton />
    <SelectPrimitive.Viewport
      className={cn(
        "p-1",
        position === "popper" &&
          "h-[var(--radix-select-trigger-height)] w-full min-w-[var(--radix-select-trigger-
width)]",
      )}
    >
      {children}
    </SelectPrimitive.Viewport>
    <SelectScrollDownButton />
  </SelectPrimitive.Content>
</SelectPrimitive.Portal>
),
);

SelectContent.displayName = SelectPrimitive.Content.displayName;

```

```

const SelectLabel = React.forwardRef(({ className, ...props }, ref) => (

```

```

<SelectPrimitive.Label
  ref={ref}
  className={cn("py-1.5 pl-8 pr-2 text-sm font-semibold", className)}
  {...props}
/>
));
SelectLabel.displayName = SelectPrimitive.Label.displayName;

const SelectItem = React.forwardRef(
  ({ className, children, ...props }, ref) => (
    <SelectPrimitive.Item
      ref={ref}
      className={cn(
        "relative flex w-full cursor-default select-none items-center rounded-sm py-1.5 pl-8 pr-
2 text-sm outline-none data-[disabled]:pointer-events-none data-[disabled]:opacity-50
focus:bg-accent focus:text-accent-foreground",
        className,
      )}
      {...props}
    >
      <span className="absolute left-2 flex h-3.5 w-3.5 items-center justify-center">
        <SelectPrimitive.ItemIndicator>
          <Check className="h-4 w-4" />
        </SelectPrimitive.ItemIndicator>
      </span>
    </span>
  )
);

```



```

    <SelectPrimitive.ItemText>{children}</SelectPrimitive.ItemText>

    </SelectPrimitive.Item>

  ),

);

SelectItem.displayName = SelectPrimitive.Item.displayName;

const SelectSeparator = React.forwardRef(({ className, ...props }, ref) => (

  <SelectPrimitive.Separator

    ref={ref}

    className={cn("-mx-1 my-1 h-px bg-muted", className)}

    {...props}

  />

));

SelectSeparator.displayName = SelectPrimitive.Separator.displayName;

export {

  Select,

  SelectGroup,

  SelectValue,

  SelectTrigger,

  SelectContent,

  SelectLabel,

  SelectItem,

  SelectSeparator,

  SelectScrollUpButton,

  SelectScrollDownButton,

```

```
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/separator.jsx

```
import * as React from "react";
```

```
import * as SeparatorPrimitive from "@radix-ui/react-separator";
```

```
import { cn } from "@lib/utls";
```

```
const Separator = React.forwardRef(  
  (  
    { className, orientation = "horizontal", decorative = true, ...props },  
    ref,  
  ) => (  
    <SeparatorPrimitive.Root  
      ref={ref}  
      decorative={decorative}  
      orientation={orientation}  
      className={cn(  
        "shrink-0 bg-border",  
        orientation === "horizontal" ? "h-[1px] w-full" : "h-full w-[1px]",  
        className,  
      )}  
      {...props}  
    />
```

```
),  
);  
  
Separator.displayName = SeparatorPrimitive.Root.displayName;  
  
export { Separator };
```

File: credit-card-fraud-detection/frontend/src/components/ui/sheet.jsx

```
import * as SheetPrimitive from "@radix-ui/react-dialog";  
import { cva } from "class-variance-authority";  
import { X } from "lucide-react";  
import * as React from "react";  
  
import { cn } from "@lib/utils";  
  
const Sheet = SheetPrimitive.Root;  
  
const SheetTrigger = SheetPrimitive.Trigger;  
  
const SheetClose = SheetPrimitive.Close;  
  
const SheetPortal = SheetPrimitive.Portal;  
  
const SheetOverlay = React.forwardRef(({ className, ...props }, ref) => (  
  <SheetPrimitive.Overlay
```

```

className={cn(
  "fixed inset-0 z-50 bg-black/80 data-[state=open]:animate-in data-
[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=open]:fade-in-0",
  className,
)}
{...props}
ref={ref}
/>
));

```

```

SheetOverlay.displayName = SheetPrimitive.Overlay.displayName;

```

```

const sheetVariants = cva(
  "fixed z-50 gap-4 bg-background p-6 shadow-lg transition ease-in-out data-
[state=open]:animate-in data-[state=closed]:animate-out data-[state=closed]:duration-300
data-[state=open]:duration-500",
  {
    variants: {
      side: {
        top: "inset-x-0 top-0 border-b data-[state=closed]:slide-out-to-top data-
[state=open]:slide-in-from-top",
        bottom:
          "inset-x-0 bottom-0 border-t data-[state=closed]:slide-out-to-bottom data-
[state=open]:slide-in-from-bottom",
        left: "inset-y-0 left-0 h-full w-3/4 border-r data-[state=closed]:slide-out-to-left data-
[state=open]:slide-in-from-left sm:max-w-sm",
        right:
          "inset-y-0 right-0 h-full w-3/4 border-l data-[state=closed]:slide-out-to-right data-
[state=open]:slide-in-from-right sm:max-w-sm",

```

```

    },
  },
  defaultVariants: {
    side: "right",
  },
},
);

```

```

const SheetContent = React.forwardRef(
  ({ side = "right", className, children, ...props }, ref) => (
    <SheetPortal>
      <SheetOverlay />
      <SheetPrimitive.Content
        ref={ref}
        className={cn(sheetVariants({ side }), className)}
        {...props}
      >
        {children}
        <SheetPrimitive.Close className="absolute right-4 top-4 rounded-sm opacity-70
ring-offset-background transition-opacity data-[state=open]:bg-secondary hover:opacity-
100 focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2 disabled:pointer-
events-none">
          <X className="h-4 w-4" />
          <span className="sr-only">Close</span>
        </SheetPrimitive.Close>
      </SheetPrimitive.Content>
    </SheetPortal>
  )

```

```
),  
);  
SheetContent.displayName = SheetPrimitive.Content.displayName;
```

```
const SheetHeader = ({ className, ...props }) => (  
  <div  
    className={cn(  
      "flex flex-col space-y-2 text-center sm:text-left",  
      className,  
    )}  
    {...props}  
  />  
);  
SheetHeader.displayName = "SheetHeader";
```

```
const SheetFooter = ({ className, ...props }) => (  
  <div  
    className={cn(  
      "flex flex-col-reverse sm:flex-row sm:justify-end sm:space-x-2",  
      className,  
    )}  
    {...props}  
  />  
);  
SheetFooter.displayName = "SheetFooter";
```

```
const SheetTitle = React.forwardRef(({ className, ...props }, ref) => (  
  <SheetPrimitive.Title  
    ref={ref}  
    className={cn("text-lg font-semibold text-foreground", className)}  
    {...props}  
  />  
));  
  
SheetTitle.displayName = SheetPrimitive.Title.displayName;  
  
const SheetDescription = React.forwardRef(({ className, ...props }, ref) => (  
  <SheetPrimitive.Description  
    ref={ref}  
    className={cn("text-sm text-muted-foreground", className)}  
    {...props}  
  />  
));  
  
SheetDescription.displayName = SheetPrimitive.Description.displayName;  
  
export {  
  Sheet,  
  SheetClose,  
  SheetContent,  
  SheetDescription,  
  SheetFooter,  
  SheetHeader,  
  SheetOverlay,
```

```
SheetPortal,  
SheetTitle,  
SheetTrigger,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/sidebar.jsx

```
import * as React from "react";  
  
import { Slot } from "@radix-ui/react-slot";  
  
import { cva } from "class-variance-authority";  
  
import { PanelLeft } from "lucide-react";  
  
  
  
import { useIsMobile } from "@/hooks/use-mobile";  
  
import { cn } from "@/lib/utils";  
  
import { Button } from "@components/ui/button";  
  
import { Input } from "@components/ui/input";  
  
import { Separator } from "@components/ui/separator";  
  
import { Sheet, SheetContent } from "@components/ui/sheet";  
  
import { Skeleton } from "@components/ui/skeleton";  
  
import {  
  Tooltip,  
  TooltipContent,  
  TooltipProvider,  
  TooltipTrigger,  
} from "@components/ui/tooltip";
```



```
const SIDEBAR_COOKIE_NAME = "sidebar:state";

const SIDEBAR_COOKIE_MAX_AGE = 60 * 60 * 24 * 7;

const SIDEBAR_WIDTH = "16rem";

const SIDEBAR_WIDTH_MOBILE = "18rem";

const SIDEBAR_WIDTH_ICON = "3rem";

const SIDEBAR_KEYBOARD_SHORTCUT = "b";

const SidebarContext = React.createContext(null);

function useSidebar() {
  const context = React.useContext(SidebarContext);
  if (!context) {
    throw new Error("useSidebar must be used within a SidebarProvider.");
  }

  return context;
}

const SidebarProvider = React.forwardRef(
  (
    {
      defaultOpen = true,
      open: openProp,
      onOpenChange: setOpenProp,
      className,
```

```

    style,

    children,

    ...props
  },

  ref,
) => {

  const isMobile = useIsMobile();

  const [openMobile, setOpenMobile] = React.useState(false);

  // This is the internal state of the sidebar.

  // We use openProp and setOpenProp for control from outside the component.

  const [_open, _setOpen] = React.useState(defaultOpen);

  const open = openProp ?? _open;

  const setOpen = React.useCallback(

    (value) => {

      const openState = typeof value === "function" ? value(open) : value;

      if (setOpenProp) {

        setOpenProp(openState);

      } else {

        _setOpen(openState);

      }

    },

    // This sets the cookie to keep the sidebar state.

    document.cookie = `${SIDEBAR_COOKIE_NAME}=${openState}; path=/; max-age=${SIDEBAR_COOKIE_MAX_AGE}`;

  ),

```

```

    [setOpenProp, open],
  );

  // Helper to toggle the sidebar.
  const toggleSidebar = React.useCallback(() => {
    return isMobile
      ? setOpenMobile((open) => !open)
      : setOpen((open) => !open);
  }, [isMobile, setOpen, setOpenMobile]);

  // Adds a keyboard shortcut to toggle the sidebar.
  React.useEffect(() => {
    const handleKeyDown = (event) => {
      if (
        event.key === SIDEBAR_KEYBOARD_SHORTCUT &&
        (event.metaKey || event.ctrlKey)
      ) {
        event.preventDefault();
        toggleSidebar();
      }
    };

    window.addEventListener("keydown", handleKeyDown);

    return () => window.removeEventListener("keydown", handleKeyDown);
  }, [toggleSidebar]);

```

```
// We add a state so that we can do data-state="expanded" or "collapsed".
```

```
// This makes it easier to style the sidebar with Tailwind classes.
```

```
const state = open ? "expanded" : "collapsed";
```

```
const contextValue = React.useMemo(  
  () => ({  
    state,  
    open,  
    setOpen,  
    isMobile,  
    openMobile,  
    setOpenMobile,  
    toggleSidebar,  
  }),  
  [  
    state,  
    open,  
    setOpen,  
    isMobile,  
    openMobile,  
    setOpenMobile,  
    toggleSidebar,  
  ],  
);  
  
return (  

```

```

<SidebarContext.Provider value={contextValue}>

  <TooltipProvider delayDuration={0}>

    <div

      style={{

        "--sidebar-width": SIDEBAR_WIDTH,

        "--sidebar-width-icon": SIDEBAR_WIDTH_ICON,

        ...style,

      }}

      className={cn(

        "group/sidebar-wrapper flex min-h-svh w-full has-[[data-variant=inset]]:bg-
sidebar",

        className,

      )}

      ref={ref}

      {...props}

    >

      {children}

    </div>

  </TooltipProvider>

</SidebarContext.Provider>

);

},

);

SidebarProvider.displayName = "SidebarProvider";

const Sidebar = React.forwardRef(

```

```
(  
  {  
    side = "left",  
    variant = "sidebar",  
    collapsible = "offcanvas",  
    className,  
    children,  
    ...props  
  },  
  ref,  
) => {  
  const { isMobile, state, openMobile, setOpenMobile } = useSidebar();  
  
  if (collapsible === "none") {  
    return (  
      <div  
        className={cn(  
          "flex h-full w-[--sidebar-width] flex-col bg-sidebar text-sidebar-foreground",  
          className,  
        )}  
        ref={ref}  
        {...props}  
      >  
        {children}  
      </div>  
    );  
  }  
}
```

```
}
```

```
if (isMobile) {
```

```
  return (
```

```
    <Sheet open={openMobile} onOpenChange={setOpenMobile} {...props}>
```

```
      <SheetContent
```

```
        data-sidebar="sidebar"
```

```
        data-mobile="true"
```

```
        className="w-[--sidebar-width] bg-sidebar p-0 text-sidebar-foreground  
[&>button]:hidden"
```

```
        style={{
```

```
          "--sidebar-width": SIDEBAR_WIDTH_MOBILE,
```

```
        }}
```

```
        side={side}
```

```
      >
```

```
        <div className="flex h-full w-full flex-col">{children}</div>
```

```
      </SheetContent>
```

```
    </Sheet>
```

```
  );
```

```
}
```

```
return (
```

```
  <div
```

```
    ref={ref}
```

```
    className="group peer hidden text-sidebar-foreground md:block"
```

```
    data-state={state}
```

```

data-collapsible={state === "collapsed" ? collapsible : ""}

data-variant={variant}

data-side={side}
>

{/* This is what handles the sidebar gap on desktop */}

<div

  className={cn(

    "relative h-svh w-[--sidebar-width] bg-transparent transition-[width] duration-200
ease-linear",

    "group-data-[collapsible=offcanvas]:w-0",

    "group-data-[side=right]:rotate-180",

    variant === "floating" || variant === "inset"

    ? "group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-
icon)+_theme(spacing.4))]"

    : "group-data-[collapsible=icon]:w-[--sidebar-width-icon]",

  )}

/>

<div

  className={cn(

    "fixed inset-y-0 z-10 hidden h-svh w-[--sidebar-width] transition-[left,right,width]
duration-200 ease-linear md:flex",

    side === "left"

    ? "left-0 group-data-[collapsible=offcanvas]:left-[calc(var(--sidebar-width)*-1)]"

    : "right-0 group-data-[collapsible=offcanvas]:right-[calc(var(--sidebar-width)*-1)]",

    // Adjust the padding for floating and inset variants.

    variant === "floating" || variant === "inset"

```



```

      ? "p-2 group-data-[collapsible=icon]:w-[calc(var(--sidebar-width-
icon)_+_theme(spacing.4)_+2px)]"

      : "group-data-[collapsible=icon]:w-[--sidebar-width-icon] group-data-
[side=left]:border-r group-data-[side=right]:border-l",

      className,

    )}

    {...props}

  >

    <div

      data-sidebar="sidebar"

      className="flex h-full w-full flex-col bg-sidebar group-data-
[variant=floating]:rounded-lg group-data-[variant=floating]:border group-data-
[variant=floating]:border-sidebar-border group-data-[variant=floating]:shadow"

    >

      {children}

    </div>

  </div>

</div>

);

},

);

Sidebar.displayName = "Sidebar";

```

```

const SidebarTrigger = React.forwardRef(
  ({ className, onClick, ...props }, ref) => {
    const { toggleSidebar } = useSidebar();

```

```

return (
  <Button
    ref={ref}
    data-sidebar="trigger"
    variant="ghost"
    size="icon"
    className={cn("h-7 w-7", className)}
    onClick={{(event) => {
      onClick?.(event);
      toggleSidebar();
    }}}
    {...props}
  >
    <PanelLeft />
    <span className="sr-only">Toggle Sidebar</span>
  </Button>
);
},
);

SidebarTrigger.displayName = "SidebarTrigger";

const SidebarRail = React.forwardRef(({ className, ...props }, ref) => {
  const { toggleSidebar } = useSidebar();

  return (
    <button

```

```

    ref={ref}

    data-sidebar="rail"

    aria-label="Toggle Sidebar"

    tabIndex={-1}

    onClick={toggleSidebar}

    title="Toggle Sidebar"

    className={cn(
      "absolute inset-y-0 z-20 hidden w-4 -translate-x-1/2 transition-all ease-linear
after:absolute after:inset-y-0 after:left-1/2 after:w-[2px] group-data-[side=left]:-right-4
group-data-[side=right]:left-0 hover:after:bg-sidebar-border sm:flex",
      "[[data-side=left]_&]:cursor-w-resize [[data-side=right]_&]:cursor-e-resize",
      "[[data-side=left][data-state=collapsed]_&]:cursor-e-resize [[data-side=right][data-
state=collapsed]_&]:cursor-w-resize",
      "group-data-[collapsible=offcanvas]:translate-x-0 group-data-
[collapsible=offcanvas]:after:left-full group-data-[collapsible=offcanvas]:hover:bg-sidebar",
      "[[data-side=left][data-collapsible=offcanvas]_&]:-right-2",
      "[[data-side=right][data-collapsible=offcanvas]_&]:-left-2",
      className,
    )}

    {...props}
  />
);
});

```

```

SidebarRail.displayName = "SidebarRail";

```

```

const SidebarInset = React.forwardRef(({ className, ...props }, ref) => {
  return (

```

```

<main
  ref={ref}
  className={cn(
    "relative flex min-h-svh flex-1 flex-col bg-background",
    "peer-data-[variant=inset]:min-h-[calc(100svh-theme(spacing.4))] md:peer-data-[variant=inset]:m-2 md:peer-data-[state=collapsed]:peer-data-[variant=inset]:ml-2 md:peer-data-[variant=inset]:ml-0 md:peer-data-[variant=inset]:rounded-xl md:peer-data-[variant=inset]:shadow",
    className,
  )}
  {...props}
/>
);
});

```

```

SidebarInset.displayName = "SidebarInset";

```

```

const SidebarInput = React.forwardRef(({ className, ...props }, ref) => {
  return (
    <Input
      ref={ref}
      data-sidebar="input"
      className={cn(
        "h-8 w-full bg-background shadow-none focus-visible:ring-2 focus-visible:ring-sidebar-ring",
        className,
      )}
      {...props}
    >

```

```
    />

  );

});

SidebarInput.displayName = "SidebarInput";

const SidebarHeader = React.forwardRef(({ className, ...props }, ref) => {

  return (

    <div

      ref={ref}

      data-sidebar="header"

      className={cn("flex flex-col gap-2 p-2", className)}

      {...props}

    />

  );

});

SidebarHeader.displayName = "SidebarHeader";

const SidebarFooter = React.forwardRef(({ className, ...props }, ref) => {

  return (

    <div

      ref={ref}

      data-sidebar="footer"

      className={cn("flex flex-col gap-2 p-2", className)}

      {...props}

    />

  );

});
```

```
});
```

```
SidebarFooter.displayName = "SidebarFooter";
```

```
const SidebarSeparator = React.forwardRef(({ className, ...props }, ref) => {
```

```
  return (
```

```
    <Separator
```

```
      ref={ref}
```

```
      data-sidebar="separator"
```

```
      className={cn("mx-2 w-auto bg-sidebar-border", className)}
```

```
      {...props}
```

```
    />
```

```
  );
```

```
});
```

```
SidebarSeparator.displayName = "SidebarSeparator";
```

```
const SidebarContent = React.forwardRef(({ className, ...props }, ref) => {
```

```
  return (
```

```
    <div
```

```
      ref={ref}
```

```
      data-sidebar="content"
```

```
      className={cn(
```

```
        "flex min-h-0 flex-1 flex-col gap-2 overflow-auto group-data-[collapsible=icon]:overflow-hidden",
```

```
        className,
```

```
      )}
```

```
      {...props}
```

```

    />

  );

});

SidebarContent.displayName = "SidebarContent";

const SidebarGroup = React.forwardRef(({ className, ...props }, ref) => {

  return (

    <div

      ref={ref}

      data-sidebar="group"

      className={cn("relative flex w-full min-w-0 flex-col p-2", className)}

      {...props}

    />

  );

});

SidebarGroup.displayName = "SidebarGroup";

```

```

const SidebarGroupLabel = React.forwardRef(

  ({ className, asChild = false, ...props }, ref) => {

    const Comp = asChild ? Slot : "div";

    return (

      <Comp

        ref={ref}

        data-sidebar="group-label"

        className={cn(

```

```

    "flex h-8 shrink-0 items-center rounded-md px-2 text-xs font-medium text-sidebar-
foreground/70 outline-none ring-sidebar-ring transition-[margin,opa] duration-200 ease-
linear focus-visible:ring-2 [&>svg]:size-4 [&>svg]:shrink-0",

    "group-data-[collapsible=icon]:-mt-8 group-data-[collapsible=icon]:opacity-0",

    className,

  )}

  {...props}

/>

);

},

);

SidebarGroupLabel.displayName = "SidebarGroupLabel";

const SidebarGroupAction = React.forwardRef(

  ({ className, asChild = false, ...props }, ref) => {

    const Comp = asChild ? Slot : "button";

    return (

      <Comp

        ref={ref}

        data-sidebar="group-action"

        className={cn(

          "absolute right-3 top-3.5 flex aspect-square w-5 items-center justify-center rounded-
md p-0 text-sidebar-foreground outline-none ring-sidebar-ring transition-transform
hover:bg-sidebar-accent hover:text-sidebar-accent-foreground focus-visible:ring-2
[&>svg]:size-4 [&>svg]:shrink-0",

          // Increases the hit area of the button on mobile.

          "after:absolute after:-inset-2 after:md:hidden",

```



```
      "group-data-[collapsible=icon]:hidden",
      className,
    )}
    {...props}
  />

);

},

);

SidebarGroupAction.displayName = "SidebarGroupAction";
```

```
const SidebarGroupContent = React.forwardRef(({ className, ...props }, ref) => (

  <div

    ref={ref}

    data-sidebar="group-content"

    className={cn("w-full text-sm", className)}

    {...props}

  />

));

SidebarGroupContent.displayName = "SidebarGroupContent";
```

```
const SidebarMenu = React.forwardRef(({ className, ...props }, ref) => (

  <ul

    ref={ref}

    data-sidebar="menu"

    className={cn("flex w-full min-w-0 flex-col gap-1", className)}

    {...props}

  />

));

SidebarMenu.displayName = "SidebarMenu";
```

```

/>

));

SidebarMenu.displayName = "SidebarMenu";

const SidebarMenuItem = React.forwardRef(({ className, ...props }, ref) => (

  <li

    ref={ref}

    data-sidebar="menu-item"

    className={cn("group/menu-item relative", className)}

    {...props}

  />

));

SidebarMenuItem.displayName = "SidebarMenuItem";

const sidebarMenuButtonVariants = cva(

  "peer/menu-button flex w-full items-center gap-2 overflow-hidden rounded-md p-2 text-left text-sm outline-none ring-sidebar-ring transition-[width,height,padding] hover:bg-sidebar-accent hover:text-sidebar-accent-foreground focus-visible:ring-2 active:bg-sidebar-accent active:text-sidebar-accent-foreground disabled:pointer-events-none disabled:opacity-50 group-has-[[data-sidebar=menu-action]]/menu-item:pr-8 aria-disabled:pointer-events-none aria-disabled:opacity-50 data-[active=true]:bg-sidebar-accent data-[active=true]:font-medium data-[active=true]:text-sidebar-accent-foreground data-[state=open]:hover:bg-sidebar-accent data-[state=open]:hover:text-sidebar-accent-foreground group-data-[collapsible=icon]:!size-8 group-data-[collapsible=icon]:!p-2 [>span:last-child]:truncate [>svg]:size-4 [>svg]:shrink-0",

  {

    variants: {

      variant: {

        default: "hover:bg-sidebar-accent hover:text-sidebar-accent-foreground",

```

outline:

```
"bg-background shadow-[0_0_0_1px_hsl(var(--sidebar-border))] hover:bg-sidebar-  
accent hover:text-sidebar-accent-foreground hover:shadow-[0_0_0_1px_hsl(var(--sidebar-  
accent))]"
```

```
},
```

```
size: {
```

```
  default: "h-8 text-sm",
```

```
  sm: "h-7 text-xs",
```

```
  lg: "h-12 text-sm group-data-[collapsible=icon]:!p-0",
```

```
},
```

```
},
```

```
defaultVariants: {
```

```
  variant: "default",
```

```
  size: "default",
```

```
},
```

```
},
```

```
);
```

```
const SidebarMenuButton = React.forwardRef(  
  
  (  
  
    {  
  
      asChild = false,  
  
      isActive = false,  
  
      variant = "default",  
  
      size = "default",  
  
      tooltip,
```

```

    className,

    ...props
  },
  ref,
) => {

  const Comp = asChild ? Slot : "button";
  const { isMobile, state } = useSidebar();

  const button = (
    <Comp
      ref={ref}
      data-sidebar="menu-button"
      data-size={size}
      data-active={isActive}
      className={cn(sidebarMenuButtonVariants({ variant, size }), className)}
      {...props}
    />
  );

  if (!tooltip) {
    return button;
  }

  if (typeof tooltip === "string") {
    tooltip = {
      children: tooltip,

```

```

    };
}

return (
  <Tooltip>
    <TooltipTrigger asChild>{button}</TooltipTrigger>
    <TooltipContent
      side="right"
      align="center"
      hidden={state !== "collapsed" || isMobile}
      {...tooltip}
    />
  </Tooltip>
);
},
);

SidebarMenuButton.displayName = "SidebarMenuButton";

const SidebarMenuAction = React.forwardRef(
  ({ className, asChild = false, showOnHover = false, ...props }, ref) => {
    const Comp = asChild ? Slot : "button";

    return (
      <Comp
        ref={ref}
        data-sidebar="menu-action"

```

```

className={cn(
  "absolute right-1 top-1.5 flex aspect-square w-5 items-center justify-center rounded-
md p-0 text-sidebar-foreground outline-none ring-sidebar-ring transition-transform peer-
hover/menu-button:text-sidebar-accent-foreground hover:bg-sidebar-accent hover:text-
sidebar-accent-foreground focus-visible:ring-2 [>svg]:size-4 [>svg]:shrink-0",
  // Increases the hit area of the button on mobile.
  "after:absolute after:-inset-2 after:md:hidden",
  "peer-data-[size=sm]/menu-button:top-1",
  "peer-data-[size=default]/menu-button:top-1.5",
  "peer-data-[size=lg]/menu-button:top-2.5",
  "group-data-[collapsible=icon]:hidden",
  showOnHover &&
  "group-focus-within/menu-item:opacity-100 group-hover/menu-item:opacity-100
data-[state=open]:opacity-100 peer-data-[active=true]/menu-button:text-sidebar-accent-
foreground md:opacity-0",
  className,
)}
{...props}
/>
);
},
);
SidebarMenuAction.displayName = "SidebarMenuAction";

const SidebarMenuBadge = React.forwardRef(({ className, ...props }, ref) => (
  <div
    ref={ref}
    data-sidebar="menu-badge"

```

```

    className={cn(
      "pointer-events-none absolute right-1 flex h-5 min-w-5 select-none items-center justify-
center rounded-md px-1 text-xs font-medium tabular-nums text-sidebar-foreground",
      "peer-hover/menu-button:text-sidebar-accent-foreground peer-data-[active=true]/menu-
button:text-sidebar-accent-foreground",
      "peer-data-[size=sm]/menu-button:top-1",
      "peer-data-[size=default]/menu-button:top-1.5",
      "peer-data-[size=lg]/menu-button:top-2.5",
      "group-data-[collapsible=icon]:hidden",
      className,
    )}
    {...props}
  />
));

SidebarMenuBadge.displayName = "SidebarMenuBadge";

```

```

const SidebarMenuSkeleton = React.forwardRef(
  ({ className, showIcon = false, ...props }, ref) => {
    // Random width between 50 to 90%.
    const width = React.useMemo(() => {
      return `${Math.floor(Math.random() * 40) + 50}%`;
    }, []);

    return (
      <div
        ref={ref}

```

```

data-sidebar="menu-skeleton"

className={cn("flex h-8 items-center gap-2 rounded-md px-2", className)}

{...props}
>

{showIcon && (
  <Skeleton
    className="size-4 rounded-md"
    data-sidebar="menu-skeleton-icon"
  />
)}

<Skeleton
  className="h-4 max-w-[--skeleton-width] flex-1"
  data-sidebar="menu-skeleton-text"
  style={{
    "--skeleton-width": width,
  }}
/>
</div>

);

},

);

SidebarMenuSkeleton.displayName = "SidebarMenuSkeleton";

const SidebarMenuSub = React.forwardRef(({ className, ...props }, ref) => (
  <ul
    ref={ref}

```



```

data-sidebar="menu-sub"

className={cn(
  "mx-3.5 flex min-w-0 translate-x-px flex-col gap-1 border-l border-sidebar-border px-
2.5 py-0.5",
  "group-data-[collapsible=icon]:hidden",
  className,
)}
{...props}
/>
));

```

```

SidebarMenuSub.displayName = "SidebarMenuSub";

```

```

const SidebarMenuSubItem = React.forwardRef(({ ...props }, ref) => (
  <li ref={ref} {...props} />
));

```

```

SidebarMenuSubItem.displayName = "SidebarMenuSubItem";

```

```

const SidebarMenuSubButton = React.forwardRef(
  ({ asChild = false, size = "md", isActive, className, ...props }, ref) => {
    const Comp = asChild ? Slot : "a";

    return (
      <Comp
        ref={ref}
        data-sidebar="menu-sub-button"
        data-size={size}

```

```

data-active={isActive}

className={cn(
  "flex h-7 min-w-0 -translate-x-px items-center gap-2 overflow-hidden rounded-md px-
2 text-sidebar-foreground outline-none ring-sidebar-ring aria-disabled:pointer-events-none
aria-disabled:opacity-50 hover:bg-sidebar-accent hover:text-sidebar-accent-foreground
focus-visible:ring-2 active:bg-sidebar-accent active:text-sidebar-accent-foreground
disabled:pointer-events-none disabled:opacity-50 [>span:last-child]:truncate [>svg]:size-
4 [>svg]:shrink-0 [>svg]:text-sidebar-accent-foreground",
  "data-[active=true]:bg-sidebar-accent data-[active=true]:text-sidebar-accent-
foreground",
  size === "sm" && "text-xs",
  size === "md" && "text-sm",
  "group-data-[collapsible=icon]:hidden",
  className,
)}
{...props}
/>
);
},
);

SidebarMenuSubButton.displayName = "SidebarMenuSubButton";

export {
  Sidebar,
  SidebarContent,
  SidebarFooter,
  SidebarGroup,
  SidebarGroupAction,

```

```
SidebarGroupContent,  
SidebarGroupLabel,  
SidebarHeader,  
SidebarInput,  
SidebarInset,  
SidebarMenu,  
SidebarMenuAction,  
SidebarMenuBadge,  
SidebarMenuButton,  
SidebarMenuItem,  
SidebarMenuSkeleton,  
SidebarMenuSub,  
SidebarMenuSubButton,  
SidebarMenuSubItem,  
SidebarProvider,  
SidebarRail,  
SidebarSeparator,  
SidebarTrigger,  
useSidebar,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/skeleton.jsx

```
import { cn } from "@lib/utils";
```

```
function Skeleton({ className, ...props }) {  
  return (  
    <div  
      className={cn("animate-pulse rounded-md bg-muted", className)}  
      {...props}  
    />  
  );  
}  
  
export { Skeleton };
```

File: credit-card-fraud-detection/frontend/src/components/ui/slider.jsx

```
import * as React from "react";  
  
import * as SliderPrimitive from "@radix-ui/react-slider";  
  
import { cn } from "@lib/utils";  
  
const Slider = React.forwardRef(({ className, ...props }, ref) => (  
  <SliderPrimitive.Root  
    ref={ref}  
    className={cn(  
      "relative flex w-full touch-none select-none items-center",  
      className,  
    )}  
  />  
)
```

```

    {...props}
  >
    <SliderPrimitive.Track className="relative h-2 w-full grow overflow-hidden rounded-
full bg-secondary">
      <SliderPrimitive.Range className="absolute h-full bg-primary" />
    </SliderPrimitive.Track>
    <SliderPrimitive.Thumb className="block h-5 w-5 rounded-full border-2 border-
primary bg-background ring-offset-background transition-colors focus-visible:outline-
none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 disabled:pointer-
events-none disabled:opacity-50" />
  </SliderPrimitive.Root>
));
Slider.displayName = SliderPrimitive.Root.displayName;

export { Slider };

```

File: credit-card-fraud-detection/frontend/src/components/ui/sonner.jsx

```

import { useTheme } from "next-themes";

import { Toaster as Sonner, toast } from "sonner";

const Toaster = ({ ...props }) => {
  const { theme = "system" } = useTheme();

  return (
    <Sonner

```

```

    theme={theme}

    className="toaster group"

    toastOptions={{
      classNames: {
        toast:
          "group toast group-[.toaster]:bg-background group-[.toaster]:text-foreground
group-[.toaster]:border-border group-[.toaster]:shadow-lg",
        description: "group-[.toast]:text-muted-foreground",
        actionButton:
          "group-[.toast]:bg-primary group-[.toast]:text-primary-foreground",
        cancelButton:
          "group-[.toast]:bg-muted group-[.toast]:text-muted-foreground",
      },
    }}

    {...props}
  />

);

};

export { Toaster, toast };

```

File: credit-card-fraud-detection/frontend/src/components/ui/switch.jsx

```

import * as React from "react";

import * as SwitchPrimitives from "@radix-ui/react-switch";

```

```
import { cn } from "@lib/utils";
```

```
const Switch = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <SwitchPrimitives.Root
```

```
    className={cn(
```

```
      "peer inline-flex h-6 w-11 shrink-0 cursor-pointer items-center rounded-full border-2  
border-transparent transition-colors data-[state=checked]:bg-primary data-  
[state=unchecked]:bg-input focus-visible:outline-none focus-visible:ring-2 focus-  
visible:ring-ring focus-visible:ring-offset-2 focus-visible:ring-offset-background  
disabled:cursor-not-allowed disabled:opacity-50",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
    ref={ref}
```

```
  >
```

```
    <SwitchPrimitives.Thumb
```

```
      className={cn(
```

```
        "pointer-events-none block h-5 w-5 rounded-full bg-background shadow-lg ring-0  
transition-transform data-[state=checked]:translate-x-5 data-[state=unchecked]:translate-x-  
0",
```

```
      )}
```

```
    />
```

```
  </SwitchPrimitives.Root>
```

```
));
```

```
Switch.displayName = SwitchPrimitives.Root.displayName;
```

```
export { Switch };
```

File: credit-card-fraud-detection/frontend/src/components/ui/table.jsx

```
import * as React from "react";
```

```
import { cn } from "@lib/utils";
```

```
const Table = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <div className="relative w-full overflow-auto">
```

```
    <table
```

```
      ref={ref}
```

```
      className={cn("w-full caption-bottom text-sm", className)}
```

```
      {...props}
```

```
    />
```

```
  </div>
```

```
));
```

```
Table.displayName = "Table";
```

```
const TableHeader = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <thead ref={ref} className={cn("[&_tr]:border-b", className)} {...props} />
```

```
));
```

```
TableHeader.displayName = "TableHeader";
```

```
const TableBody = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <tbody
```



```
      ref={ref}

      className={cn("[&_tr:last-child]:border-0", className)}

      {...props}
    />
  ));

  TableBody.displayName = "TableBody";
```

```
const TableFooter = React.forwardRef(({ className, ...props }, ref) => (
  <tfoot
    ref={ref}

    className={cn(
      "border-t bg-muted/50 font-medium [&>tr]:last:border-b-0",
      className,
    )}

    {...props}
  />
));

TableFooter.displayName = "TableFooter";
```

```
const TableRow = React.forwardRef(({ className, ...props }, ref) => (
  <tr
    ref={ref}

    className={cn(
      "border-b transition-colors data-[state=selected]:bg-muted hover:bg-muted/50",
      className,
    )}

    {...props}
  />
));
```

```
    {...props}

  />

));

TableRow.displayName = "TableRow";
```

```
const TableHead = React.forwardRef(({ className, ...props }, ref) => (

  <th

    ref={ref}

    className={cn(

      "h-12 px-4 text-left align-middle font-medium text-muted-foreground

[&:has([role=checkbox])]:pr-0",

      className,

    )}

    {...props}

  />

));

TableHead.displayName = "TableHead";
```

```
const TableCell = React.forwardRef(({ className, ...props }, ref) => (

  <td

    ref={ref}

    className={cn("p-4 align-middle [&:has([role=checkbox])]:pr-0", className)}

    {...props}

  />

));

TableCell.displayName = "TableCell";
```

```
const TableCaption = React.forwardRef(({ className, ...props }, ref) => (  
  <caption  
    ref={ref}  
    className={cn("mt-4 text-sm text-muted-foreground", className)}  
    {...props}  
  />  
));  
TableCaption.displayName = "TableCaption";
```

```
export {  
  Table,  
  TableHeader,  
  TableBody,  
  TableFooter,  
  TableHead,  
  TableRow,  
  TableCell,  
  TableCaption,  
};
```

File: credit-card-fraud-detection/frontend/src/components/ui/tabs.jsx

```
import * as React from "react";  
import * as TabsPrimitive from "@radix-ui/react-tabs";
```

```
import { cn } from "@lib/utils";
```

```
const Tabs = TabsPrimitive.Root;
```

```
const TabsList = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <TabsPrimitive.List
```

```
    ref={ref}
```

```
    className={cn(
```

```
      "inline-flex h-10 items-center justify-center rounded-md bg-muted p-1 text-muted-foreground",
```

```
      className,
```

```
    )}
```

```
    {...props}
```

```
  />
```

```
));
```

```
TabsList.displayName = TabsPrimitive.List.displayName;
```

```
const TabsTrigger = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <TabsPrimitive.Trigger
```

```
    ref={ref}
```

```
    className={cn(
```

```
      "inline-flex items-center justify-center whitespace-nowrap rounded-sm px-3 py-1.5 text-sm font-medium ring-offset-background transition-all data-[state=active]:bg-background data-[state=active]:text-foreground data-[state=active]:shadow-sm focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 disabled:pointer-events-none disabled:opacity-50",
```

```
      className,
```

```

    }}

    {...props}

  />

));

TabsTrigger.displayName = TabsPrimitive.Trigger.displayName;

const TabsContent = React.forwardRef(({ className, ...props }, ref) => (
  <TabsPrimitive.Content
    ref={ref}
    className={cn(
      "mt-2 ring-offset-background focus-visible:outline-none focus-visible:ring-2 focus-
visible:ring-ring focus-visible:ring-offset-2",
      className,
    )}
    {...props}
  />
));

TabsContent.displayName = TabsPrimitive.Content.displayName;

export { Tabs, TabsList, TabsTrigger, TabsContent };

```

File: credit-card-fraud-detection/frontend/src/components/ui/textarea.jsx

```
import * as React from "react";
```

```

import { cn } from "@lib/utils";

const Textarea = React.forwardRef(({ className, ...props }, ref) => {

  return (

    <textarea

      className={cn(

        "flex min-h-[80px] w-full rounded-md border border-input bg-background px-3 py-2
        text-sm ring-offset-background placeholder:text-muted-foreground focus-visible:outline-
        none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 disabled:cursor-
        not-allowed disabled:opacity-50",

        className,

      )}

      ref={ref}

      {...props}

    />

  );

});

Textarea.displayName = "Textarea";

export { Textarea };

```

File: credit-card-fraud-detection/frontend/src/components/ui/toast.jsx

```

import * as React from "react";

import * as ToastPrimitives from "@radix-ui/react-toast";

import { cva } from "class-variance-authority";

```

```
import { X } from "lucide-react";
```

```
import { cn } from "@lib/utls";
```

```
const ToastProvider = ToastPrimitives.Provider;
```

```
const ToastViewport = React.forwardRef(({ className, ...props }, ref) => (  
  <ToastPrimitives.Viewport  
    ref={ref}  
    className={cn(  
      "fixed top-0 z-[100] flex max-h-screen w-full flex-col-reverse p-4 sm:bottom-0 sm:right-0  
sm:top-auto sm:flex-col md:max-w-[420px]",  
      className,  
    )}  
    {...props}  
  />  
));  
ToastViewport.displayName = ToastPrimitives.Viewport.displayName;
```

```
const toastVariants = cva(  
  "group pointer-events-auto relative flex w-full items-center justify-between space-x-4  
overflow-hidden rounded-md border p-6 pr-8 shadow-lg transition-all data-  
[swipe=cancel]:translate-x-0 data-[swipe=end]:translate-x-[var(--radix-toast-swipe-end-x)]  
data-[swipe=move]:translate-x-[var(--radix-toast-swipe-move-x)] data-  
[swipe=move]:transition-none data-[state=open]:animate-in data-[state=closed]:animate-  
out data-[swipe=end]:animate-out data-[state=closed]:fade-out-80 data-[state=closed]:slide-  
out-to-right-full data-[state=open]:slide-in-from-top-full data-[state=open]:sm:slide-in-  
from-bottom-full",  
  {  

```

```

variants: {
  variant: {
    default: "border bg-background text-foreground",
    destructive:
      "destructive group border-destructive bg-destructive text-destructive-foreground",
  },
},
defaultVariants: {
  variant: "default",
},
},
);

```

```

const Toast = React.forwardRef(({ className, variant, ...props }, ref) => {
  return (
    <ToastPrimitives.Root
      ref={ref}
      className={cn(toastVariants({ variant }), className)}
      {...props}
    />
  );
});

```

```

Toast.displayName = ToastPrimitives.Root.displayName;

```

```

const ToastAction = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Action

```



```
ref={ref}
```

```
className={cn(
```

```
  "inline-flex h-8 shrink-0 items-center justify-center rounded-md border bg-transparent
  px-3 text-sm font-medium ring-offset-background transition-colors group-
  [.destructive]:border-muted/40 hover:bg-secondary group-[.destructive]:hover:border-
  destructive/30 group-[.destructive]:hover:bg-destructive group-[.destructive]:hover:text-
  destructive-foreground focus:outline-none focus:ring-2 focus:ring-ring focus:ring-offset-2
  group-[.destructive]:focus:ring-destructive disabled:pointer-events-none disabled:opacity-
  50",
```

```
    className,
```

```
  )}
```

```
  {...props}
```

```
/>
```

```
));
```

```
ToastAction.displayName = ToastPrimitives.Action.displayName;
```

```
const ToastClose = React.forwardRef(({ className, ...props }, ref) => (
```

```
  <ToastPrimitives.Close
```

```
    ref={ref}
```

```
    className={cn(
```

```
      "absolute right-2 top-2 rounded-md p-1 text-foreground/50 opacity-0 transition-opacity
      group-hover:opacity-100 group-[.destructive]:text-red-300 hover:text-foreground group-
      [.destructive]:hover:text-red-50 focus:opacity-100 focus:outline-none focus:ring-2 group-
      [.destructive]:focus:ring-red-400 group-[.destructive]:focus:ring-offset-red-600",
```

```
      className,
```

```
    )}
```

```
    toast-close=""
```

```
    {...props}
```

```
>
```

```
<X className="h-4 w-4" />

</ToastPrimitives.Close>

));

ToastClose.displayName = ToastPrimitives.Close.displayName;

const ToastTitle = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Title
    ref={ref}
    className={cn("text-sm font-semibold", className)}
    {...props}
  />
));

ToastTitle.displayName = ToastPrimitives.Title.displayName;

const ToastDescription = React.forwardRef(({ className, ...props }, ref) => (
  <ToastPrimitives.Description
    ref={ref}
    className={cn("text-sm opacity-90", className)}
    {...props}
  />
));

ToastDescription.displayName = ToastPrimitives.Description.displayName;

export {
  ToastProvider,
  ToastViewport,
```

```
    Toast,  
    ToastTitle,  
    ToastDescription,  
    ToastClose,  
    ToastAction,  
  };
```

File: credit-card-fraud-detection/frontend/src/components/ui/toaster.jsx

```
import { useToast } from "@hooks/use-toast";  
  
import {  
  Toast,  
  ToastClose,  
  ToastDescription,  
  ToastProvider,  
  ToastTitle,  
  ToastViewport,  
} from "@components/ui/toast";  
  
export function Toaster() {  
  const { toasts } = useToast();  
  
  return (  
    <ToastProvider>  
      {toasts.map(function ({ id, title, description, action, ...props }) {
```

```

return (
  <Toast key={id} {...props}>
    <div className="grid gap-1">
      {title && <ToastTitle>{title}</ToastTitle>}
      {description && (
        <ToastDescription>{description}</ToastDescription>
      )}
    </div>
    {action}
    <ToastClose />
  </Toast>
);
}}
<ToastViewport />
</ToastProvider>
);
}

```

File: credit-card-fraud-detection/frontend/src/components/ui/toggle-group.jsx

```

import * as React from "react";

import * as ToggleGroupPrimitive from "@radix-ui/react-toggle-group";

import { cn } from "@lib/utls";

import { toggleVariants } from "@components/ui/toggle";

```

```
const ToggleGroupContext = React.createContext({  
  size: "default",  
  variant: "default",  
});
```

```
const ToggleGroup = React.forwardRef(  
  ({ className, variant, size, children, ...props }, ref) => (  
    <ToggleGroupPrimitive.Root  
      ref={ref}  
      className={cn("flex items-center justify-center gap-1", className)}  
      {...props}  
    >  
      <ToggleGroupContext.Provider value={{ variant, size }}>  
        {children}  
      </ToggleGroupContext.Provider>  
    </ToggleGroupPrimitive.Root>  
  ),  
);
```

```
ToggleGroup.displayName = ToggleGroupPrimitive.Root.displayName;
```

```
const ToggleGroupItem = React.forwardRef(  
  ({ className, children, variant, size, ...props }, ref) => {  
    const context = React.useContext(ToggleGroupContext);
```

```

return (
  <ToggleGroupPrimitive.Item
    ref={ref}
    className={cn(
      toggleVariants({
        variant: context.variant || variant,
        size: context.size || size,
      }),
      className,
    )}
    {...props}
  >
    {children}
  </ToggleGroupPrimitive.Item>
);
},
);

ToggleGroupItem.displayName = ToggleGroupPrimitive.Item.displayName;

export { ToggleGroup, ToggleGroupItem };

```

File: credit-card-fraud-detection/frontend/src/components/ui/toggle.jsx

```
import * as React from "react";
```

```
import * as TogglePrimitive from "@radix-ui/react-toggle";

import { cva } from "class-variance-authority";

import { cn } from "@lib/utils";

const toggleVariants = cva(
  "inline-flex items-center justify-center rounded-md text-sm font-medium ring-offset-background transition-colors hover:bg-muted hover:text-muted-foreground focus-visible:outline-none focus-visible:ring-2 focus-visible:ring-ring focus-visible:ring-offset-2 disabled:pointer-events-none disabled:opacity-50 data-[state=on]:bg-accent data-[state=on]:text-accent-foreground",
  {
    variants: {
      variant: {
        default: "bg-transparent",
        outline:
          "border border-input bg-transparent hover:bg-accent hover:text-accent-foreground",
      },
      size: {
        default: "h-10 px-3",
        sm: "h-9 px-2.5",
        lg: "h-11 px-5",
      },
    },
    defaultVariants: {
      variant: "default",
      size: "default",
    },
  }
);
```

```
    },  
    },  
  );  
  
const Toggle = React.forwardRef(  
  ({ className, variant, size, ...props }, ref) => (  
    <TogglePrimitive.Root  
      ref={ref}  
      className={cn(toggleVariants({ variant, size, className }))}  
      {...props}  
    />  
  ),  
);
```

```
Toggle.displayName = TogglePrimitive.Root.displayName;
```

```
export { Toggle, toggleVariants };
```

File: credit-card-fraud-detection/frontend/src/components/ui/tooltip.jsx

```
import * as React from "react";
```

```
import * as TooltipPrimitive from "@radix-ui/react-tooltip";
```

```
import { cn } from "@lib/utils";
```



```
const TooltipProvider = TooltipPrimitive.Provider;
```

```
const Tooltip = TooltipPrimitive.Root;
```

```
const TooltipTrigger = TooltipPrimitive.Trigger;
```

```
const TooltipContent = React.forwardRef(  
  ({ className, sideOffset = 4, ...props }, ref) => (  
    <TooltipPrimitive.Content  
      ref={ref}  
      sideOffset={sideOffset}  
      className={cn(  
        "z-50 overflow-hidden rounded-md border bg-popover px-3 py-1.5 text-sm text-  
popover-foreground shadow-md animate-in fade-in-0 zoom-in-95 data-  
[state=closed]:animate-out data-[state=closed]:fade-out-0 data-[state=closed]:zoom-out-95  
data-[side=bottom]:slide-in-from-top-2 data-[side=left]:slide-in-from-right-2 data-  
[side=right]:slide-in-from-left-2 data-[side=top]:slide-in-from-bottom-2",  
        className,  
      )}  
      {...props}  
    />  
  ),  
);
```

```
TooltipContent.displayName = TooltipPrimitive.Content.displayName;
```

```
export { Tooltip, TooltipTrigger, TooltipContent, TooltipProvider };
```

File: credit-card-fraud-detection/frontend/src/components/ui/use-toast.js

```
import { useToast, toast } from "@hooks/use-toast";
```

```
export { useToast, toast };
```

File: credit-card-fraud-detection/frontend/src/context/AppContext.jsx

```
import React, { createContext, useContext, useState, useCallback, useEffect } from "react";
```

```
import { authApi, transactionApi, userApi } from "@lib/api";
```

```
const AppContext = createContext(undefined);
```

```
export function AppProvider({ children }) {
```

```
  const [currentUser, setCurrentUser] = useState(() => {
```

```
    const saved = localStorage.getItem("cc_fraud_user");
```

```
    return saved ? JSON.parse(saved) : null;
```

```
  });
```

```
  const [loading, setLoading] = useState(false);
```

```
  useEffect(() => {
```

```
    if (currentUser) {
```

```
      localStorage.setItem("cc_fraud_user", JSON.stringify(currentUser));
```

```
    } else {
```

```
    localStorage.removeItem("cc_fraud_user");
  }
}, [currentUser]);

const loginAdmin = useCallback(async (username, password) => {
  setLoading(true);
  try {
    const resp = await authApi.loginAdmin(username, password);
    if (resp.success) {
      setCurrentUser({ role: "admin", username: "admin", token: resp.token });
    }
  } catch (error) {
    console.error("Admin Login failed:", error);
    throw error;
  } finally {
    setLoading(false);
  }
}, []);
```

```
const loginUser = useCallback(async (email, password) => {
  setLoading(true);
  try {
    const resp = await authApi.loginUser(email, password);
    if (resp) {
      setCurrentUser({ ...resp, role: "user" });
    }
  }
```

```
    } catch (error) {  
      console.error("User Login failed:", error);  
      throw error;  
    } finally {  
      setLoading(false);  
    }  
  }, []);
```

```
const logout = useCallback(() => {  
  setCurrentUser(null);  
}, []);
```

```
return (  
  <AppContext.Provider  
    value={{  
      currentUser,  
      loginAdmin,  
      loginUser,  
      logout,  
      loading,  
    }}  
  >  
    {children}  
  </AppContext.Provider>  
);  
}
```

```
export function useApp() {  
  const ctx = useContext(AppContext);  
  if (!ctx) throw new Error("useApp must be used within AppProvider");  
  return ctx;  
}
```

File: credit-card-fraud-detection/frontend/src/hooks/use-mobile.jsx

```
import * as React from "react";
```

```
const MOBILE_BREAKPOINT = 768;
```

```
export function useIsMobile() {
```

```
  const [isMobile, setIsMobile] = React.useState(undefined);
```

```
  React.useEffect(() => {
```

```
    const mql = window.matchMedia(`(max-width: ${MOBILE_BREAKPOINT - 1}px)`);
```

```
    const onChange = () => {
```

```
      setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
```

```
    };
```

```
    mql.addEventListener("change", onChange);
```

```
    setIsMobile(window.innerWidth < MOBILE_BREAKPOINT);
```

```
    return () => mql.removeEventListener("change", onChange);
```

```
  }, []);
```

```
    return !!isMobile;
}
```

File: credit-card-fraud-detection/frontend/src/hooks/use-toast.js

```
import * as React from "react";
```

```
const TOAST_LIMIT = 1;
```

```
const TOAST_REMOVE_DELAY = 1000000;
```

```
const actionTypes = {
  ADD_TOAST: "ADD_TOAST",
  UPDATE_TOAST: "UPDATE_TOAST",
  DISMISS_TOAST: "DISMISS_TOAST",
  REMOVE_TOAST: "REMOVE_TOAST",
};
```

```
let count = 0;
```

```
function genId() {
  count = (count + 1) % Number.MAX_SAFE_INTEGER;
  return count.toString();
}
```

```
const toastTimeouts = new Map();
```

```
const addToRemoveQueue = (toastId) => {  
  if (toastTimeouts.has(toastId)) {  
    return;  
  }  
}
```

```
const timeout = setTimeout(() => {  
  toastTimeouts.delete(toastId);  
  dispatch({  
    type: "REMOVE_TOAST",  
    toastId: toastId,  
  });  
}, TOAST_REMOVE_DELAY);
```

```
toastTimeouts.set(toastId, timeout);  
};
```

```
export const reducer = (state, action) => {  
  switch (action.type) {  
    case "ADD_TOAST":  
      return {  
        ...state,  
        toasts: [action.toast, ...state.toasts].slice(0, TOAST_LIMIT),  
      };  
  }  
};
```

```
case "UPDATE_TOAST":
```

```
  return {  
    ...state,  
    toasts: state.toasts.map((t) =>  
      t.id === action.toast.id ? { ...t, ...action.toast } : t,  
    ),  
  };  
}
```

```
case "DISMISS_TOAST": {
```

```
  const { toastId } = action;
```

```
  // ! Side effects ! - This could be extracted into a dismissToast() action,
```

```
  // but I'll keep it here for simplicity
```

```
  if (toastId) {  
    addToRemoveQueue(toastId);  
  } else {  
    state.toasts.forEach((toast) => {  
      addToRemoveQueue(toast.id);  
    });  
  }  
}
```

```
  return {  
    ...state,  
    toasts: state.toasts.map((t) =>  
      t.id === toastId || toastId === undefined  
      ? {  
        ...t,  
        dismissed: true,  
        timeout: 0,  
      } : t,  
    ),  
  };  
}
```



```

        ...t,

        open: false,

    }

    : t,

),

};

}

case "REMOVE_TOAST":

    if (action.toastId === undefined) {

        return {

            ...state,

            toasts: [],

        };

    }

    return {

        ...state,

        toasts: state.toasts.filter((t) => t.id !== action.toastId),

    };

}

};

```

```

const listeners = [];

```

```

let memoryState = { toasts: [] };

```

```

function dispatch(action) {

```

```
memoryState = reducer(memoryState, action);

listeners.forEach((listener) => {

  listener(memoryState);

});

}

function toast({ ...props }) {

  const id = genId();

  const update = (props) =>

    dispatch({

      type: "UPDATE_TOAST",

      toast: { ...props, id },

    });

  const dismiss = () => dispatch({ type: "DISMISS_TOAST", toastId: id });

  dispatch({

    type: "ADD_TOAST",

    toast: {

      ...props,

      id,

      open: true,

      onOpenChange: (open) => {

        if (!open) dismiss();

      },

    },

  },
```

```
});
```

```
return {  
  id: id,  
  dismiss,  
  update,  
};  
}
```

```
function useToast() {  
  const [state, setState] = React.useState(memoryState);  
  
  React.useEffect(() => {  
    listeners.push(setState);  
    return () => {  
      const index = listeners.indexOf(setState);  
      if (index > -1) {  
        listeners.splice(index, 1);  
      }  
    };  
  }, [state]);  
  
  return {  
    ...state,  
    toast,  
    dismiss: (toastId) => dispatch({ type: "DISMISS_TOAST", toastId }),  
  };  
}
```

```
};
```

```
}
```

```
export { useToast, toast };
```

File: credit-card-fraud-detection/frontend/src/index.css

```
@import url('https://fonts.googleapis.com/css2?
family=Montserrat:wght@600;700;800&family=Inter:wght@400;500;600;700&display=sw
ap');
```

```
@tailwind base;
```

```
@tailwind components;
```

```
@tailwind utilities;
```

```
@layer base {
```

```
:root {
```

```
--background: 0 0% 100%;
```

```
--foreground: 0 0% 15%;
```

```
--card: 0 0% 100%;
```

```
--card-foreground: 0 0% 15%;
```

```
--popover: 0 0% 100%;
```

```
--popover-foreground: 0 0% 15%;
```

--primary: 45 93% 47%;

--primary-foreground: 0 0% 100%;

--secondary: 45 93% 92%;

--secondary-foreground: 0 0% 15%;

--muted: 0 0% 92%;

--muted-foreground: 0 0% 50%;

--accent: 45 93% 47%;

--accent-foreground: 0 0% 100%;

--destructive: 0 84.2% 60.2%;

--destructive-foreground: 0 0% 100%;

--border: 0 0% 92%;

--input: 0 0% 92%;

--ring: 45 93% 47%;

--radius: 0.75rem;

--warning: 38 92% 50%;

--warning-foreground: 0 0% 0%;

--success: 142 71% 45%;

--success-foreground: 0 0% 100%;

--sidebar-background: 0 0% 98%;

--sidebar-foreground: 0 0% 15%;

--sidebar-primary: 45 93% 47%;

--sidebar-primary-foreground: 0 0% 100%;

--sidebar-accent: 45 93% 47%;

--sidebar-accent-foreground: 0 0% 100%;

--sidebar-border: 0 0% 92%;

--sidebar-ring: 45 93% 47%;

--glass-bg: rgba(255, 255, 255, 0.8);

--glass-border: rgba(192, 132, 8, 0.1);

}

}

@layer base {

* {

 @apply border-border;

}

body {

 @apply bg-background text-foreground font-body antialiased;

}

h1,

```
h2,  
h3,  
h4,  
h5,  
h6 {  
    font-family: 'Montserrat', sans-serif;  
    font-weight: 700;  
}  
}
```

```
@layer components {  
    .glass-card {  
        background: var(--glass-bg);  
        backdrop-filter: blur(16px);  
        -webkit-backdrop-filter: blur(16px);  
        border: 1px solid var(--glass-border);  
    }  
  
    .pulse-danger {  
        animation: pulse-danger 2s cubic-bezier(0.4, 0, 0.6, 1) infinite;  
    }  
  
    .pulse-live {  
        animation: pulse-live 1.5s cubic-bezier(0.4, 0, 0.6, 1) infinite;  
    }  
}
```

```
@keyframes pulse-danger {
```

```
  0%,
```

```
  100% {
```

```
    opacity: 1;
```

```
  }
```

```
  50% {
```

```
    opacity: 0.6;
```

```
  }
```

```
}
```

```
@keyframes pulse-live {
```

```
  0%,
```

```
  100% {
```

```
    box-shadow: 0 0 0 0 rgba(239, 68, 68, 0.7);
```

```
  }
```

```
  50% {
```

```
    box-shadow: 0 0 0 6px rgba(239, 68, 68, 0);
```

```
  }
```

```
}
```


File: credit-card-fraud-detection/frontend/src/lib/api.js

```
import axios from "axios";
```

```
const API_BASE_URL = "https://detect-api.projectbro.in";
```

```
const api = axios.create({  
  baseURL: API_BASE_URL,  
  headers: {  
    "Content-Type": "application/json",  
  },  
});
```

```
export const authApi = {  
  register: async (userData) => {  
    const response = await api.post("/users/register", userData);  
    return response.data;  
  },  
  loginUser: async (email, password) => {  
    const response = await api.post("/users/login", { email, password });  
    return response.data;  
  },  
  loginAdmin: async (username, password) => {  
    const response = await api.post("/admin/login", { username, password });  
    return response.data;  
  },  
};
```

```
};
```

```
export const userApi = {  
  getById: async (userId) => {  
    const response = await api.get(`/users/${userId}`);  
    return response.data;  
  },  
};
```

```
export const transactionApi = {  
  getUserId: async (userId, limit = 100, offset = 0) => {  
    const response = await api.get(`/transactions/${userId}?  
limit=${limit}&offset=${offset}`);  
    return response.data;  
  },  
  simulate: async (simulationData) => {  
    const response = await api.post("/transactions/simulate", simulationData);  
    return response.data;  
  },  
  report: async (transactionId) => {  
    const response = await api.post("/transactions/report", { transaction_id: transactionId  
});  
    return response.data;  
  },  
};
```

```

export const alertsApi = {
  getAlerts: async (userId) => {
    const response = await api.get(`/alerts/${userId}`);
    return response.data;
  },
};

export const adminApi = {
  getUsers: async () => {
    const response = await api.get("/admin/users");
    return response.data;
  },
  getTransactions: async (limit = 100, offset = 0) => {
    const response = await api.get(`/admin/transactions?
limit=${limit}&offset=${offset}`);
    return response.data;
  },
  triggerHighRisk: async (userId, amountMultiplier) => {
    const response = await api.post("/admin/trigger-high-risk", { user_id: userId,
amount_multiplier: amountMultiplier });
    return response.data;
  },
  trainModel: async () => {
    const response = await api.post("/admin/train");
    return response.data;
  },
};

```

```
getSummary: async () => {  
  const response = await api.get("/admin/analytics/summary");  
  return response.data;  
},  
};  
  
export default api;
```

File: credit-card-fraud-detection/frontend/src/lib/utils.js

```
import { clsx } from "clsx";  
  
import { twMerge } from "tailwind-merge";  
  
export function cn(...inputs) {  
  return twMerge(clsx(inputs));  
}
```

File: credit-card-fraud-detection/frontend/src/main.jsx

```
import { createRoot } from "react-dom/client";  
  
import App from "./App";  
  
import "./index.css";  
  
createRoot(document.getElementById("root")).render(<App />);
```

File: credit-card-fraud-detection/frontend/src/pages/Alerts.jsx

```
import { useEffect, useState } from "react";
import { useNavigate } from "react-router-dom";
import { useApp } from "@context/AppContext";
import { alertsApi } from "@lib/api";
import { Badge } from "@components/ui/badge";
import { Button } from "@components/ui/button";
import {
  Table,
  TableHeader,
  TableRow,
  TableHead,
  TableBody,
  TableCell,
} from "@components/ui/table";
import { Shield, ArrowLeft, AlertTriangle, ShieldAlert, Bell } from "lucide-react";
import { toast } from "sonner";

const riskBadge = (level) => {
  level = level.toLowerCase();
  if (level === "fraud" || level === "flagged")
    return <Badge className="bg-destructive/20 text-destructive border-destructive/30">Fraud</Badge>;
```

```
if (level === "suspicious")

  return <Badge className="bg-warning/20 text-warning border-
warning/30">Suspicious</Badge>;

  return <Badge className="bg-success/20 text-success border-
success/30">Safe</Badge>;

};
```

```
const Alerts = () => {

  const { currentUser } = useApp();

  const navigate = useNavigate();

  const [alerts, setAlerts] = useState([]);

  const [loading, setLoading] = useState(true);

  const [stats, setStats] = useState({ fraud: 0, suspicious: 0 });

  useEffect(() => {

    if (!currentUser) navigate("/");

  });

  const fetchAlerts = async () => {

    if (!currentUser?.user_id) return;

    try {

      setLoading(true);

      const res = await alertsApi.getAlerts(currentUser.user_id);

      setAlerts(res.alerts || []);

      setStats({

        fraud: res.total_fraud_count || 0,

        suspicious: res.total_suspicious_count || 0,
```

```

    });

    } catch (err) {

        console.error("Error fetching alerts:", err);

        toast.error("Failed to load alerts");

    } finally {

        setLoading(false);

    }

};

fetchAlerts();

}, [currentUser, navigate]);

if (!currentUser) return null;

return (

    <div className="min-h-screen bg-background">

        {/* Header */}

        <header className="border-b border-border bg-card flex flex-col md:flex-row
md:items-center gap-2 md:gap-4 px-3 md:px-6 py-3 md:py-4">

            <Button

                variant="ghost"

                size="sm"

                onClick={() => navigate("/dashboard")}

                className="text-muted-foreground text-xs md:text-sm w-fit"

            >

                <ArrowLeft className="w-4 h-4" /> Back

```

```
</Button>

<div className="flex items-center gap-2 md:gap-3">

  <div className="w-8 h-8 rounded-lg bg-destructive/20 flex items-center justify-
center">

    <Bell className="w-4 h-4 text-destructive" />

  </div>

  <span className="font-heading font-bold text-base md:text-lg">Security
Alerts</span>

</div>

</header>

<main className="p-3 md:p-6 max-w-6xl mx-auto space-y-4 md:space-y-6">

  {/* Stats */}

  <div className="grid grid-cols-1 md:grid-cols-2 gap-3 md:gap-6">

    <div className="glass-card rounded-xl p-4 md:p-6 flex flex-col justify-center
border-l-4 border-l-destructive">

      <h3 className="text-xs md:text-sm font-medium text-destructive">🚨 Fraud
Alerts</h3>

      <p className="text-2xl md:text-3xl font-heading font-bold mt-2 text-destructive">
{stats.fraud}</p>

      <p className="text-xs text-muted-foreground mt-1">Transactions flagged as
fraud</p>

    </div>

    <div className="glass-card rounded-xl p-4 md:p-6 flex flex-col justify-center
border-l-4 border-l-warning">

      <h3 className="text-xs md:text-sm font-medium text-warning">⚠ Suspicious
Alerts</h3>

      <p className="text-2xl md:text-3xl font-heading font-bold mt-2 text-warning">
{stats.suspicious}</p>
```



```
<p className="text-xs text-muted-foreground mt-1">Transactions with unusual activity</p>
```

```
</div>
```

```
</div>
```

```
{/* Alerts Table */}
```

```
<div className="glass-card rounded-xl overflow-hidden">
```

```
<div className="p-3 md:p-4 border-b border-border bg-card/50">
```

```
<h2 className="font-heading font-semibold text-base md:text-lg flex items-center gap-2">
```

```
<AlertTriangle className="w-4 h-4 md:w-5 md:h-5 text-destructive" />
```

```
Security Alerts ({alerts.length})
```

```
</h2>
```

```
<p className="text-xs md:text-sm text-muted-foreground mt-1">
```

```
{alerts.length === 0
```

```
? "No fraud or suspicious alerts detected. Your account is secure!"
```

```
: "Review these potentially fraudulent or suspicious transactions"}
```

```
</p>
```

```
</div>
```

```
{loading ? (
```

```
<div className="p-6 md:p-8 text-center text-muted-foreground text-sm">Loading alerts...</div>
```

```
) : alerts.length === 0 ? (
```

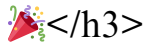
```
<div className="p-8 md:p-12 text-center">
```

```
<div className="flex justify-center mb-4">
```

```
<Shield className="w-10 h-10 md:w-12 md:h-12 text-success opacity-50" />
```

</div>

<h3 className="text-base md:text-lg font-semibold text-foreground">All Clear!



<p className="text-xs md:text-sm text-muted-foreground mt-2">No fraudulent or suspicious transactions detected.</p>

</div>

) : (

<div className="overflow-x-auto">

<Table className="text-xs md:text-sm">

<TableHeader>

<TableRow className="border-border">

<TableHead className="text-xs">Date</TableHead>

<TableHead className="text-xs">Amount</TableHead>

<TableHead className="text-xs hidden md:table-cell">Location</TableHead>

<TableHead className="text-xs hidden lg:table-cell">Device</TableHead>

<TableHead className="text-xs hidden xl:table-cell">Mode</TableHead>

<TableHead className="text-xs">Type</TableHead>

<TableHead className="text-xs hidden sm:table-cell">Risk</TableHead>

<TableHead className="text-xs">Status</TableHead>

</TableRow>

</TableHeader>

<TableBody>

{alerts.map((alert) => (

<TableRow key={alert.trans_id} className="border-border hover:bg-secondary/50">

<TableCell className="text-xs text-muted-foreground">

```

        {new Date(alert.timestamp).toLocaleDateString()}
    </TableCell>

    <TableCell className="font-semibold text-xs">₹ {alert.amount.toFixed(0)}
</TableCell>

    <TableCell className="hidden md:table-cell text-xs">{alert.location}
</TableCell>

    <TableCell className="hidden lg:table-cell text-xs">{alert.device}
</TableCell>

    <TableCell className="hidden xl:table-cell text-xs">{alert.mode}
</TableCell>

    <TableCell className="text-xs min-w-[80px]">{riskBadge(alert.risk_label)}
</TableCell>

    <TableCell className="hidden sm:table-cell">

        <div className="flex items-center gap-1">

            <div className="w-10 bg-secondary rounded-full h-1.5">

                <div

                    className={`h-full rounded-full ${alert.risk_score > 0.7
                        ? "bg-destructive"
                        : alert.risk_score > 0.4
                        ? "bg-warning"
                        : "bg-success"
                    }`}

                    style={{ width: `${(alert.risk_score || 0) * 100}%` }}

                />

            </div>

            <span className="text-xs font-medium w-8">{((alert.risk_score || 0) *
100).toFixed(0)}%</span>

        </div>

```

```
</TableCell>
```

```
<TableCell>
```

```
{alert.reported ? (
```

```
    <Badge className="bg-primary/20 text-primary border-primary/30 text-  
xs">Reported</Badge>
```

```
) : (
```

```
    <Badge variant="outline" className="text-muted-foreground text-  
xs">Pending</Badge>
```

```
))
```

```
</TableCell>
```

```
</TableRow>
```

```
)))
```

```
</TableBody>
```

```
</Table>
```

```
</div>
```

```
)}
```

```
</div>
```

```
{/* Info Box */}
```

```
<div className="glass-card rounded-xl p-3 md:p-4 bg-primary/5 border border-  
primary/20">
```

```
<p className="text-xs md:text-sm text-foreground">
```

```
<strong>💡 Tip:</strong> These alerts represent transactions that our fraud  
detection system has identified as potentially fraudulent or suspicious based on your  
transaction patterns. If you recognize any of these as legitimate, you can report them in  
your transaction history.
```

```
</p>
```

```
</div>
```

```
    </main>

  </div>

);

};

export default Alerts;
```

File: credit-card-fraud-detection/frontend/src/pages/Dashboard.jsx

```
import { useEffect } from "react";
import { useNavigate } from "react-router-dom";
import { useApp } from "@context/AppContext";
import UserDashboard from "@components/dashboard/UserDashboard";
import AdminDashboard from "@components/dashboard/AdminDashboard";

const Dashboard = () => {
  const { currentUser } = useApp();
  const navigate = useNavigate();

  useEffect(() => {
    if (!currentUser) navigate("/");
  }, [currentUser, navigate]);

  if (!currentUser) return null;
```

```
    return currentUser.role === "admin" ? <AdminDashboard /> : <UserDashboard />;  
};  
  
export default Dashboard;
```

File: credit-card-fraud-detection/frontend/src/pages/Login.jsx

```
import { useState } from "react";  
import { useNavigate } from "react-router-dom";  
import { Shield, Eye, EyeOff, ArrowRight, AlertCircle } from "lucide-react";  
import { Button } from "@components/ui/button";  
import { Input } from "@components/ui/input";  
import { Label } from "@components/ui/label";  
import { useApp } from "@context/AppContext";  
import { toast } from "sonner";  
  
const Login = () => {  
    const [role, setRole] = useState("user");  
    const [email, setEmail] = useState("");  
    const [username, setUsername] = useState("");  
    const [password, setPassword] = useState("");  
    const [errors, setErrors] = useState({});  
    const [showPass, setShowPass] = useState(false);  
    const { loginAdmin, loginUser, loading } = useApp();  
    const navigate = useNavigate();
```

```
const validateForm = () => {  
  const newErrors = {};  
  
  if (role === "admin") {  
    if (!username.trim()) newErrors.username = "Username is required";  
    if (!password.trim()) newErrors.password = "Password is required";  
  } else {  
    if (!email.trim()) newErrors.email = "Email is required";  
    else if (!/^[^\\s@]+@[^\\s@]+\\.^[^\\s@]+$/.test(email)) newErrors.email = "Please enter a  
valid email";  
    if (!password.trim()) newErrors.password = "Password is required";  
  }  
  
  setErrors(newErrors);  
  return Object.keys(newErrors).length === 0;  
};  
  
const handleSubmit = async (e) => {  
  e.preventDefault();  
  
  if (!validateForm()) {  
    toast.error("Please fill in all required fields");  
    return;  
  }  
}
```

```

try {
  if (role === "admin") {
    await loginAdmin(username, password);
    toast.success("Admin login successful!");
  } else {
    await loginUser(email, password);
    toast.success("Welcome back! Redirecting to dashboard...");
  }
  navigate("/dashboard");
} catch (error) {
  const errorMsg = error.response?.data?.detail || error.message || "Login failed. Please check your credentials.";
  toast.error(errorMsg);
}
};

return (
  <div className="min-h-screen flex items-center justify-center bg-background relative overflow-hidden p-4">
    <div className="absolute top-1/4 left-1/2 -translate-x-1/2 w-[600px] h-[600px] bg-primary/10 rounded-full blur-[120px] pointer-events-none" />
    <div className="w-full max-w-md relative z-10">
      <div className="flex flex-col items-center mb-8 md:mb-10">
        <div className="w-12 h-12 md:w-16 md:h-16 rounded-2xl bg-primary/20 flex items-center justify-center mb-4 border border-primary/30">
          <Shield className="w-6 h-6 md:w-8 md:h-8 text-primary" />

```



```
</div>
```

```
<h1 className="font-heading text-2xl md:text-3xl font-bold tracking-tight text-center">
```

```
Credit Card Fraud Detection
```

```
</h1>
```

```
</div>
```

```
<form onSubmit={handleSubmit} className="glass-card rounded-xl p-4 md:p-6 space-y-4 md:space-y-5">
```

```
<div className="flex rounded-lg bg-secondary p-1">
```

```
{["user", "admin"].map((r) => (
```

```
<button
```

```
key={r}
```

```
type="button"
```

```
onClick={() => setRole(r)}
```

```
className={`flex-1 py-2 text-xs md:text-sm font-medium rounded-md transition-all ${role === r
```

```
? "bg-primary text-primary-foreground shadow"
```

```
: "text-muted-foreground hover:text-foreground"
```

```
}``}
```

```
>
```

```
{r === "user" ? "Card Holder" : "Administrator"}
```

```
</button>
```

```
))}
```

```
</div>
```

```
{role === "admin" ? (
```

<>

<div className="space-y-2">

<Label htmlFor="username">Username</Label>

<Input

id="username"

type="text"

value={username}

onChange={(e) => {

setUsername(e.target.value);

if (errors.username) setErrors((p) => ({ ...p, username: "" }));

}}

placeholder="admin"

className={`bg-secondary border-border \${errors.username ? "border-destructive" : ""}`}

/>

{errors.username && <p className="text-sm text-destructive flex items-center gap-1"><AlertCircle className="w-3 h-3" />{errors.username}</p>}

</div>

<div className="space-y-2">

<Label htmlFor="password">Password</Label>

<div className="relative">

<Input

id="password"

type={showPass ? "text" : "password"}

value={password}

```

      onChange={(e) => {
        setPassword(e.target.value);
        if (errors.password) setErrors((p) => ({ ...p, password: "" }));
      }}
      placeholder="Enter password"
      className={`bg-secondary border-border pr-10 ${errors.password ? "border-destructive" : ""}`}
    />
    <button
      type="button"
      onClick={() => setShowPass(!showPass)}
      className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground hover:text-foreground"
    >
      {showPass ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4" />}
    </button>
  </div>

  {errors.password && <p className="text-sm text-destructive flex items-center gap-1"><AlertCircle className="w-3 h-3" />{errors.password}</p>}

</div>
</>
):(
<>
<div className="space-y-2">
  <Label htmlFor="email">Email Address</Label>
  <Input

```

```

      id="email"

      type="email"

      value={email}

      onChange={(e) => {

        setEmail(e.target.value);

        if (errors.email) setErrors((p) => ({ ...p, email: "" }));

      }}

      placeholder="john@example.com"

      className={`bg-secondary border-border ${errors.email ? "border-destructive"
: ""}`}

    />

    {errors.email && <p className="text-sm text-destructive flex items-center gap-
1"><AlertCircle className="w-3 h-3" />{errors.email}</p>}

  </div>

```

```

<div className="space-y-2">

  <Label htmlFor="password">Password</Label>

  <div className="relative">

    <Input

      id="password"

      type={showPass ? "text" : "password"}

      value={password}

      onChange={(e) => {

        setPassword(e.target.value);

        if (errors.password) setErrors((p) => ({ ...p, password: "" }));

      }}

    >

```

```

placeholder="Enter password"

className={`bg-secondary border-border pr-10 ${errors.password ? "border-
destructive" : ""}`}
/>

<button

type="button"

onClick={() => setShowPass(!showPass)}

className="absolute right-3 top-1/2 -translate-y-1/2 text-muted-foreground
hover:text-foreground"

>

{showPass ? <EyeOff className="w-4 h-4" /> : <Eye className="w-4 h-4"
/>}

</button>

</div>

{errors.password && <p className="text-sm text-destructive flex items-center
gap-1"><AlertCircle className="w-3 h-3" /> {errors.password} </p>}

</div>

</>

)}

<Button type="submit" disabled={loading} className="w-full gap-2 text-sm
md:text-base py-2 md:py-2.5">

{loading ? "Signing In..." : "Sign In"} <ArrowRight className="w-4 h-4" />

</Button>

<div className="text-center text-xs md:text-sm text-muted-foreground mt-4">

<p>

New user?{" "}

```

```
    <a
      href="/register"
      onClick={(e) => {
        e.preventDefault();
        navigate("/register");
      }}
      className="text-primary hover:underline font-medium"
    >
      Register here
    </a>
  </p>
</div>
</form>
</div>
</div>
);
};
```

```
export default Login;
```

File: credit-card-fraud-detection/frontend/src/pages/NotFound.jsx

```
import { useLocation } from "react-router-dom";
import { useEffect } from "react";
```

```
const NotFound = () => {

  const location = useLocation();

  useEffect(() => {

    console.error(

      "404 Error: User attempted to access non-existent route:",

      location.pathname,

    );

  }, [location.pathname]);

  return (

    <div className="flex min-h-screen items-center justify-center bg-muted">

      <div className="text-center">

        <h1 className="mb-4 text-4xl font-bold">404</h1>

        <p className="mb-4 text-xl text-muted-foreground">

          Oops! Page not found

        </p>

        <a href="/" className="text-primary underline hover:text-primary/90">

          Return to Home

        </a>

      </div>

    </div>

  );

};

export default NotFound;
```

File: credit-card-fraud-detection/frontend/src/pages/Portfolio.jsx

```
import { useNavigate } from "react-router-dom";

import { Shield, Zap, Brain, AlertTriangle, TrendingUp, Lock, ArrowRight } from "lucide-react";

import { Button } from "@components/ui/button";

const Portfolio = () => {

  const navigate = useNavigate();

  const features = [

    {

      icon: <Brain className="w-8 h-8 md:w-10 md:h-10" />,

      title: "AI-Powered Detection",

      description: "Machine Learning model trained on thousands of transactions to identify fraud patterns with high accuracy."

    },

    {

      icon: <AlertTriangle className="w-8 h-8 md:w-10 md:h-10" />,

      title: "Real-time Alerts",

      description: "Instant fraud detection and alerts for suspicious transactions, keeping your account secure 24/7."

    },

    {

      icon: <Lock className="w-8 h-8 md:w-10 md:h-10" />,
```



```
    title: "Secure & Private",

    description: "Bank-grade security with encrypted data storage and secure
authentication protocols."

  },

  {

    icon: <TrendingUp className="w-8 h-8 md:w-10 md:h-10" />,

    title: "Advanced Analytics",

    description: "Comprehensive dashboard with transaction analytics and risk scoring for
better insights."

  },

  {

    icon: <Zap className="w-8 h-8 md:w-10 md:h-10" />,

    title: "Fast & Reliable",

    description: "Lightning-fast fraud detection with 99.9% uptime and real-time
processing."

  },

  {

    icon: <Shield className="w-8 h-8 md:w-10 md:h-10" />,

    title: "Smart Prevention",

    description: "Adaptive fraud detection that learns from user feedback and improves
over time."

  }

];

const technologies = [

  { name: "React", icon: "⚛️" },

  { name: "FastAPI", icon: "⚡" },
```

```
{ name: "Python", icon: "🐍" },
{ name: "Machine Learning", icon: "🤖" },
{ name: "SQLite", icon: "💾" },
{ name: "Tailwind CSS", icon: "🎨" }
];
```

```
const stats = [
  { label: "Fraud Detection Accuracy", value: "96%+" },
  { label: "Real-time Processing", value: "<100ms" },
  { label: "User Transactions", value: "10,000+" },
  { label: "System Uptime", value: "99.9%" }
];
```

```
return (
  <div className="min-h-screen bg-background overflow-hidden">
    { /* Navigation */ }

    <nav className="fixed top-0 w-full bg-background/80 backdrop-blur-md border-b
border-border z-50 px-4 md:px-6 py-4">
      <div className="max-w-7xl mx-auto flex items-center justify-between">
        <div className="flex items-center gap-2 md:gap-3">
          <div className="w-8 h-8 md:w-10 md:h-10 rounded-lg bg-primary/20 flex items-
center justify-center">
            <Shield className="w-4 h-4 md:w-5 md:h-5 text-primary" />
          </div>
          <span className="font-heading font-bold text-base md:text-xl">DetectAI</span>
        </div>
      </div>
    </div>
  </div>
);
```

<Button

onClick={() => navigate("/login")}

className="text-xs md:text-sm gap-2"

>

Login <ArrowRight className="w-3 h-3 md:w-4 md:h-4" />

</Button>

</div>

</nav>

{/* Hero Section */}

<section className="min-h-screen flex items-center justify-center px-4 md:px-6 pt-20 md:pt-0 relative overflow-hidden">

<div className="absolute top-20 left-10 w-72 h-72 bg-primary/10 rounded-full blur-3xl opacity-50 -z-10" />

<div className="absolute bottom-10 right-10 w-96 h-96 bg-primary/5 rounded-full blur-3xl opacity-50 -z-10" />

<div className="max-w-6xl mx-auto text-center space-y-6 md:space-y-8">

<div className="inline-block">

<div className="px-4 py-2 rounded-full bg-primary/10 border border-primary/20">

<p className="text-xs md:text-sm font-medium text-primary">🚀 Advanced AI-Powered Solution</p>

</div>

</div>

<h1 className="font-heading text-3xl md:text-5xl lg:text-6xl font-bold leading-tight">

Credit Card

Fraud Detection

</h1>

>

Advanced machine learning system that detects fraudulent credit card transactions in real-time. Protect your finances with intelligent threat detection and instant alerts.

</p>

<div className="flex flex-col sm:flex-row gap-3 md:gap-4 justify-center pt-4 md:pt-8">

<Button

size="lg"

onClick={() => navigate("/login")}

className="text-sm md:text-base gap-2"

>

Get Started <ArrowRight className="w-4 h-4 md:w-5 md:h-5" />

</Button>

<Button

size="lg"

variant="outline"

onClick={() => navigate("/register")}

className="text-sm md:text-base"

>

Sign Up <ArrowRight className="w-4 h-4 md:w-5 md:h-5" />

</Button>

</div>

{/* Stats */}

<div className="grid grid-cols-2 md:grid-cols-4 gap-4 md:gap-8 pt-12 md:pt-16">

{stats.map((stat, idx) => (

<div key={idx} className="text-center">

<p className="text-2xl md:text-3xl font-heading font-bold text-primary">

{stat.value}</p>

<p className="text-xs md:text-sm text-muted-foreground mt-1 md:mt-2">

{stat.label}</p>

</div>

))}

</div>

</div>

</section>

{/* Features Section */}

<section className="py-16 md:py-24 px-4 md:px-6 bg-card/30 border-y border-border">

<div className="max-w-6xl mx-auto space-y-12 md:space-y-16">

<div className="text-center space-y-4">

<h2 className="text-2xl md:text-4xl font-heading font-bold">Powerful
Features</h2>

<p className="text-sm md:text-base text-muted-foreground max-w-2xl mx-auto">

Everything you need to detect and prevent fraud with confidence.

</p>

</div>

<div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-6 md:gap-8">

{features.map((feature, idx) => (

<div key={idx} className="glass-card rounded-xl p-6 md:p-8 hover:shadow-lg transition-all group cursor-pointer">

<div className="w-12 h-12 md:w-16 md:h-16 rounded-lg bg-primary/10 flex items-center justify-center mb-4 md:mb-6 group-hover:bg-primary/20 transition-colors text-primary">

{feature.icon}

</div>

<h3 className="text-lg md:text-xl font-heading font-bold mb-2 md:mb-3">
{feature.title}</h3>

<p className="text-sm md:text-base text-muted-foreground leading-relaxed">
{feature.description}</p>

</div>

)))

</div>

</div>

</section>

{/* How It Works Section */}

<section className="py-16 md:py-24 px-4 md:px-6">

<div className="max-w-6xl mx-auto space-y-12 md:space-y-16">

<div className="text-center space-y-4">

<h2 className="text-2xl md:text-4xl font-heading font-bold">How It Works</h2>

<p className="text-sm md:text-base text-muted-foreground max-w-2xl mx-auto">

Three simple steps to secure your credit card transactions.

</p>

</div>

<div className="grid grid-cols-1 md:grid-cols-3 gap-6 md:gap-8">

{[

{

num: "01",

title: "Input Transaction",

desc: "Register your account and simulate or perform transactions to test the system."

},

{

num: "02",

title: "AI Analysis",

desc: "Our ML model analyzes your transaction patterns and detects anomalies instantly."

},

{

num: "03",

title: "Instant Alert",

desc: "Receive real-time alerts and view detailed fraud risk analysis on your dashboard."

}

].map((step, idx) => (

<div key={idx} className="relative">

<div className="glass-card rounded-xl p-6 md:p-8 h-full hover:shadow-lg transition-all">

```
<div className="text-4xl md:text-5xl font-heading font-bold text-primary/20
mb-4">{step.num}</div>

<h3 className="text-lg md:text-xl font-heading font-bold mb-3">{step.title}
</h3>

<p className="text-sm md:text-base text-muted-foreground">{step.desc}</p>
</div>

{idx < 2 && (

  <div className="hidden md:block absolute top-1/2 -right-4 transform -
translate-y-1/2">

    <ArrowRight className="w-8 h-8 text-primary/30" />

  </div>

  )}

</div>

)}}

</div>

</div>

</section>

{/* Technology Stack */}

<section className="py-16 md:py-24 px-4 md:px-6 bg-card/30 border-y border-
border">

  <div className="max-w-6xl mx-auto space-y-12 md:space-y-16">

    <div className="text-center space-y-4">

      <h2 className="text-2xl md:text-4xl font-heading font-bold">Built With Modern
Technology</h2>

      <p className="text-sm md:text-base text-muted-foreground max-w-2xl mx-auto">

        Cutting-edge technologies for robust, scalable, and secure fraud detection.

      </p>
```


</div>

```
<div className="grid grid-cols-2 md:grid-cols-3 lg:grid-cols-6 gap-4 md:gap-6">
  {technologies.map((tech, idx) => (
    <div
      key={idx}
      className="glass-card rounded-xl p-6 md:p-8 flex flex-col items-center justify-
center gap-3 hover:shadow-lg transition-all group cursor-pointer"
    >
      <span className="text-3xl md:text-4xl group-hover:scale-110 transition-
transform">{tech.icon}</span>
      <p className="text-sm md:text-base font-medium text-center">{tech.name}</p>
    </div>
  ))}
</div>
</section>
```

```
{/* Key Metrics */}
<section className="py-16 md:py-24 px-4 md:px-6">
  <div className="max-w-6xl mx-auto">
    <div className="glass-card rounded-2xl p-8 md:p-12">
      <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-4 gap-8 md:gap-12">
        <div>
          <p className="text-3xl md:text-4xl font-heading font-bold text-primary">96%+
        </p>
```

<p className="text-sm md:text-base text-muted-foreground mt-2">Detection Accuracy</p>

</div>

<div>

<p className="text-3xl md:text-4xl font-heading font-bold text-primary"><100ms</p>

<p className="text-sm md:text-base text-muted-foreground mt-2">Response Time</p>

</div>

<div>

<p className="text-3xl md:text-4xl font-heading font-bold text-primary">24/7</p>

<p className="text-sm md:text-base text-muted-foreground mt-2">Continuous Monitoring</p>

</div>

<div>

<p className="text-3xl md:text-4xl font-heading font-bold text-primary">99.9%</p>

<p className="text-sm md:text-base text-muted-foreground mt-2">System Uptime</p>

</div>

</div>

</div>

</div>

</section>

{/* CTA Section */}

<section className="py-16 md:py-24 px-4 md:px-6 bg-gradient-to-r from-primary/10 via-transparent to-primary/5 border-y border-border">

```
<div className="max-w-4xl mx-auto text-center space-y-6 md:space-y-8">

  <h2 className="text-2xl md:text-4xl font-heading font-bold">

    Ready to Secure Your Transactions?

  </h2>

  <p className="text-sm md:text-base text-muted-foreground max-w-2xl mx-auto">

    Join thousands of users protecting their finances with our advanced fraud detection
    system.

  </p>

  <div className="flex flex-col sm:flex-row gap-3 md:gap-4 justify-center pt-4 md:pt-
8">

    <Button

      size="lg"

      onClick={() => navigate("/login")}

      className="text-sm md:text-base gap-2"

    >

      Get Started Now <ArrowRight className="w-4 h-4 md:w-5 md:h-5" />

    </Button>

    <Button

      size="lg"

      variant="outline"

      onClick={() => navigate("/register")}

      className="text-sm md:text-base gap-2"

    >

      Sign Up <ArrowRight className="w-4 h-4 md:w-5 md:h-5" />

    </Button>

  </div>
```

```
</div>
```

```
</section>
```

```
{/* Footer */}
```

```
<footer className="border-t border-border bg-card/50 py-8 md:py-12 px-4 md:px-6">
```

```
<div className="max-w-6xl mx-auto text-center">
```

```
<div className="flex items-center justify-center gap-2 mb-4">
```

```
<Shield className="w-5 h-5 text-primary" />
```

```
<span className="font-heading font-bold text-base md:text-lg">DetectAI</span>
```

```
</div>
```

```
<p className="text-xs md:text-sm text-muted-foreground">
```

© 2026 Credit Card Fraud Detection. Advanced AI-powered fraud detection system protecting your financial security.

```
</p>
```

```
</div>
```

```
</footer>
```

```
</div>
```

```
);
```

```
};
```

```
export default Portfolio;
```

File: credit-card-fraud-detection/frontend/src/pages/Register.jsx

```
import { useState } from "react";
```

```
import { useNavigate } from "react-router-dom";

import { Shield, ArrowRight, ArrowLeft, Check, AlertCircle } from "lucide-react";

import { Button } from "@components/ui/button";

import { Input } from "@components/ui/input";

import { Label } from "@components/ui/label";

import { useApp } from "@context/AppContext";

import { authApi } from "@lib/api";

import { toast } from "sonner";
```

```
const Register = () => {

  const [form, setForm] = useState({

    name: "",

    email: "",

    mobile: "",

    cardNumber: "",

    password: "",

    monthlyIncome: "",

    dailyLimit: "",

  });

  const [errors, setErrors] = useState({});

  const [showPass, setShowPass] = useState(false);

  const [loading, setLoading] = useState(false);

  const { loginUser } = useApp();

  const navigate = useNavigate();

  const set = (key, val) => {
```

```
setForm((p) => ({ ...p, [key]: val }));

// Clear error for this field when user starts typing

if (errors[key]) {

  setErrors((p) => ({ ...p, [key]: "" }));

}

};

const validateForm = () => {

  const newErrors = {};

  if (!form.name.trim()) newErrors.name = "Full name is required";

  if (!form.email.trim()) newErrors.email = "Email address is required";

  else if (!/^[^s@]+@[^s@]+\.[^s@]+$/.test(form.email)) newErrors.email = "Please enter
a valid email address";

  if (!form.mobile.trim()) newErrors.mobile = "Mobile number is required";

  else if (!/^\d{10}$/.test(form.mobile.replace(/D/g, ""))) newErrors.mobile = "Mobile
number must be 10 digits";

  if (!form.cardNumber.trim()) newErrors.cardNumber = "Card number is required";

  else if (!/^\d{12,19}$/.test(form.cardNumber.replace(/D/g, ""))) newErrors.cardNumber
= "Card number must be 12-19 digits";

  if (!form.password.trim()) newErrors.password = "Password is required";

  else if (form.password.length < 6) newErrors.password = "Password must be at least 6
characters";
```

```
    if (!form.monthlyIncome) newErrors.monthlyIncome = "Monthly income is required";  
    else if (parseFloat(form.monthlyIncome) <= 0) newErrors.monthlyIncome = "Monthly  
income must be greater than 0";
```

```
    if (!form.dailyLimit) newErrors.dailyLimit = "Daily spending limit is required";  
    else if (parseFloat(form.dailyLimit) <= 0) newErrors.dailyLimit = "Daily limit must be  
greater than 0";
```

```
    setErrors(newErrors);  
    return Object.keys(newErrors).length === 0;  
};
```

```
const handleComplete = async (e) => {
```

```
    e.preventDefault();
```

```
    if (!validateForm()) {  
        toast.error("Please fix the errors above");  
        return;  
    }
```

```
    setLoading(true);
```

```
    try {
```

```
        const userData = {
```

```
            name: form.name,
```

```
            email: form.email,
```

```
            mobile: form.mobile,
```

```

    card_number: form.cardNumber,

    password: form.password,

    monthly_income: parseFloat(form.monthlyIncome),

    daily_limit: parseFloat(form.dailyLimit),

};

const registeredUser = await authApi.register(userData);
toast.success("Registration successful! Logging you in...");

// automatically log in the user after register
await loginUser(form.email, form.password);
navigate("/dashboard");
} catch (error) {

    console.error("Registration failed:", error);

    const errorMsg = error.response?.data?.detail || error.message || "Registration failed.
Please try again.";

    toast.error(errorMsg);

} finally {

    setLoading(false);

}

};

return (

    <div className="min-h-screen flex items-center justify-center bg-background p-4
relative overflow-hidden">

        <div className="absolute top-1/3 left-1/2 -translate-x-1/2 w-[500px] h-[500px] bg-
primary/8 rounded-full blur-[100px] pointer-events-none" />

```



```

<div className="w-full max-w-lg relative z-10">

  <div className="flex items-center gap-2 md:gap-3 mb-4 md:mb-6">

    <div className="w-8 h-8 md:w-10 md:h-10 rounded-xl bg-primary/20 flex items-center justify-center border border-primary/30">

      <Shield className="w-4 h-4 md:w-5 md:h-5 text-primary" />

    </div>

    <div>

      <h1 className="font-heading text-base md:text-xl font-bold">Register Card Holder</h1>

      <p className="text-xs md:text-sm text-muted-foreground">Create a new credit card holder profile</p>

    </div>

  </div>

  <form onSubmit={handleComplete} className="glass-card rounded-xl p-4 md:p-6 space-y-3 md:space-y-4">

    <div className="space-y-2">

      <Label>Full Name</Label>

      <Input

        value={form.name}

        onChange={(e) => set("name", e.target.value)}

        placeholder="John Doe"

        className={`bg-secondary border-border ${errors.name ? "border-destructive" : ""}}`

      />

      {errors.name && <p className="text-sm text-destructive flex items-center gap-1">
<AlertCircle className="w-3 h-3" />{errors.name}</p>}

```

</div>

<div className="space-y-2">

<Label>Email Address</Label>

<Input

type="email"

value={form.email}

onChange={(e) => set("email", e.target.value)}

placeholder="john@example.com"

className={`bg-secondary border-border \${errors.email ? "border-destructive" : ""}`}

/>

{errors.email && <p className="text-sm text-destructive flex items-center gap-1">
<AlertCircle className="w-3 h-3" />{errors.email}</p>}

</div>

<div className="grid grid-cols-1 md:grid-cols-2 gap-3 md:gap-4">

<div className="space-y-2">

<Label className="text-sm">Mobile Number</Label>

<Input

value={form.mobile}

onChange={(e) => set("mobile", e.target.value)}

placeholder="9876543210"

className={`bg-secondary border-border text-sm \${errors.mobile ? "border-destructive" : ""}`}

/>

```
      {errors.mobile} && <p className="text-xs md:text-sm text-destructive flex items-center gap-1"><AlertCircle className="w-3 h-3" />{errors.mobile}</p>}
```

```
</div>
```

```
<div className="space-y-2">
```

```
  <Label className="text-sm">Credit Card Number</Label>
```

```
  <Input
```

```
    value={form.cardNumber}
```

```
    onChange={(e) => set("cardNumber", e.target.value)}
```

```
    placeholder="1111222233334444"
```

```
    className={`bg-secondary border-border text-sm ${errors.cardNumber ? "border-destructive" : ""}`}`
```

```
  />
```

```
  {errors.cardNumber} && <p className="text-xs md:text-sm text-destructive flex items-center gap-1"><AlertCircle className="w-3 h-3" />{errors.cardNumber}</p>}
```

```
</div>
```

```
</div>
```

```
<div className="space-y-2">
```

```
  <Label>Password</Label>
```

```
  <Input
```

```
    type={showPass ? "text" : "password"}
```

```
    value={form.password}
```

```
    onChange={(e) => set("password", e.target.value)}
```

```
    placeholder="....."
```

```
    className={`bg-secondary border-border ${errors.password ? "border-destructive" : ""}`}`
```

```
  />
```

```
{errors.password} && <p className="text-sm text-destructive flex items-center gap-1"><AlertCircle className="w-3 h-3" />{errors.password}</p>
```

```
</div>
```

```
<div className="grid grid-cols-1 md:grid-cols-2 gap-3 md:gap-4">
```

```
<div className="space-y-2">
```

```
<Label className="text-sm">Monthly Income (₹)</Label>
```

```
<Input
```

```
type="number"
```

```
value={form.monthlyIncome}
```

```
onChange={(e) => set("monthlyIncome", e.target.value)}
```

```
placeholder="500000"
```

```
className={`bg-secondary border-border text-sm ${errors.monthlyIncome ? "border-destructive" : ""}`}`
```

```
/>
```

```
{errors.monthlyIncome} && <p className="text-xs md:text-sm text-destructive flex items-center gap-1"><AlertCircle className="w-3 h-3" />{errors.monthlyIncome}</p>
```

```
</div>
```

```
<div className="space-y-2">
```

```
<Label className="text-sm">Daily Spending Limit (₹)</Label>
```

```
<Input
```

```
type="number"
```

```
value={form.dailyLimit}
```

```
onChange={(e) => set("dailyLimit", e.target.value)}
```

```
placeholder="50000"
```

```
className={`bg-secondary border-border text-sm ${errors.dailyLimit ? "border-destructive" : ""}`}`
```

```
/>
```

```
    {errors.dailyLimit} && <p className="text-xs md:text-sm text-destructive flex items-center gap-1"><AlertCircle className="w-3 h-3" />{errors.dailyLimit}</p>}
```

```
</div>
```

```
</div>
```

```
<div className="flex flex-col-reverse md:flex-row md:justify-between gap-3 mt-4 md:mt-6 pt-4 border-t border-border">
```

```
  <Button type="button" variant="ghost" onClick={() => navigate("/login")} className="gap-1 text-sm md:text-base">
```

```
    <ArrowLeft className="w-4 h-4" /> Back to Login
```

```
</Button>
```

```
  <Button type="submit" disabled={loading} className="gap-1 text-sm md:text-base">
```

```
    {loading ? "Registering..." : "Complete Registration"} <Check className="w-4 h-4" />
```

```
</Button>
```

```
</div>
```

```
</form>
```

```
</div>
```

```
</div>
```

```
);
```

```
};
```

```
export default Register;
```

```
import { describe, it, expect } from "vitest";
```

```
describe("example", () => {  
  it("should pass", () => {  
    expect(true).toBe(true);  
  });  
});
```

File: credit-card-fraud-detection/frontend/src/test/setup.js

```
import "@testing-library/jest-dom";
```

```
Object.defineProperty(window, "matchMedia", {  
  writable: true,  
  value: (query) => ({  
    matches: false,  
    media: query,  
    onchange: null,  
    addListener: () => {},  
    removeListener: () => {},  
    addEventListener: () => {},  
    removeEventListener: () => {},  
    dispatchEvent: () => {},  
  }),
```

```
});
```

File: credit-card-fraud-detection/frontend/tailwind.config.js

```
export default {
  darkMode: ["class"],
  content: [
    "./pages/**/*.{js,jsx}",
    "./components/**/*.{js,jsx}",
    "./app/**/*.{js,jsx}",
    "./src/**/*.{js,jsx}",
  ],
  prefix: "",
  theme: {
    container: {
      center: true,
      padding: "2rem",
      screens: {
        "2xl": "1400px",
      },
    },
    extend: {
      fontFamily: {
        body: ["Inter", "sans-serif"],
        heading: ["Montserrat", "sans-serif"],
      },
    },
  },
}
```

```
},
colors: {
  border: "hsl(var(--border))",
  input: "hsl(var(--input))",
  ring: "hsl(var(--ring))",
  background: "hsl(var(--background))",
  foreground: "hsl(var(--foreground))",
  primary: {
    DEFAULT: "hsl(var(--primary))",
    foreground: "hsl(var(--primary-foreground))",
  },
  secondary: {
    DEFAULT: "hsl(var(--secondary))",
    foreground: "hsl(var(--secondary-foreground))",
  },
  destructive: {
    DEFAULT: "hsl(var(--destructive))",
    foreground: "hsl(var(--destructive-foreground))",
  },
  muted: {
    DEFAULT: "hsl(var(--muted))",
    foreground: "hsl(var(--muted-foreground))",
  },
  accent: {
    DEFAULT: "hsl(var(--accent))",
    foreground: "hsl(var(--accent-foreground))",
```


},

popover: {

 DEFAULT: "hsl(var(--popover))",

 foreground: "hsl(var(--popover-foreground))",

},

card: {

 DEFAULT: "hsl(var(--card))",

 foreground: "hsl(var(--card-foreground))",

},

warning: {

 DEFAULT: "hsl(var(--warning))",

 foreground: "hsl(var(--warning-foreground))",

},

success: {

 DEFAULT: "hsl(var(--success))",

 foreground: "hsl(var(--success-foreground))",

},

sidebar: {

 DEFAULT: "hsl(var(--sidebar-background))",

 foreground: "hsl(var(--sidebar-foreground))",

 primary: "hsl(var(--sidebar-primary))",

 "primary-foreground": "hsl(var(--sidebar-primary-foreground))",

 accent: "hsl(var(--sidebar-accent))",

 "accent-foreground": "hsl(var(--sidebar-accent-foreground))",

 border: "hsl(var(--sidebar-border))",

 ring: "hsl(var(--sidebar-ring))",

```
    },  
  
    },  
  
    borderRadius: {  
      lg: "var(--radius)",  
      md: "calc(var(--radius) - 2px)",  
      sm: "calc(var(--radius) - 4px)",  
    },  
  
    keyframes: {  
      "accordion-down": {  
        from: { height: "0" },  
        to: { height: "var(--radix-accordion-content-height)" },  
      },  
      "accordion-up": {  
        from: { height: "var(--radix-accordion-content-height)" },  
        to: { height: "0" },  
      },  
    },  
  
    animation: {  
      "accordion-down": "accordion-down 0.2s ease-out",  
      "accordion-up": "accordion-up 0.2s ease-out",  
    },  
  },  
  
  animation: {  
    "accordion-down": "accordion-down 0.2s ease-out",  
    "accordion-up": "accordion-up 0.2s ease-out",  
  },  
  
  plugins: [require("tailwindcss-animate")],  
};
```

File: credit-card-fraud-detection/frontend/vite.config.js

```
import { defineConfig } from "vite";

import react from "@vitejs/plugin-react-swc";

import path from "path";

// https://vitejs.dev/config/

export default defineConfig(({ mode }) => ({
  server: {
    host: ":::",
    port: 8081,
    hmr: {
      overlay: false,
    },
  },
  preview: {
    allowedHosts: ['.projectbro.in']
  },
  plugins: [react()],
  resolve: {
    alias: {
      "@": path.resolve(__dirname, "./src"),
    },
  },
}));
```

File: credit-card-fraud-detection/frontend/vitest.config.js

```
import { defineConfig } from "vitest/config";

import react from "@vitejs/plugin-react-swc";

import path from "path";

export default defineConfig({
  plugins: [react()],
  test: {
    environment: "jsdom",
    globals: true,
    setupFiles: ["./src/test/setup.js"],
    include: ["src/**/*.test.js", "src/**/*.spec.js", "src/**/*.jsx"],
  },
  resolve: {
    alias: { "@": path.resolve(__dirname, "./src") },
  },
});
```

17. Testing and Validation

Testing was performed using simulated transactions to evaluate fraud detection capability.

Test scenarios included normal transactions, suspicious transactions, and high-risk fraudulent transactions.

18. Results and Discussion

The Random Forest model successfully classified transactions into Safe, Suspicious, and Fraud categories.

[PLACEHOLDER – RESULTS SCREENSHOTS]

19. Conclusion

The project demonstrates how machine learning can be applied to detect fraudulent credit card transactions. The integration of a machine learning model with a modern web application provides an effective platform for analyzing transaction behavior.

20. Future Scope

- Integration with real banking APIs
- Real-time fraud monitoring systems
- Deep learning based fraud detection models
- Mobile application development

21. References

- [scikit-learn Documentation](#)
- [FastAPI Documentation](#)
- [React Documentation](#)
- Research papers on credit card fraud detection